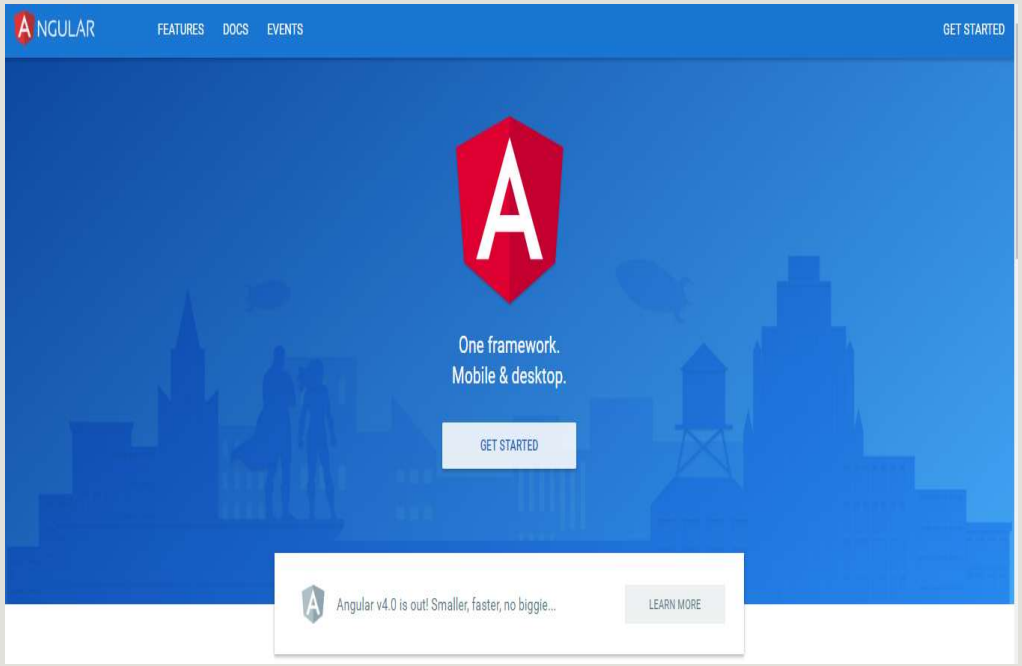
Angular Introduction

AYMEN SELLAOUTI

Références


ANGULAR
FEATURES
DOCS
EVENTS
GET STARTED
One framework.
Mobile & desktop.
GET STARTED
Angular v4.0 is out! Smaller, faster, no biggie...
LEARN MORE


Learn Angular
Gravissez tous les concepts jusqu'au sommet
Maîtriser les concepts du Framework Angular
pour développer des applications robustes
Auteur : William KOZA

Plan du Cours

1. Introduction
2. Les composants
3. Les composants Standalone
4. Le Binding
5. Les signaux
6. Le nouveau control Flow
7. Interaction entre composants
8. Directives
9. Pipes
10. Injection de dépendances
11. Routing
12. Signal Form
13. HTTP
14. Lazy Loading

PRÉ-REQUIS

- HTML 5 / CSS 3 / Bootstrap



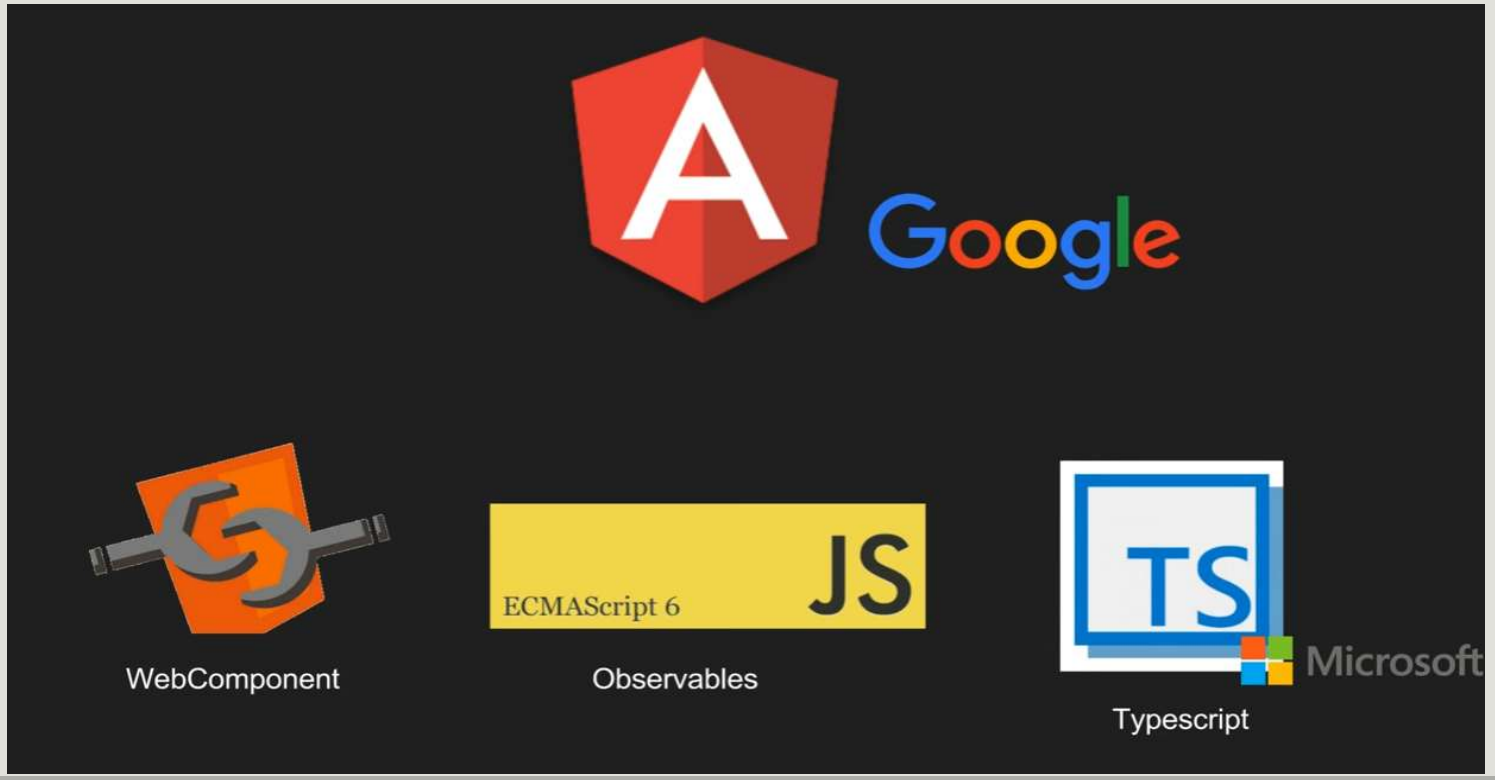
- JavaScript



- Programmation Orientée Objet

POO

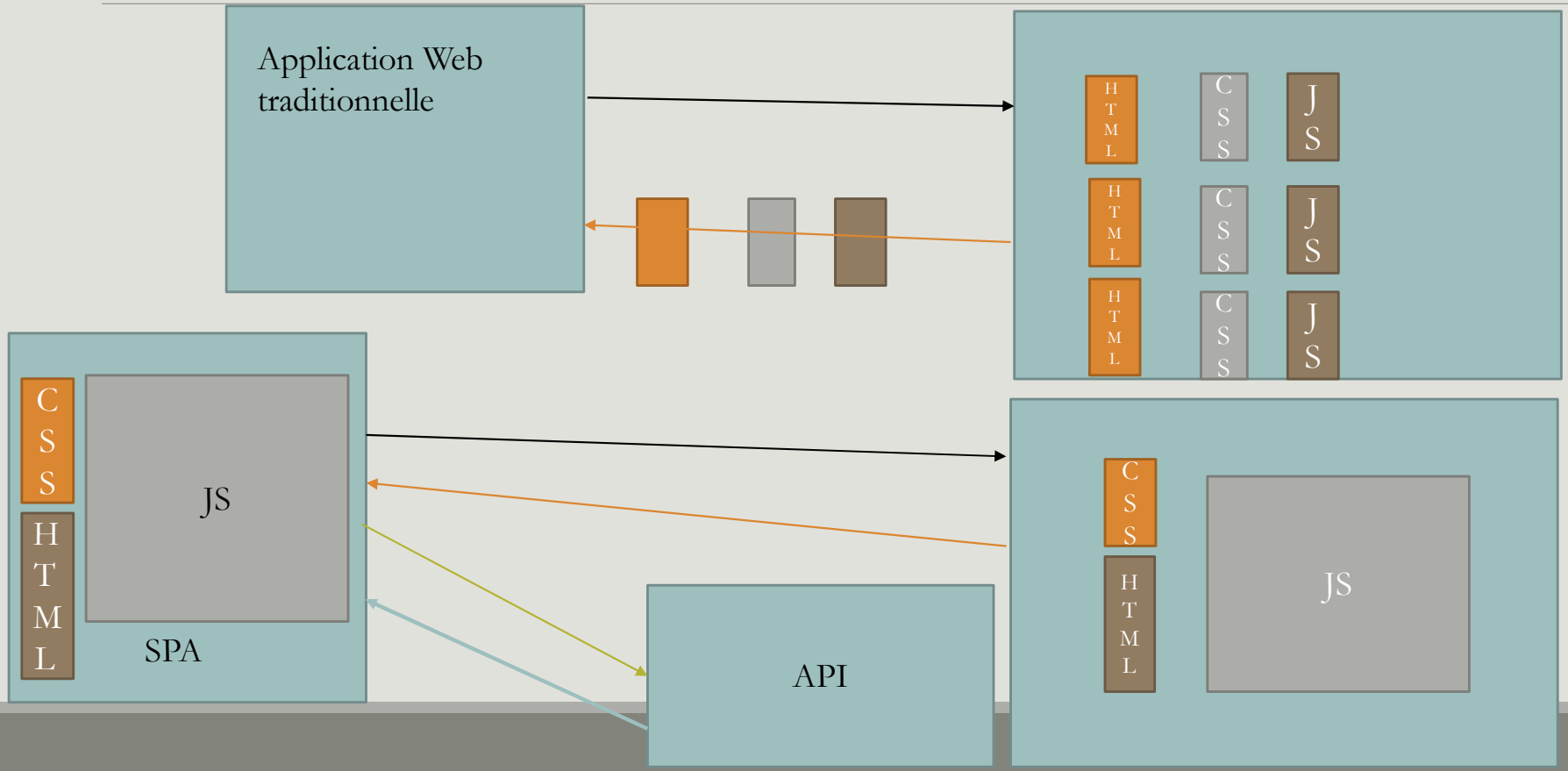
C’est quoi Angular?



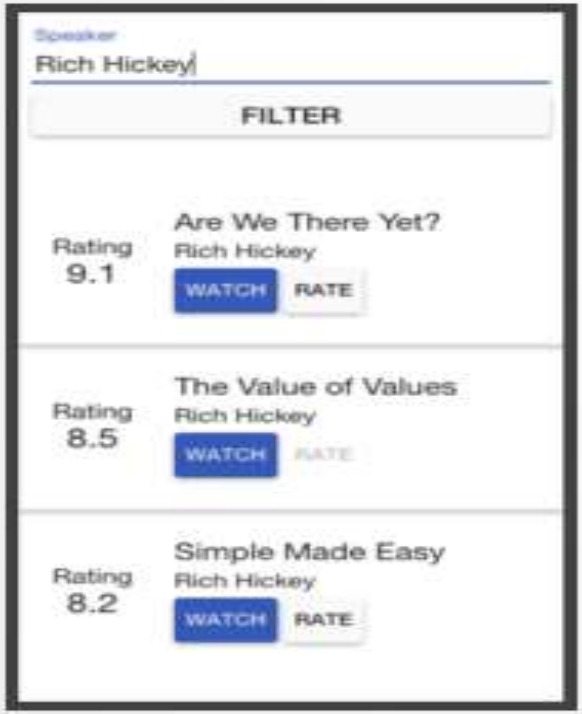
C’est quoi Angular?

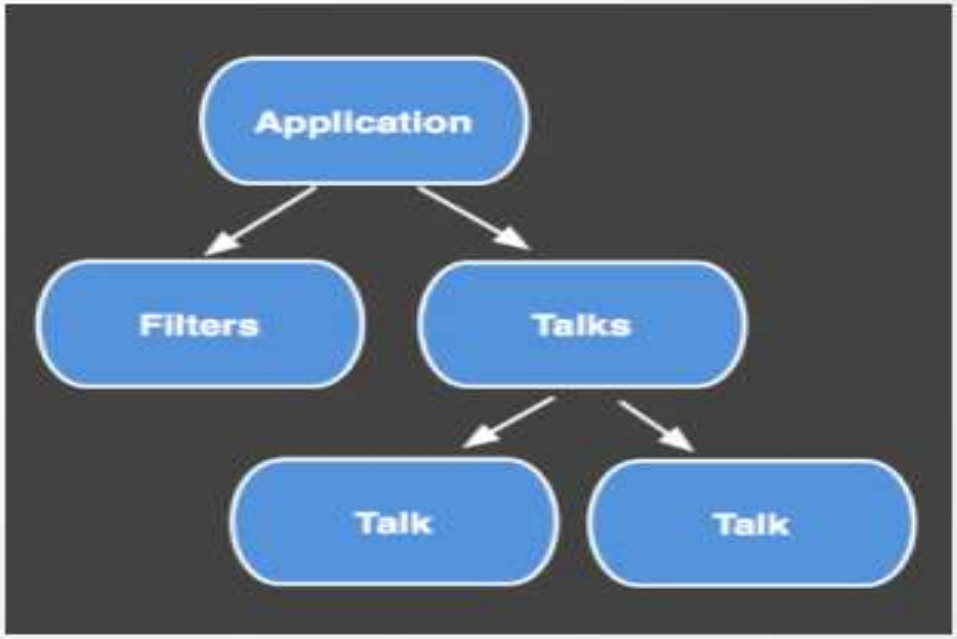
- Framework JS
- SPA
- Supporte plusieurs langages : ES5, EC6, TypeScript, Dart
- Rapide
- Orienté Composant

SPA

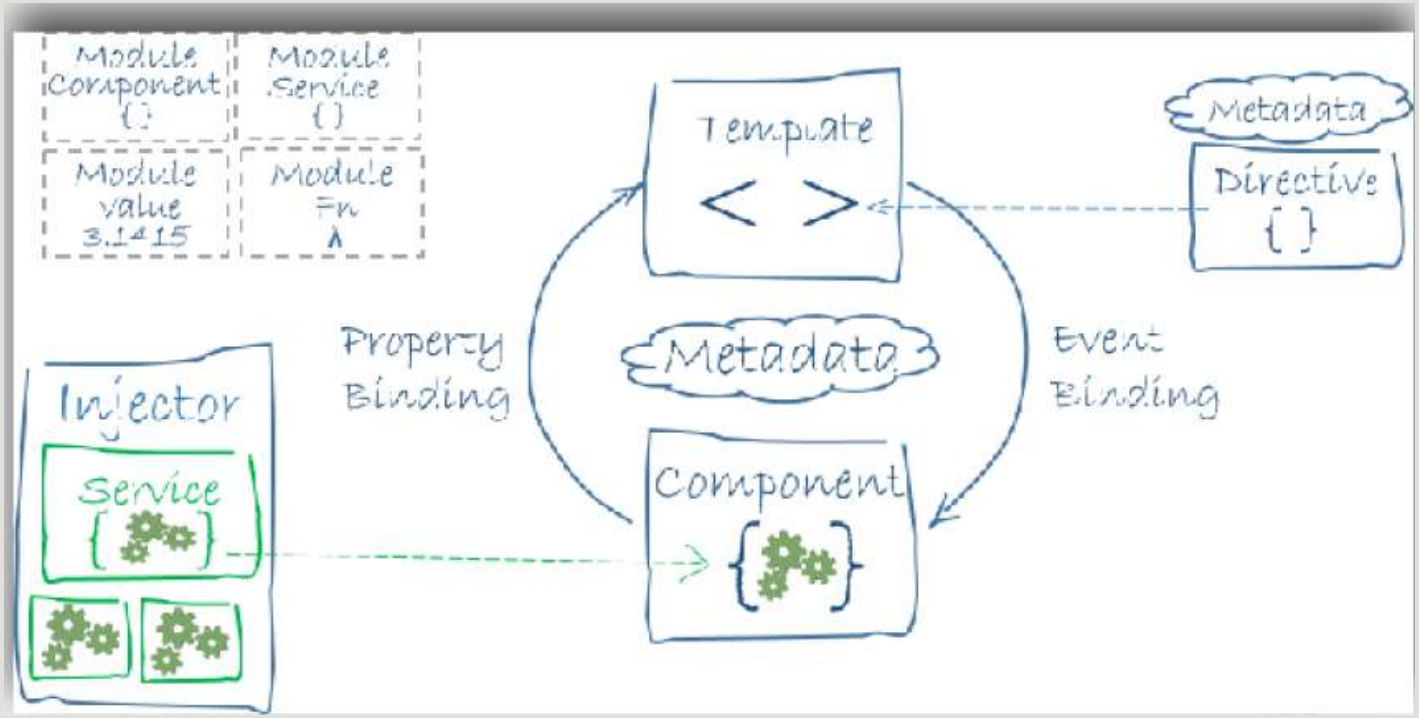

Application Web traditionnelle
HTML
CSS
JS
HTML
CSS
JS
HTML
CSS
JS
CSS
JS
HTML
SPA
CSS
HTML
JS
API

Angular : Arbre de composants





Architecture Angular



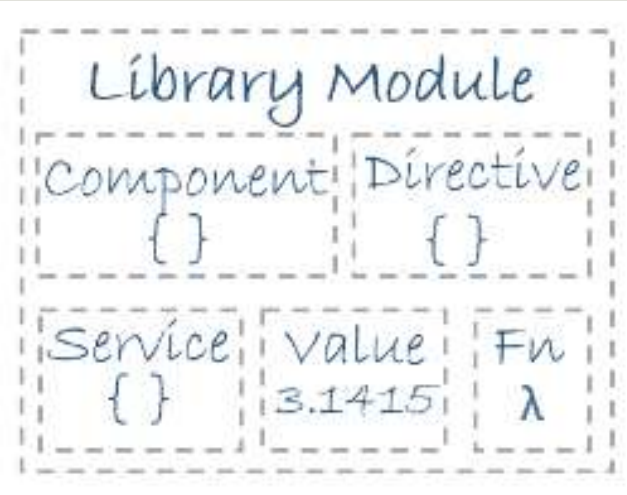
Les librairies d'Angular

- Ensemble de modules JS

- Des librairies qui contiennent un ensemble de fonctionnalités.

- Toutes les librairies d'Angular sont préfixées par @angular

- Récupérable à travers un import JavaScript.



- Exemple pour récupérer l'annotation component : import { Component } from '@angular/core';

Les composants

- Le composant est la partie principale d'Angular.
- Un composant s'occupe d'une partie de la vue.
- L'interaction entre le composant et la vue se fait à travers une API.

Template

- Un Template est le complément du composant.
- C'est la vue associée au composant.
- Elle représente le code HTMl géré par le composant.

Les métas data



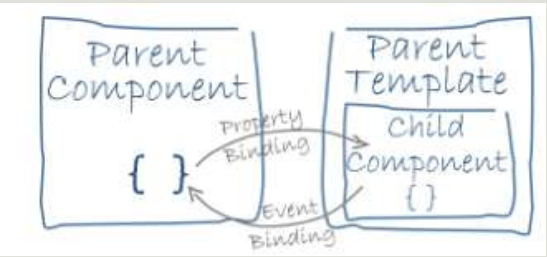
- Appelé aussi « decorator », ce sont des informations permettant de décrire les classes.

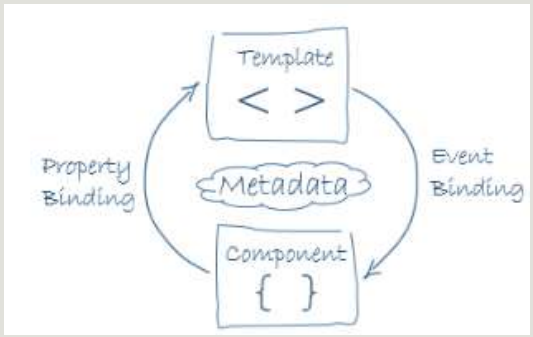
- @Component permet d'identifier la classe comme étant un composant angular.

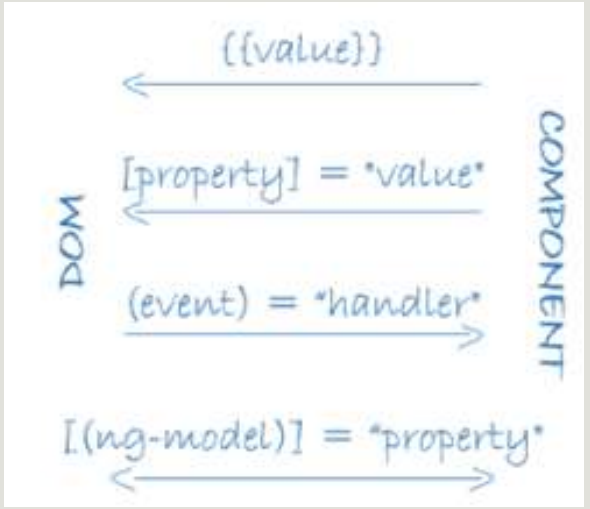
Le Data Binding

- Le data binding est le mécanisme qui permet de mapper des éléments du DOM avec des propriétés et des méthodes du composant.

- Le Data Binding permettra aussi de faire communiquer les composants.







Les directives

- Les directives Angular sont des classes avec la métadata @Directive. Elle permettent de modifier le DOM et de rendre les Template dynamiques.



- Apparaissent dans des éléments HTML comme les attributs.
- Un composant est une directive à laquelle Angular a associé un Template.
- Ils existe deux autres types de directives :
  - Directives structurelles
  - Directive d'attributs

Les services

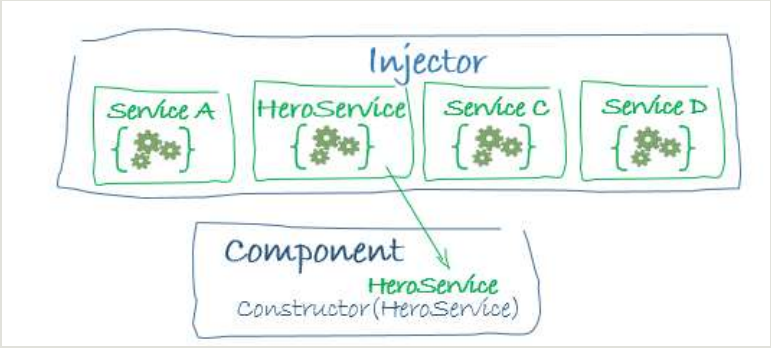
- Classes permettant d'encapsuler des traitements métiers.



- Doivent être légers.

- Associées aux composants et autres classes par injection de dépendances.





Installation d'Angular

Deux méthodes pour installer un projet Angular.

- Cloner ou télécharger le QuickStart seed proposé par Angular.
- Utiliser le Angular-cli afin d'installer un nouveau projet (conseillé).
- Remarque : L'installation de NodeJs est obligatoire afin de pouvoir utiliser son npm (Node Package Manager).

Installation d'Angular QuickStart

Deux méthodes

- Télécharger directement le projet du dépôt Git

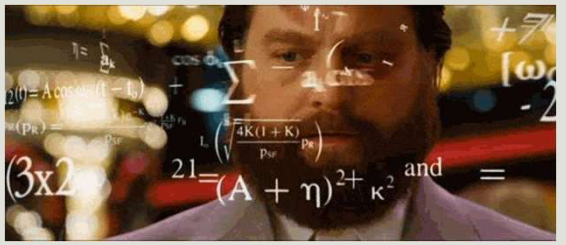
  https://github.com/angular/quickstart

- Ou bien le cloner à l'aide de la commande suivante :

  git clone https://github.com/angular/quickstart.git quickstart

- Se positionner sur le projet
- Installer les dépendances à l'aide de npm : npm install
- lancer le projet à l'aide de npm : npm start

Installation d'Angular
Angular Cli



- Nous allons installer notre première application en utilisant angular Cli.
- Si vous avez Node c'est bon, sinon, installer NodeJs sur votre machine. Vous devez avoir une version de node nécessaire pour la version Angular que vous installez.
- Une fois installé vous disposez de npm qui est le Node Package Manager. Afin de vérifier si vous avez NodeJs installé, tapez npm –v.
- Installer maintenant le Cli en tapant la : npm install -g @angular/cli
  - npm install -g @angular/cli@**20.3.14** installe la version **20.3.14**
  - **npm view @angular/cli** affiche la liste des versions de la cli
- Installer un nouveau projet à l'aide de la commande **ng new nomProjet**
- **npx @angular/cli@19.2.15** new projectName
- Afin d'avoir du help pour le cli tapez **ng help**
- Lancer le projet en utilisant la commande **ng serve**

https://cli.angular.io/

Repo Github du Projet

- Pour notre formation, cloner ce projet dans lequel vous allez trouvez les exemples et corrections des exercices:

git clone https://github.com/aymensellaouti/ngMinDef19126.git

Quelques commandes du Cli

Commande	Utilisation
Component	ng g component my-new-component
Directive	ng g directive my-new-directive
Pipe	ng g pipe my-new-pipe
Service	ng g service my-new-service
Class	ng g class my-new-class
Interface	ng g interface my-new-interface
Module	ng g module my-module

Ajouter Bootstrap

On peut ajouter Bootstrap de plusieurs façons :

1- Via le CDN à ajouter dans le fichier index.html

2- En le téléchargeant du site officiel

3- Via npm

- npm install bootstrap --save

Ajouter Bootstrap

Pour ajouter les dossiers téléchargés on peut le faire de deux façons :

1- En l'ajoutant dans index.html

2- En ajoutant le chemin des dépendances dans les tableaux styles et scripts dans le fichier angular.json:

```
"styles": [
"../node_modules/bootstrap/dist/css/bootstrap.min.css",
"styles.css",
],
"scripts": [
"../node_modules/jquery/dist/jquery.min.js",
"../node_modules/popper.js/dist/umd/popper.min.js",
"../node_modules/bootstrap/dist/js/bootstrap.min.js"
],
```

Ajouter Bootstrap

Ajouter dans le fichier src/style.css un import de vos bibliothèques.

@import "./../node_modules/bootstrap/dist/css/bootstrap.css";

Pour la majorité des bibliothèques, vous avez un fichier **package.json** avec les différents paths, vous permettant de ne pas spécifier le path complet, c'est le cas de bootstrap.

@import **'bootstrap';**

```
"main": "dist/js/bootstrap.js",
"module": "dist/js/bootstrap.esm.js",
"sass": "scss/bootstrap.scss",
"style": "dist/css/bootstrap.css",
```

Essayer la même chose avec font-awesome.

Angular
Les composants

AYMEN SELLAOUTI

Qu'est ce qu'un composant (Component)

- Un **composant** est une **classe** qui permet de **gérer une vue**. Il se charge **uniquement** de cette vue la.

Plus simplement, un composant est un fragment HTML géré par une classe JS (component en angular et controller en angularJS)

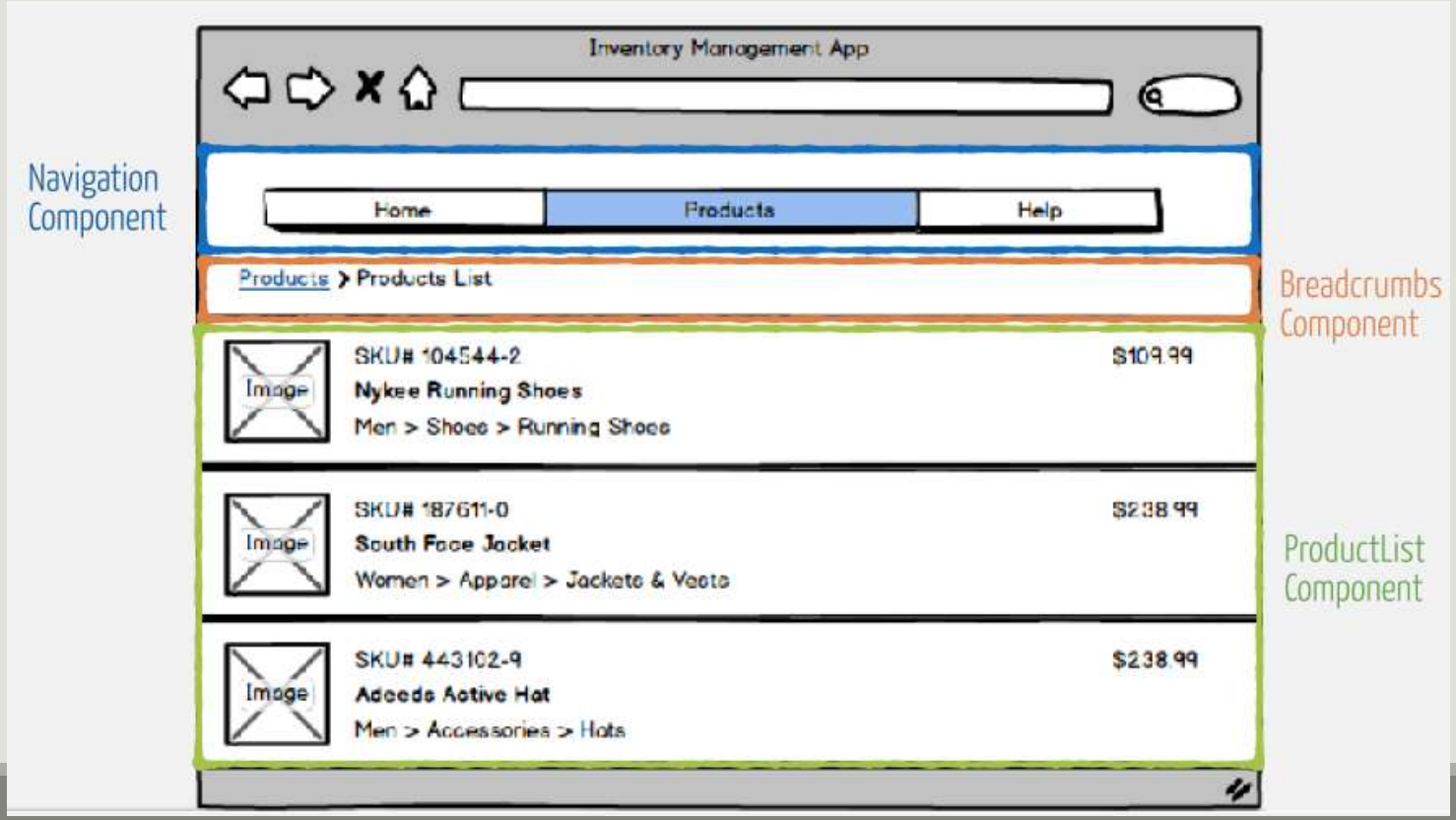
- Une application Angular est un **arbre de composants**

- La racine de cet arbre est l'application lancée par le navigateur au lancement.

- Un composant est :
  - Composable (normal c'est un composant)
  - Réutilisable
  - Hiérarchique (n'oublier pas c'est un arbre)

NB : Dans le reste du cours les mots composant et component représentent toujours un composant Angular.

Quelques exemples



Quelques exemples



Quelques exemples



Premier Composant

```
@Component({
  selector: 'app-standalone',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './standalone.component.html',
  styleUrls: ['./standalone.component.css']
})
export class StandaloneComponent {}
```

Chargement de la classe Component

Le décorateur @Component permet d'ajouter un comportement à notre classe et de spécifier que c'est un Composant Angular.

selector permet de spécifier le tag (nom de la balise) associé ce composant

templateUrl: spécifie l'url du template associé au composant

styleUrls: tableau des feuilles de styles associé à ce composant

Import: permet d'importer toutes les dépendances du composant

Export de la classe afin de pouvoir l'utiliser

Création d'un composant

- Deux méthodes pour créer un composant
  - Manuelle
  - Avec le Cli
- Manuelle
  - Créer la classe
  - Importer Component
  - Ajouter l'annotation et l'objet qui la décore
  - **Si l'application est modulaire,** ajouter le composant dans le **AppModule(app.module.ts)** dans l'attribut **declarations**
- Cli
  - Avec la commande ng **g**enerate component my-new-component ou son raccourci ng g c my-new-component

Création d'un composant

- La commande generate possède plusieurs options

OPTION	DESCRIPTION
--inlineStyle=true|false	Inclus les styles css dans le composant Aliases: -s
--inlineTemplate=true|false	Inclus le template dans le composant Aliases: -t
--prefix=*prefix*	Le préfixe à appliquer pour la génération des composants Valeur par défaut: app Aliases: -p

Angular
Les standalones composants

AYMEN SELLAOUTI

NgModules

- Voyons ensemble un ancien code d'angular avec les modules

https://www.youtube.com/watch?v=DA3efofhpq4&ab_channel=ngVikings

Pourquoi les NgModules

- La principale raison de l'introduction initiale de NgModules était pragmatique : **regrouper des blocs de construction qui sont utilisés ensemble**.
- Ceci permet non seulement d'augmenter la commodité pour les développeurs, mais également pour **le compilateur Angular**.
- En effet le NgModule permet de **définir le contexte de compilation**. A partir de ce contexte, le compilateur apprend où le programme est autorisé à appeler quels composants.
- Les deux parties **imports et declarations** permettent de définir ce contexte de compilation.

https://www.youtube.com/watch?v=DA3efofhpq4&ab_channel=ngVikings

Pourquoi les NgModules

```
@NgModule({
  declarations: [
    AppComponent,
    ColorComponent,
    RainbowDirective,
    Btc2usdPipe,   ],
  imports: [
    BrowserModule,
    FormsModule,
],
  providers: [
    AuthInterceptorProvider,
    UUID_PROVIDER,
  ],
  bootstrap: [AppComponent],
})
export class AppModule {}
```

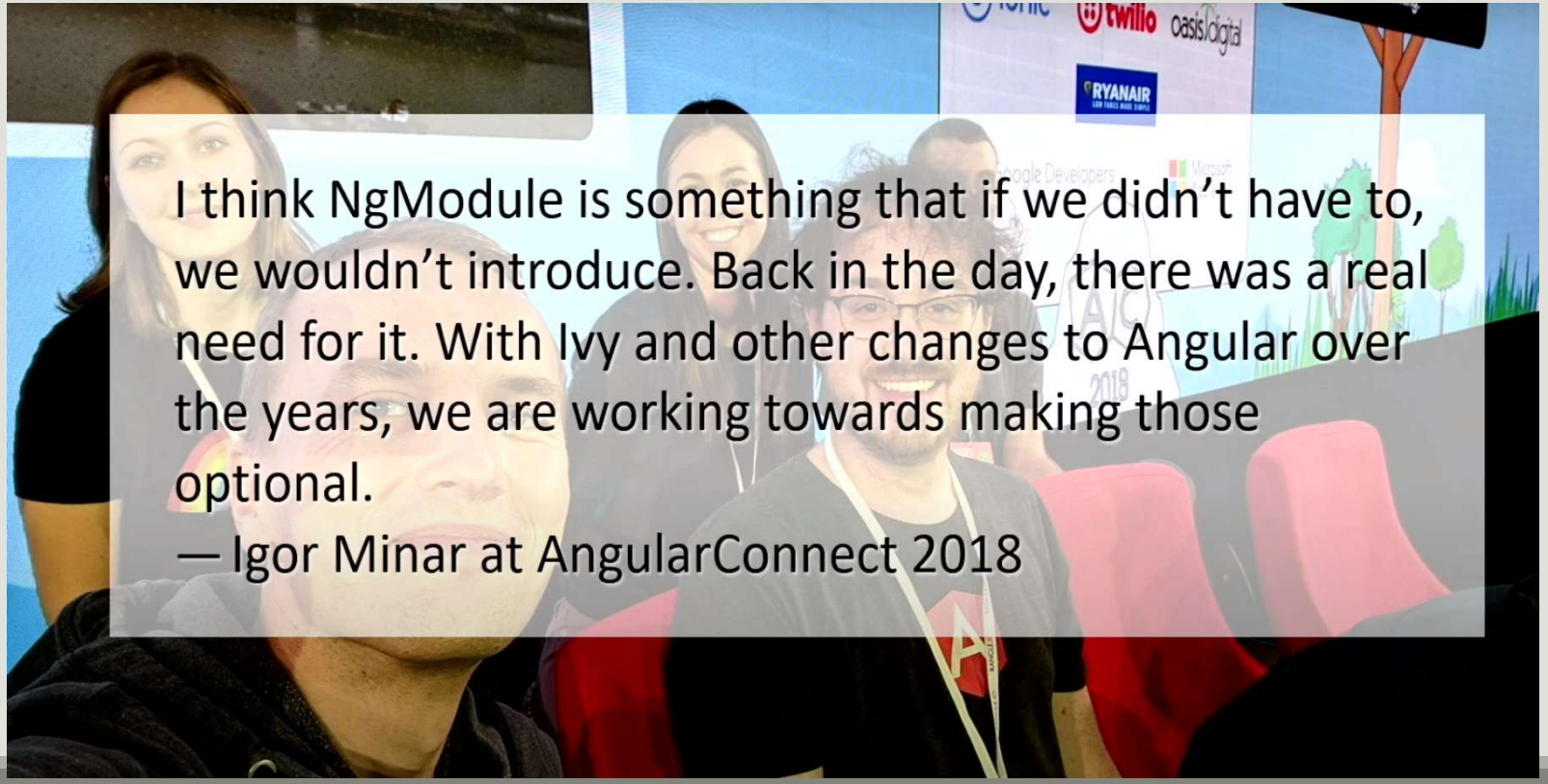
Contexte de compilation

Pourquoi les standalone Components ?

- Le premier problème qui s'est posé est **l'ambiguïté** que pose ce **deuxième système de module en parallèle à celui de EcmaScript.**
- Ceci a **compliqué l'apprentissage** pour les nouveaux apprenants.
- Ceci a poussé la Team Angular à faire en sorte qu'avec Ivy, le nouveau compilateur d'Angular, **on ait un contexte de compilation par composant**.
- C'est ce qui a permis la naissance des standalone component.

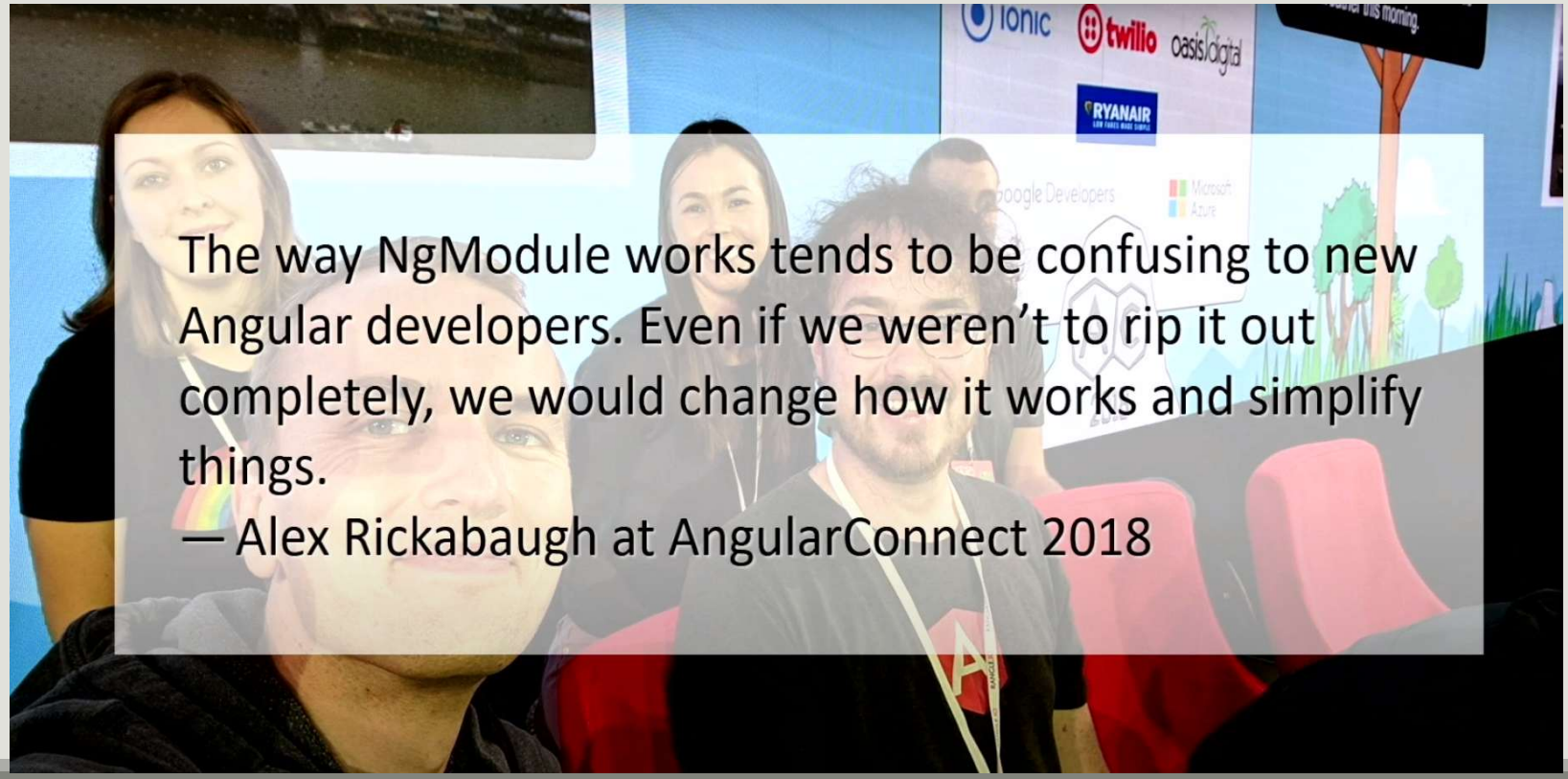
https://www.youtube.com/watch?v=DA3efofhpq4&ab_channel=ngVikings

Pourquoi les standalone Components ?



https://www.youtube.com/watch?v=DA3efofhpq4&ab_channel=ngVikings

Pourquoi les standalone Components ?
Single **C**omponent **A**ngular **M**odule
SCAM



https://www.youtube.com/watch?v=DA3efofhpq4&ab_channel=ngVikings

Pourquoi les standalone Components ?
Single **C**omponent **A**ngular **M**odule
SCAM

- La deuxième problématique est la **granularité de lazy loading**.
- La question que beaucoup d'équipes se sont posée est : Pourquoi devons nous faire du **lazy loading pour tout un module quand nous avons besoin de juste un composant ?**
- C'est la ou le pattern **SCAM (Single Component Angular Module)** a été introduit, c'est l'ancétre du Standalone Component

https://www.youtube.com/watch?v=DA3efofhpq4&ab_channel=ngVikings

Standalone Component
Les composants autonomes

- Un **standalone component** est un **composant indépendant d'un module**.
- Les composants standalone offrent un **moyen simplifié** de créer des applications Angular.
- Les applications existantes peuvent éventuellement et progressivement adopter le nouveau style autonome sans aucune modification majeure.
- Ceci est aussi **applicable** aux **directives** et aux **pipes**.
- L'utilisation d'une telle vision **simplifie l'apprentissage d'Angular**
- Elle **facilite aussi plusieurs aspects** comme le **lazy loading**

Standalone Component
Les composants autonomes

- Une **nouvelle propriété a été introduite** au niveau de l'objet de configuration du composant, c'est la propriété **standalone**.
- Si elle est à **true**, l'élément est **considéré standalone.**
- **A partir d'angular 19, la valeur par défaut est true, donc plus besoin de la mettre**
- Ces éléments **ne doivent plus être associés** à un **tableau de déclaration** d'un NgModule.
- Une nouvelle clé **import** permet aussi **d'importer les dépendances nécessaires**, que ce soit d'autres standalone components ou d'autres NgModule.
- Pour faire simple un composant Standalone est un **composant autonome**, avant c'est sa **famille** (Le NgModule auquel il appartient) qui été **en charge de ses dépendances**(imports et declarations). **Maintenant, c'est à lui de le faire**

Standalone Component
Les composants autonomes

- Si vous avez travaillé avec une **version antérieure à Angular18**. Afin de créer un *standalone* component, procéder comme vous le faite d'habitude avec un composant et ajouter l'option **–standalone (à partir d'Angular 18 c'est devenu le comportement par défaut).**

```
@Component({
  selector: 'app-standalone',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './standalone.component.html',
  styleUrls: ['./standalone.component.css']
})
export class StandaloneComponent {}
```

Standalone Component
Les composants autonomes

- Les **imports définissent le contexte de compilation du composant** : tous les autres blocs de construction que les composants autonomes sont autorisés à utiliser.
- Elles permettent donc **d'importer d'autres composants autonomes, mais aussi des NgModules existants**.
- Ceci rend le **composant autonome et augmente ainsi sa réutilisabilité**.
- Elle nous oblige également à réfléchir aux dépendances du composant.
- **Cette tâche s'avère extrêmement monotone et chronophage**. Ce qu'on faisant une fois dans le module on le fera pour chaque composant.
- Les ides essayent de remédier à ca via les autocompletes

```
@Component({
 selector: 'app-standalone',
 standalone: true,
 imports: [
            CommonModule,
            AnotherStandaloneComponent
 ],
 templateUrl: './standalone.component.html',
 styleUrls: ['./standalone.component.css']
})
export class StandaloneComponent {}
```

Standalone Component
Le modèle mental

- Même si **ce n'est pas géré de la même manière par le compilateur d'Angular**, vous pouvez vois un **standalone component comme un NgModule avec un seul composant**

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  imports: [
    RouterOutlet,
    HighlightDirective,
    CardComponent
  ],
  standalone: true,
})
export class AppComponent {
```

```
@NgModule({
    declarations: [AppComponent],
    imports: [
        BrowserModule,
        AppRoutingModule,
        FormsModule,
    ],
})
export class AppModule {}
```

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
})
export class AppComponent {}
```

Standalone Component
Les composants autonomes
Utilisation dans les NgModule

- Si vous voulez utiliser un **Standalone** Component **dans un NgModule**, **importer** le **comme** vous importer **un module**.
- Avec cette manière de faire, un **composant standalone peut être importé dans plusieurs modules**, alors qu'avant **un composant ne pouvait être déclaré que dans un seul module**.

```
@NgModule({
  declarations: [],
    imports: [
        BrowserModule,
        AppRoutingModule,
        FormsModule,
        HttpClientModule,
        StandaloneComponent
    ],
  providers: [TestService],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Standalone Component
Bootstraper une application avec un Standalone Component

- Afin de **bootstraper** votre application avec un **standalone component**, utilisez la fonction **bootstrapApplication** dans **main.ts**, à la place de **platformBrowserDynamic().bootstrapModule**, en lui passant le **Standalone Component que vous voulez déclencher au lancement de l'application.**
- Dans **index.html** appelez **ce Standalone Component.**

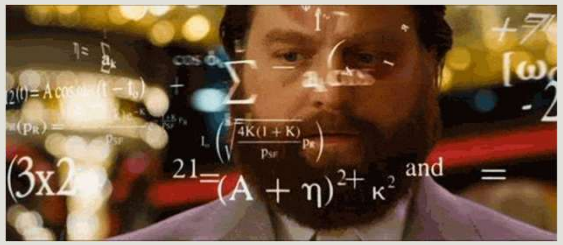
```
bootstrapApplication(StandaloneComponent);
```

main.ts

```
<body>
<div class="container">
  <app-standalone></app-standalone>
</div>
</body>
</html>
```

index.html

Migration vers les standalones



Angular offre un moyen **de migrer automatiquement une partie** de votre application Angular modulaire (15.2.0) et plus vers une application standalone.

- Vous pouvez suivre les différentes étapes du Guide qui présente des parties automatiques et d'autres manuelles.
- Exécutez la migration **dans l'ordre indiqué** ci-dessous, **en vérifiant que votre code est généré et exécuté entre chaque étape** :

1. Exécutez ng g @angular/core:standalone et sélectionnez "Convert all components, directives and pipes to standalone"
2. Exécutez ng g @angular/core:standalone et sélectionnez "Remove unnecessary NgModule classes" , **cette étape risque de garder plusieurs modules.**
3. Exécutez ng g @angular/core:standalone et sélectionnez "Bootstrap the project using standalone APIs".

Même si cet outil est très efficace, il vous reste quelques modules et factorisation à gérer manuellement

https://angular.io/guide/standalone-migration

Que contient le composant ?

- Le composant dispose de deux parties :
  - La partie HTML qui représente la Vue
  - La partie TS décrivant l'état et le comportement du composant

L'état et le comportement d'un composant

```ts
export class FirstComponent {
  //Propriétés : état State
  name = 'First Component';
  isHidden = false;
  message = '';
  //Méthodes : comportement Behaviour
  showHide() {
    this.isHidden = !this.isHidden;
  }
  changeMessage(newMessage: string) {
    this.message = newMessage;
  }
}
```

Classe

```html
<!-- attributs: l'état de la balise -->
<p hidden> First Component works!</p>
<div class="alert alert-info">
  Je test le binding
</div>
<p>Le contenu de l'input est : </p>
<input
  type="text"
  class="form-control"
>
<img src="assets/images/as.jpg" alt="aymen">
```

Template



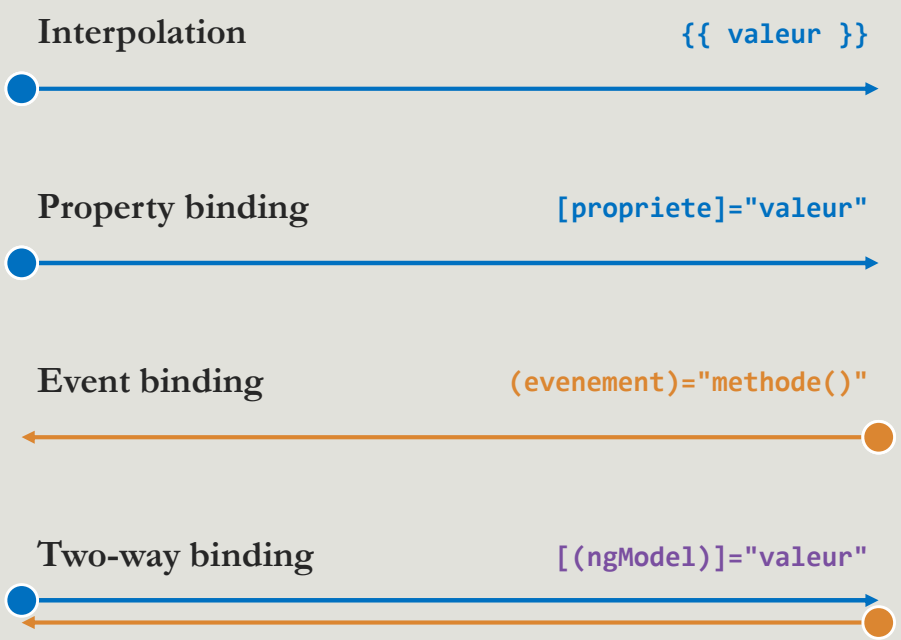
- **Et si on voulait afficher quelque chose dans notre template ?**
- **Et si on voulait contrôler un attribut de notre template ?**
- **Et si on voulait réagir à un événement qui survient dans notre template ?**

Data binding : le sens de l'information

Composant

fichier .ts

état : propriétés et signaux
comportement : méthodes



Vue

template .html

texte affiché
propriétés du DOM (src, disabled…)
événements (click, input…)

Interpolation et property binding

composant.ts

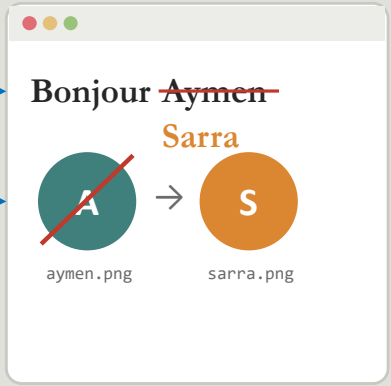
```
export class ProfilComponent {
  nom = signal('Aymen')
  photo = signal('aymen.png')

  changer() {
    this.nom.set('Sarra')
    this.photo.set('sarra.png')
  }
}
```

template.html

```
<h2>Bonjour {{ nom() }}</h2>
<img [src]="photo()">
```

navigateur



Un seul sens : le composant écrit, la vue affiche. `{{ }}` produit du texte ; `[ ]` vise une propriété du DOM (src, disabled, value…), pas un attribut HTML. Un signal se lit avec ses parenthèses : `nom()`, `photo()`.

Afficher des des propriétés dans le Template Interpolation

- L'interpolation permet de projeter des valeurs de propriétés dans votre template.
- Angular utilise la syntaxe "double accolades {{ }} " pour l'interpolation

```
export class FirstComponent {
  //Propriétés : état State

  name = 'First Component';
  isHidden = false;
  message = '';
  //Méthodes : comportement Behaviour
  showHide() {
    this.isHidden = !this.isHidden;
  }
  changeMessage(newMessage: string) {
    this.message = newMessage;
  }
}
```

```
<!-- attributs: l'état de la balise -->
<p hidden> First Component works!</p>
<p hidden> {{name}} works!</p>
<div class="alert alert-info">
  Je test le binding
</div>
<p>Le contenu de l'input est : </p>
<input
  type="text"
  class="form-control"
>
<img src="assets/images/as.jpg" alt="aymen">
```



Contrôler un attribut de notre template Property Binding

- Afin de **contrôler un attribut d'une des balises de notre Template**, angular nous fournit le concept de **Binding de propriété** (**Property Binding**).
- C'est un Binding unidirectionnel.
- La propriété liée au composant est interprétée avant d'être ajoutée au Template.
- syntaxe: [propriété]="**varOuCte**"

```
<div [style.backgroundColor]="color">
  Color
</div>
```

Contrôler un attribut de notre template Property Binding

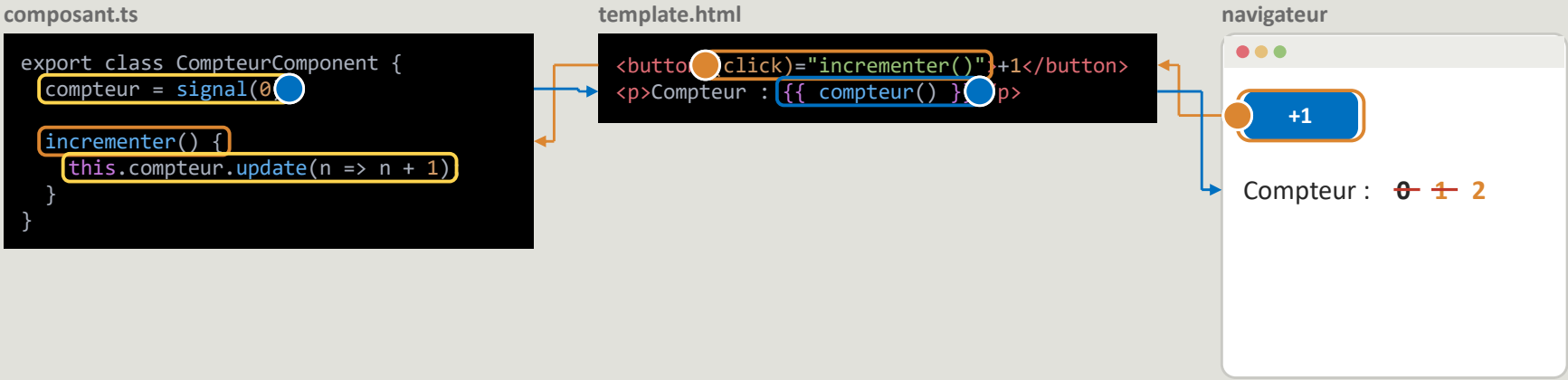
```ts
export class FirstComponent {
  //Propriétés : état State
  name = 'First Component';
  isHidden = false;
  message = '';
  imagePath = 'assets/images/as.jpg';
  //Méthodes : comportement Behaviour
  showHide() {
    this.isHidden = !this.isHidden;
  }
  changeMessage(newMessage: string) {
    this.message = newMessage;
  }
}
```

```html
<!-- attributs: l'état de la balise -->
<p hidden> {{name}} works!</p>
<p [hidden]="isHidden"> {{name}} works!</p>

<div class="alert alert-info">
  Je test le binding
</div>
<p>Le contenu de l'input est : {{ message}}</p>
<input
  type="text"
  class="form-control"
>
<img src="assets/images/as.jpg" alt="aymen">
<img [src]="imagePath" alt="aymen">
```



Event binding : de la vue au composant



La boucle complète : ( ) remonte l'événement vers une méthode, la méthode change l'état, {{ }} redescend la nouvelle valeur. On ne touche jamais au DOM soi-même : on change l'état, la vue suit.

Réagir à un événement qui survient dans notre template Event Binding

- Afin **d'écouter et de réagir** à **un événement déclenché dans notre Template**, angular nous fournit le concept de **Binding d'événement** (**Event Binding**).
- C'est un **binding unidirectionnel**, Il permet d'interagir du DOM vers le composant. L'interaction se fait à travers les **événements**.
- Syntaxe : **(evenement)**="**methodeAExecuter**()">

```
<a (click)="goToCv()" >Go to Cv</a>
```

Réagir à un événement qui survient dans notre template Event Binding

```ts
export class FirstComponent {
  //Propriétés : état State
  name = 'First Component';
  isHidden = false;
  message = '';
  //Méthodes : comportement Behaviour
  showHide() {
    this.isHidden = !this.isHidden;
  }
  changeMessage(newMessage: string) {
    this.message = newMessage;
  }
}
```

```html
<!-- attributs: l'état de la balise -->
<p [hidden]="isHidden"> {{name}} works!</p>
<!-- Au click appelle la fonction showHide -->
<div (click)="showHide()" class="alert alert-info">
                    Je test le binding
</div>
<p>Le contenu de l'input est : {{ message}}</p>
<input
  type="text"
  class="form-control"
>
```

TEMPLATE REFERENCE

- Dans certains cas d'utilisation, vous avez besoin de récupérer **la référence d'un objet du DOM dans votre template**. Par exemple la référence d'un input pour accéder à sa valeur.
- Pour ce faire, on utilise le symbole # pour la création d'une variable de référence.

```typescript
export class FirstComponent {
  //Propriétés : état State
  name = 'First Component';
  isHidden = false;
  message = '';
  //Méthodes : comportement Behaviour
  showHide() {
    this.isHidden = !this.isHidden;
  }
  changeMessage(newMessage: string) {
    this.message = newMessage;
  }
}
```

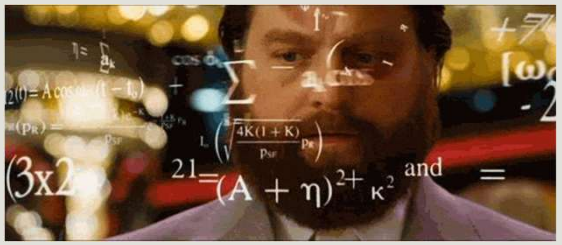
Classe

```html
<!-- attributs: l'état de la balise -->
<p hidden> First Component works!</p>
<div class="alert alert-info">
  Je test le binding
</div>
<p>Le contenu de l'input est : </p>
<input
  #myInput
  (change)="changeMessage(myInput.value)"
  type="text"
  class="form-control"
>
<img src="assets/images/as.jpg" alt="aymen">
```

Template



Exercice



- Créer un nouveau composant. Ajouter y un Div et un input de type texte.
- Fait en sorte que lorsqu'on écrive une couleur dans l'input, ca devienne la couleur du Div.
- Ajouter un bouton. Au click sur le bouton, il faudra que le Div reprenne sa couleur par défaut.
- Ps : pour accéder au contenu d'un élément du Dom utiliser #nom dans la balise et accéder ensuite à cette propriété via le nom.
- Pour accéder à une propriété de style d'un élément on peut binder la propriété **`[style.nomPropriété]`** exemple **`[style.backgroundColor]`**



Two-way binding : [( )] = [ ] + ( )

composant.ts

```
export class RechercheComponent {
  terme = signal('')

  effacer() {
    this.terme.set('')
  }
}
```

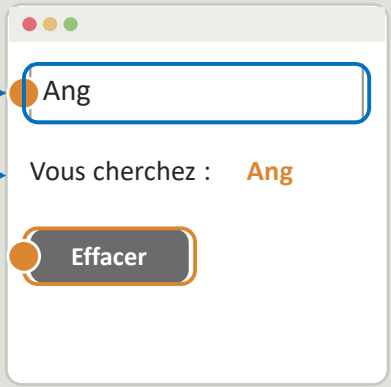
terme() = ~~''~~ ~~'Ang'~~ ''

template.html

```
<input [(ngModel)]="terme">
<!-- équivaut à : -->
<input [ngModel]="terme()"
 (ngModelChange)="terme.set($event)">

<p>Vous cherchez : {{ terme() }}</p>
<button (click)="effacer()">Effacer</button>
```

navigateur

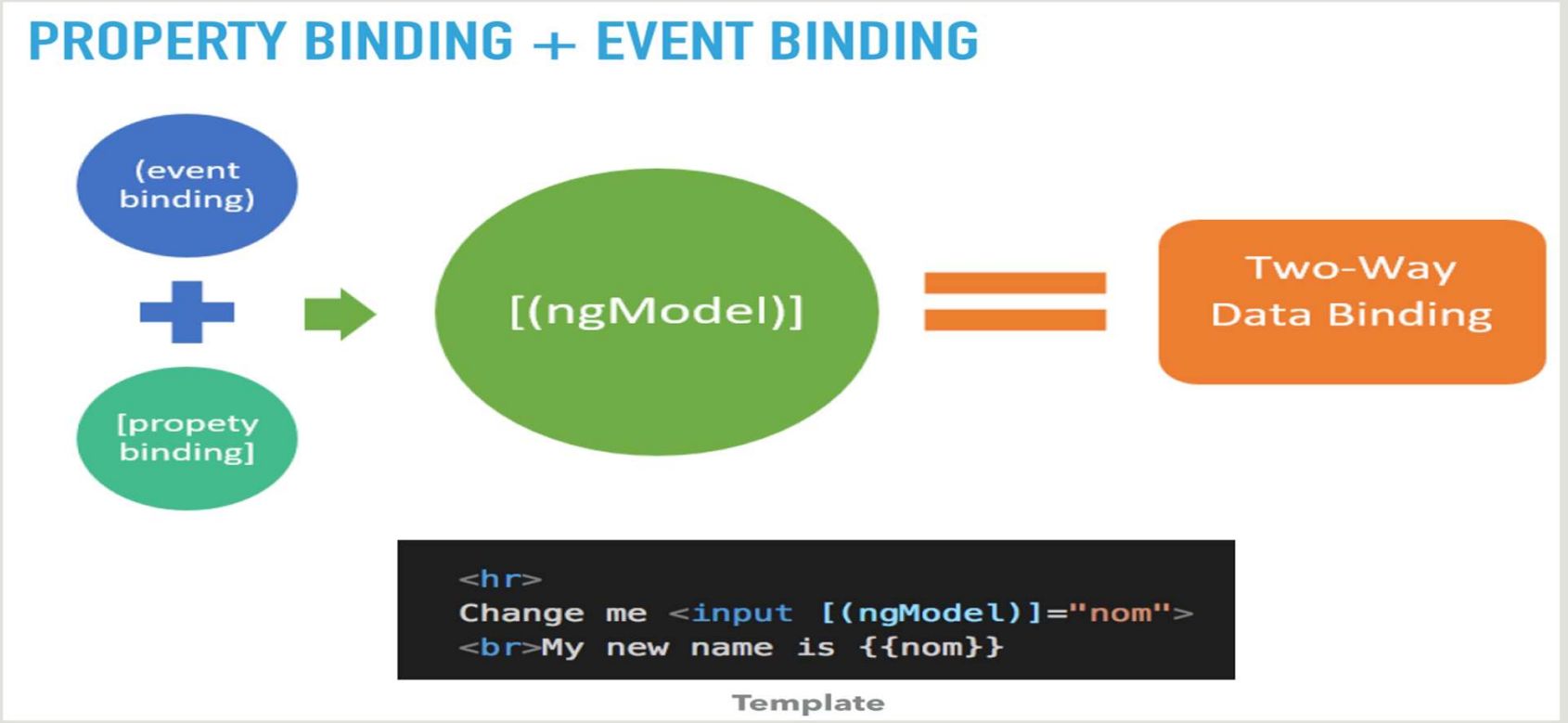


`[(x)]` est un raccourci : la valeur descend par `[x]` et remonte par `(xChange)`. Avec un signal, on lie le signal lui-même : `[(ngModel)]="terme"`, sans parenthèses. Pour un composant maison, `model()` fournit les deux.

Two way Binding

- Binding Bi-directionnel
- Permet d'interagir du Dom vers le composant et du composant vers le DOM.
- Se fait à travers la directive **ngModel** (on reviendra sur le concept de directive plus en détail)
- Syntaxe :
  - **[(ngModel)]=property**
  - Afin de pouvoir utiliser ngModel vous devez importer le FormsModule dans app.module.ts

Two way Binding



Property Binding et Event Binding

```
import { Component} from '@angular/core';

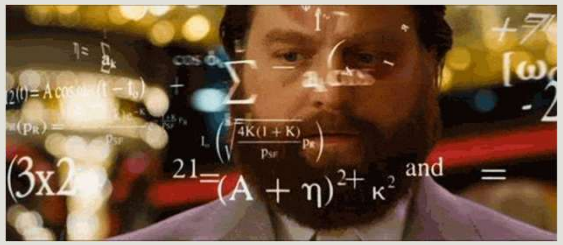
@Component({
  selector: 'app-two-way',
  templateUrl: './two-way.component.html',
  styleUrls: ['./two-way.component.css']
})
export class TwoWayComponent  {
  two:string="myInitValue";
}
```

Component

```
<hr>
Change me if u can
<input
[(ngModel)]="two">
<br>
it's always me :d
{{two}}
```

Template

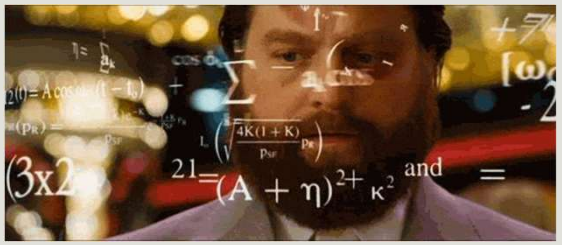
Exercice

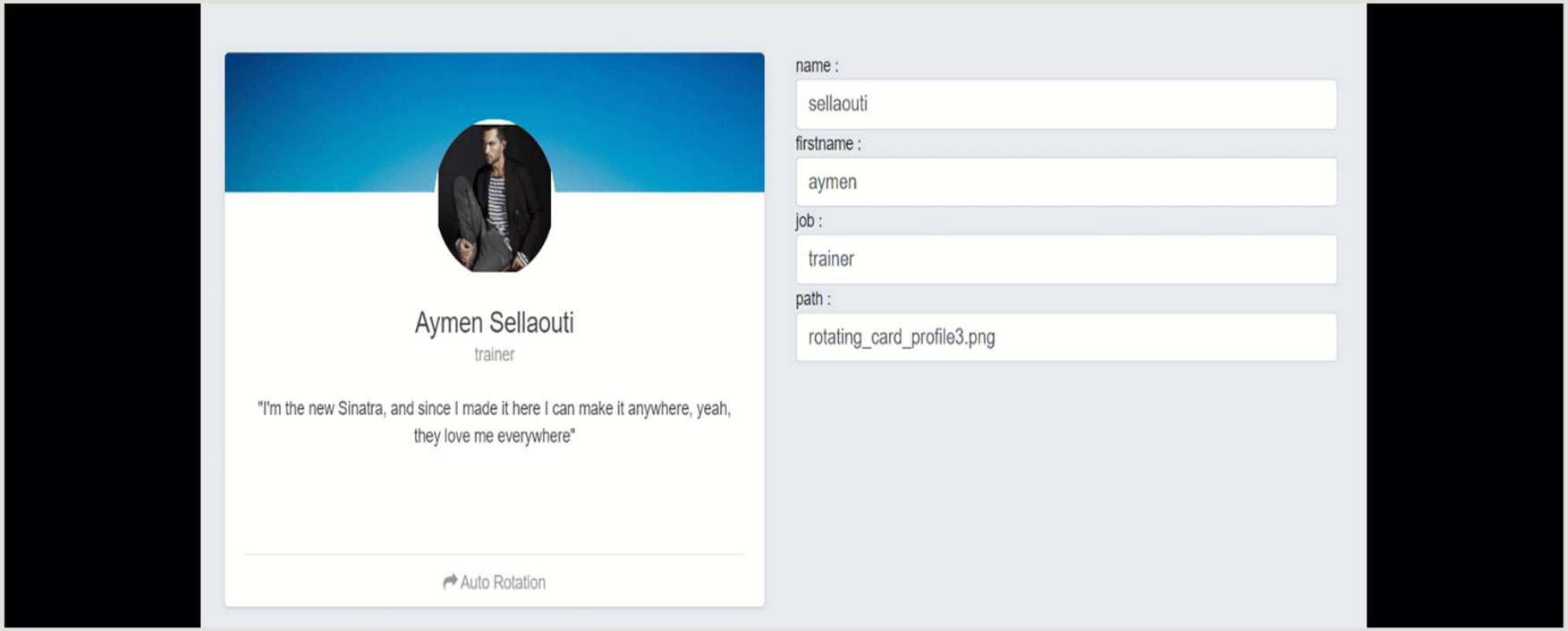


- Le but de cet exercice est de créer un aperçu de carte visite.
- Créer un composant
- Préparer une interface permettant de saisir d'un coté les données à insérer dans une carte visite. De l'autre coté et instantanément les données de la carte seront mis à jour.
- Préparer l'affichage de la carte visite. Vous pouvez utiliser ce thème gratuit :

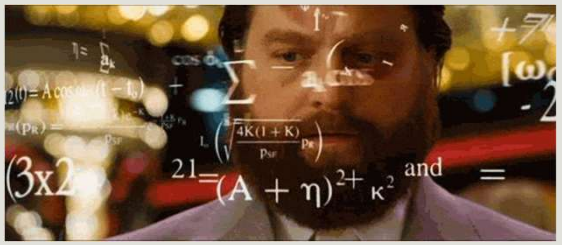
https://www.creative-tim.com/product/rotating-css-card

Exercice




Aymen Sellaouti
trainer
"I'm the new Sinatra, and since I made it here I can make it anywhere, yeah, they love me everywhere"
Auto Rotation
name :
sellaouti
firstname :
aymen
job :
trainer
path :
rotating_card_profile3.png

Exercice



Two Way Binding



Nom :
Sellaouti

Prénom :
aymen

Job :
Enseignant

image :
as.jpg

Citation Favorite :
tant qu'il y a de la vie il y a de l'espoir

Décrivez nous votre travail :
J enseigne aux étudiants les technos du Web

Mots clé de votre travail :
HTML CSS JS PHP Symfony Angular

Two Way Binding



Nom :
Sellaouti

Prénom :
aymen

Job :
Enseignant

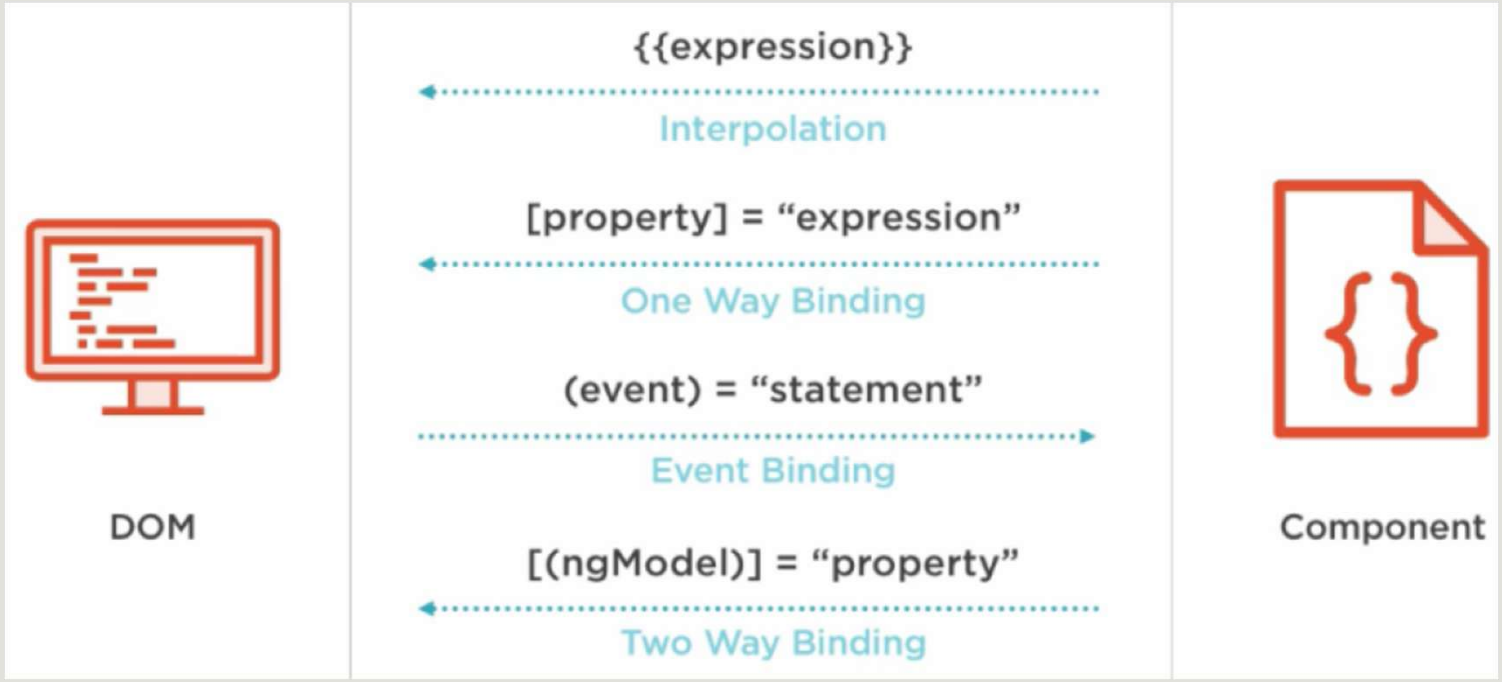
image :
as.jpg

Citation Favorite :
tant qu'il y a de la vie il y a de l'espoir

Décrivez nous votre travail :
J enseigne aux étudiants les technos du Web

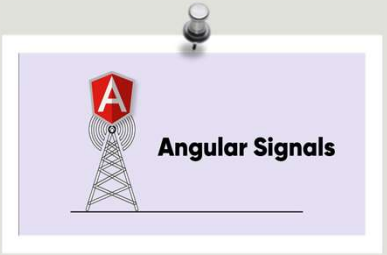
Mots clé de votre travail :
HTML CSS JS PHP Symfony Angular

Résumé : Property Binding



Les Signaux

AYMEN SELLAOUTI

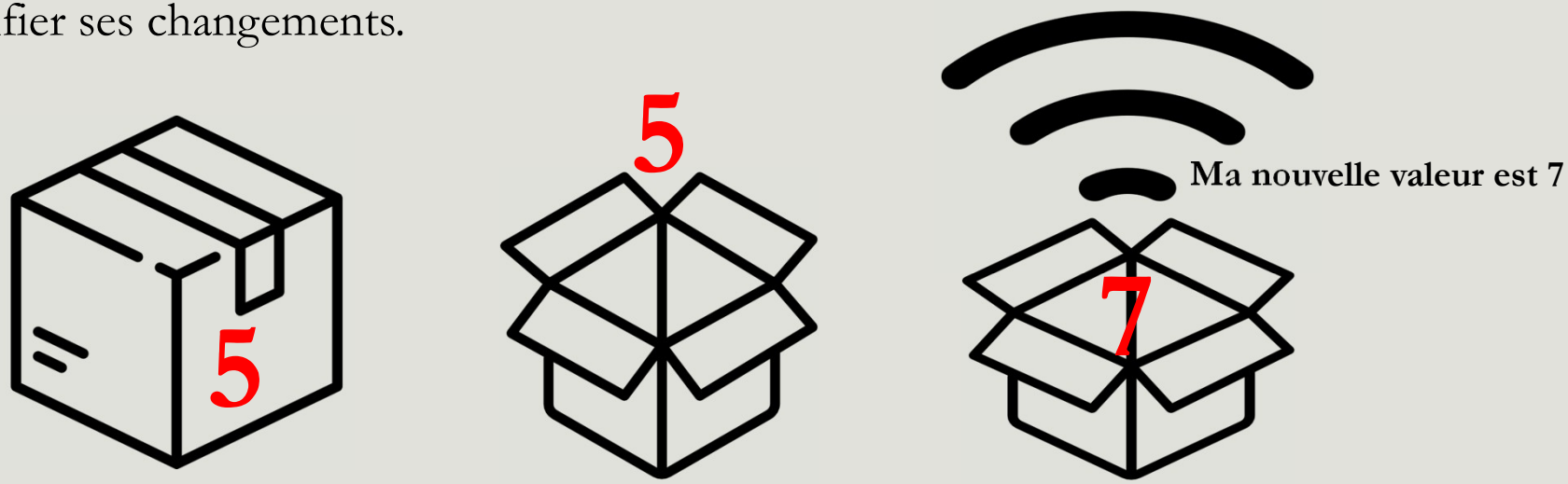


Qu’est ce qu’un Signal ?



- Un signal agit comme une **enveloppe (wrapper) autour d'une valeur**, ayant la capacité **d'avertir les consommateurs lorsque la valeur change**.

- On peut modéliser ca en disant qu’un signal c’est une donné qui a le pouvoir de notifier ses changements.

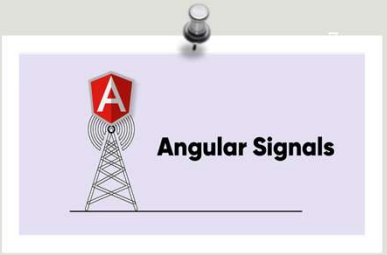


Qu'est ce qu'un Signal ?



- Un signal est une **primitive réactive** qui représente une valeur et qui nous permet de
  - **suivre ses changements** au fil du temps.
  - **modifier** cette même valeur en **notifiant tous ceux qui en dépendent**.
- Elle permet de définir l'état réactif de votre application et d'avoir une **identification précise de quels composants sont impactés par un changement**.

Qu'est ce qu'un Signal ?



- Les signaux sont un concept utilisé dans plusieurs frameworks (Qwik, SolidJs, Vue, KnockoutJs)
- Le concept du **Signal** dans Angular est une fonctionnalité introduite en '**Developer Preview**' dans la version **16** de la bibliothèque @angular/core et **stable à partir de Angular 17**.
- Il a pour objectif **de simplifier le développement** en donnant une **alternative plus simple que RxJS** pour gérer **certains cas de réactivité** d'une façon plus simple.

Pourquoi intégrer les signaux



Reactivity Everywhere

Les signaux permettent de réagir aux changements d'état (State) n'importe où dans notre code et pas seulement au sein d'un composant.

Precision Updates

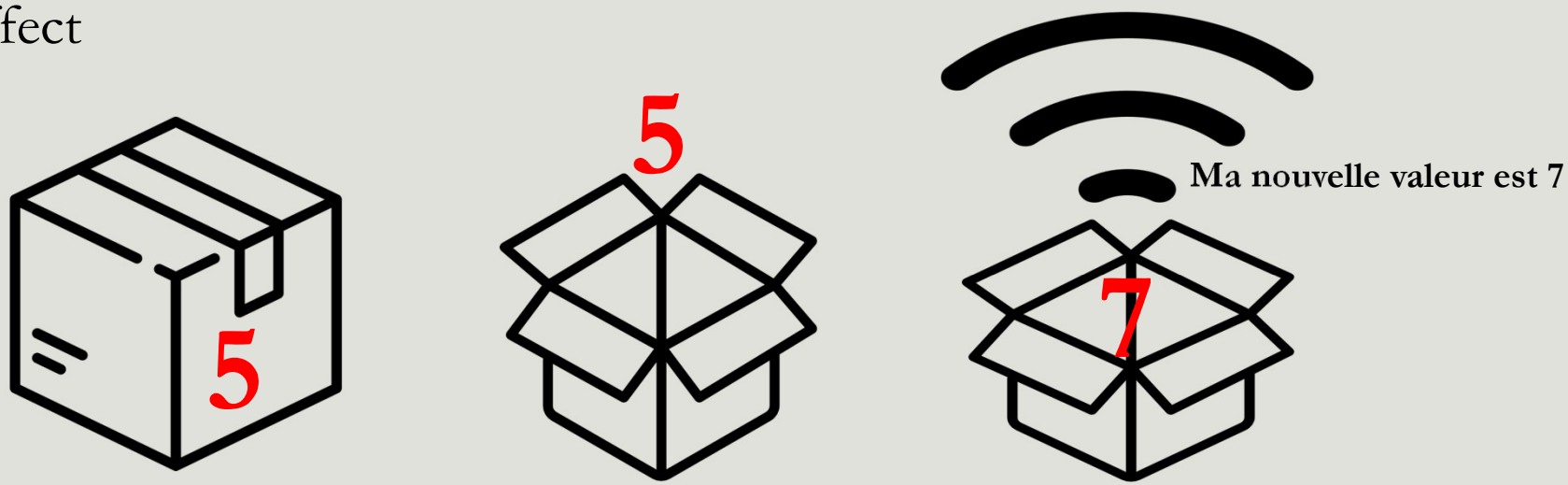
Les signaux boostent les performances de votre application en réduisant le travail que fait Angular pour garder le DOM à jour avec les valeurs des données

Lightweight Dependencies

Les signaux pèsent 2KB, n'ont pas besoin de charger des dépendances tierces et ne représentent aucun coût de démarrage lorsque votre application se charge.

API

- Angular propose trois principales primitives pour utiliser les signaux :
  - signal
  - computed
  - effect



Créer un signal via l'api signal()

- La fonction **signal** permet d'initialiser une variable et d'informer Angular et son contexte à chaque fois que sa valeur change.
- Elle retourne un objet de type **WritableSignal.**
- **Un signal doit avoir une valeur initiale**

```
@Component({
  selector: 'app-signal-api',
  standalone: true,
  imports: [],
  styleUrl: './signal-api.component.css',
  template: ` <h1>Hello World</h1> `,
})
export class SignalApiComponent {
    lastname: WritableSignal<string> = signal('sellaouti');
}
```



Récupérer la valeur d'un signal

- Afin de **récupérer la valeur d'un signal** il suffit de l'appeler comme une fonction.

```
@Component({
  selector: 'app-signal-api',
  standalone: true,
  imports: [],
  styleUrl: './signal-api.component.css',
  template: ` <h1>Hello {{ lastname() }}</h1> `,
})
export class SignalApiComponent {
     lastname: WritableSignal<string> = signal('sellaouti');
}
```

sellaouti



Les méthodes de modifications de signal : set() et update()

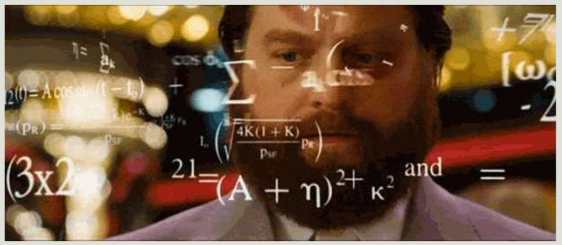
- Pour modifier la valeur d'un **signal**, on peut passer par :
- La Méthode **set**, qui permet **d'affecter une nouvelle valeur au signal**.
- La méthode **update**, qui permet de **calculer** une **nouvelle valeur** d'un signal **en fonction de sa valeur précédente**.

```
@Component({
  selector: 'app-signal-api',
  standalone: true,
  imports: [],
  template: `
    <h1>Hello {{ lastname() }}</h1>
    <input type="number" #input
          (change)="setCounter(+input.value)"
    />
    <h2 (click)="increment()">
      Click here. You clicked {{ counter() }} times
    </h2>
  `,
})
export class SignalApiComponent {
  lastname = signal('aymen');
  counter = signal(0);
  increment() {
    this.counter.update((currentValue) => currentValue + 1);
  }
  setCounter(val: number) {
    this.counter.set(val);
  }
}
```

Quand utiliser un signal et non une variable standard ?

- **Utilisez les signals**
  - pour chaque **variable d'état de votre composant dont la valeur change**.
  - lorsque vous voulez **suivre les changements** de votre variable
  - quand la **valeur de votre variable** doit **changer quand d'autres variables changent**
- **N'utilisez pas les signals**
  - quand la **valeur ne change pas**
  - Quand c'est une **variable locale** que vous **n'utilisez pas dans votre ui** ou dans dans **calculs réactifs**
  - pours les **événements**
  - pour les **opérations asynchrones**

Exercice



- Reprenez L'exercice 'ColorComponent' et rotatingCardComponent en utilisant les signaux.

Cas d'utilisation

- Soit le composant SumComponent. Que se passe t il si on incrément x ou y ?

```
export class SumComponent {
  x = 3;
  y = 5;
  z = this.x + this.y;
}
```

```
<input type="number" [(ngModel)]="x"
class="form-control">
<input type="number" [(ngModel)]="y"
class="form-control">
{{ x }} + {{ y }} = {{ z }}
```

computed()

- L'api **computed**() permet de créer un **nouveau signal** dont la valeur **dépend d'autres signaux**.
- Lorsqu'un **signal est mis à jour**, **tous ses signaux dépendants seront alors automatiquement mis à jour**.
- On note que **computed() retourne un objet de type Signal** et non **WritableSignal**.
- Vous **ne pouvez pas modifier un computed**, il n'est pas Writable

```
lastname = signal('aymen');
firstname = signal('sellaouti');
fullname = computed(() => `${this.firstname()} ${this.lastname()}`)
```

computed()
Modifier un signal dans un computed

- computed() s'attend à ce que le calcul **soit léger et rapide**, et **n'entraîne aucun effet secondaire**.
- Vous **ne pouvez pas modifier un signal dans un computed**, ceci provoquera une **erreur**.
- Ne **modifiez aucune variable** dans les computed ; pas seulement les signaux, mais aussi les variables extérieures à la fonction.
- Ne **modifiez pas le DOM** et **n'exécutez pas de tâches asynchrones**.

```
fullname = computed(() => {
    console.log('i am computing....');
    this.firstname.set('ccc');
    return `${this.firstname()} ${this.lastname()}`;
  });
```

```
▶ERROR Error: NG0600: Writing to VM1270:1
signals is not allowed in a `computed` or
an `effect` by default. Use
`allowSignalWrites` in the
`CreateEffectOptions` to enable this inside
effects.
```

computed()

- Pour identifier un signal qui a changé, et donc si on doit exécuter un computed, Angular utilise **Object.is**.
- Object.is() permet de déterminer si deux valeurs sont identiques. Deux valeurs sont considérées identiques si :
  - elles sont toutes les deux undefined
  - elles sont toutes les deux null
  - elles sont toutes les deux true ou toutes les deux false
  - elles sont des chaînes de caractères de la même longueur et avec les mêmes caractères (dans le même ordre)
  - **elles sont toutes les deux le même objet (même référence)**
  - elles sont des nombres et
    - sont toutes les deux égales à +0
    - sont toutes les deux égales à -0
    - sont toutes les deux égales à NaN

computed()
Comment ca marche ?

- L'arbre de dépendance est crée dynamiquement à chaque appel du computed.
- Il faut donc faire très attention dans la définition de vos computed lorsqu'il y a des traitements conditionnels.

```
@Component({
  template: `
    <h3>Counter value {{ $counter() }}</h3>
    <h3>Derived counter: {{ $derivedCounter() }}</h3>
    <button (click)="increment()">Increment</button>
    <button (click)="multiplier = 10">Set multiplier to 10</button>
  `,})
export class ComputedProblemComponent {
  $counter = signal(0);
  multiplier: number = 0;
  $derivedCounter = computed(() => {
    if (this.multiplier < 10) {
      return 0;
    } else {
      return this.$counter() * this.multiplier;
    }
  });
  increment() {
    console.log(`Updating counter...`);
    this.counter.set(this.$counter() + 1);
  }
}
```



https://blog.angular-university.io/angular-signals/

computed()
Exclure un signal de l'arbre de dépendance

- Si, pour une raison ou une autre, vous voulez lire une valeur d'un signal dans un comptued mais sans l'intégrer dans l'arbre de dépendance, vous pouvez utiliser l'Api **untracked**.
- Dans cet exemple le computed fullname ne sera mis à jour que si le signal firstname change

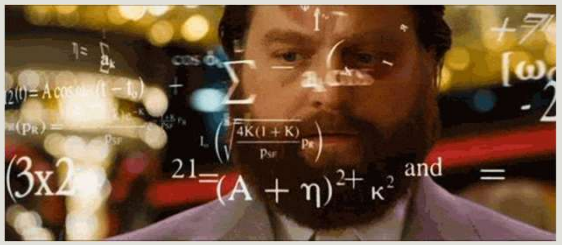
```
lastname = signal('sellaouti');
firstname = signal('aymen');
fullname = computed(() => `${this.firstname()} ${untracked(this.lastname)}`);
```

computed()
Les computed, autres propriétés

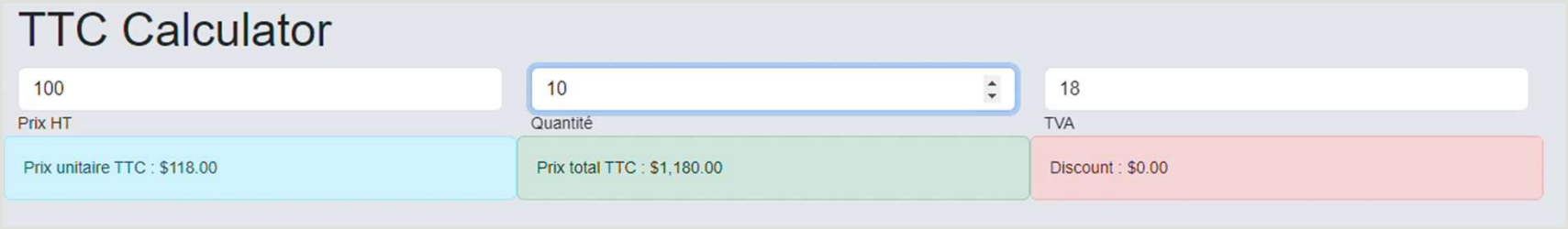
- Les computed sont paresseux (**lazy**), ce qui signifie que computed **n'est invoquée que lorsque quelqu'un s'intéresse (lit) à sa valeur**. Cela permet **d'optimiser les performances** en évitant les calculs inutiles.
- Les computed sont **automatiquement supprimés** lorsque la référence du signal calculée devient hors de portée.
- Cela garantit que les ressources **sont libérées et qu'aucune opération de nettoyage explicite n'est requise**.
- Ceci est due à l'utilisation des weakRefrence avec les signaux

https://en.wikipedia.org/wiki/Weak_reference

Exercice



- Créer un composant TTC qui permet de calculer le prix TTC d'un produit selon le nombre de pièces et la TVA. Sachant que l'utilisateur peut changer toutes les valeurs et que par défaut la quantité est de 1 le prix est de 0 et la tva est de 18.
- Si le nombre de pièces est entre 10 et 15 une remise de 20% est appliquée.
- Si le nombre de pièces est supérieur à 15 une remise de 30% est appliquée.



L'API effect()

- Dans certains cas d'utilisation, vous avez besoin d'être notifié par le changement d'un signal pour **effectuer un effet de bord**, donc faire un **traitement sans pour autant créer un nouveau signal ou en modifier d'autres**.
- Pensez à un log, update du localStorage,…
- L'effect va être réexécuté si l'un des signaux qu'il utilise émet une nouvelle valeur.

L'API effect()

- Vous pouvez **créer un effet** via la fonction **effect**.
- Les effets s'exécutent toujours **au moins une fois**.
- Lorsqu'un effect est exécuté, il **suit tous les signaux qu'il contient**. Chaque fois que l'une de ces valeurs de **signal change**, **l'effet se reproduit**.
- L'effet est **semblable aux computed**, les effets gardent une **trace de leurs dépendances de manière dynamique** et ne suivent que les **signaux** qui ont été **lus lors de l'exécution la plus récente**.

```
constructor() {
 effect(() => {
   console.log(`count:${this.counter()}`);
 });
}
```

```
private logEffect = effect(() => {
  console.log(`
     The current count is:${this.counter()}
  `);
});
```

L'API effect()

- Moment d'exécution : Un effect s'exécute pendant le cycle de détection des changements, après que les signaux suivis ont été mis à jour, mais avant que le DOM ne soit complètement rendu.
- Depuis Angular 19, les effets de composant (component effects) sont intégrés au processus de détection des changements, ce qui garantit un timing plus prévisible.
- Les effets seront exécutés le **nombre minimum de fois**. Si un effet **dépend** de **plusieurs signaux** et que plusieurs d'entre eux **changent simultanément**, **une seule exécution de l'effect sera programmée**.
- **Limitation** : Ne **doit pas être utilisé pour des manipulations directes du DOM**, car **le DOM peut ne pas être encore mis à jour.**

```
export class EffectComponent {
  counter = signal(0);
  constructor() {
    this.counter.set(1);
    this.counter.set(2);
  }
  private logEffect = effect(() => {
    console.log(`
      The current count is:
      ${this.counter()}`);
  });
}
```

L'API afterRenderEffect()

- La primitive **afterRenderEffect** (**introduite dans Angular 19**) est une **extension de effect** qui **s'exécute après que le rendu du DOM** est terminé. Elle est conçue pour les cas où vous **devez interagir avec le DOM** ou **effectuer des opérations qui dépendent de son état final**.

```
@ViewChild('myDiv') myDiv!: ElementRef<HTMLDivElement>;
constructor() {
  effect(() => {
    console.log( `myDiv width effect: ${this.myDiv.nativeElement.offsetWidth}px`);
    console.log(`myDiv height effect: ${this.myDiv.nativeElement.offsetHeight}px`);
  });
  afterRenderEffect(() => {
    console.log(`myDiv width afterRenderEffect:${this.myDiv.nativeElement.offsetWidth}px`);
    console.log(`myDiv height afterRenderEffect: ${this.myDiv.nativeElement.offsetHeight}px`);
  });
}
```

Writablesignals et signals

- Par défaut, tous les **signaux** créés par la fonction Signal sont de type **WritableSignal**. Ainsi, n'importe quelle entité peut modifier sa valeur (via les méthodes set() et update()).
- Si on désire **interdire toute modification de la valeur d'un signal**, Angular nous propose la méthode **asReadOnly().**
- Les **signaux** **créés par computed** sont, **par défaut, read only.**

```
  private currencies = signal(['USD', 'EUR', 'GBP']);
  currenciesSignal = this.currencies.asReadonly();
```

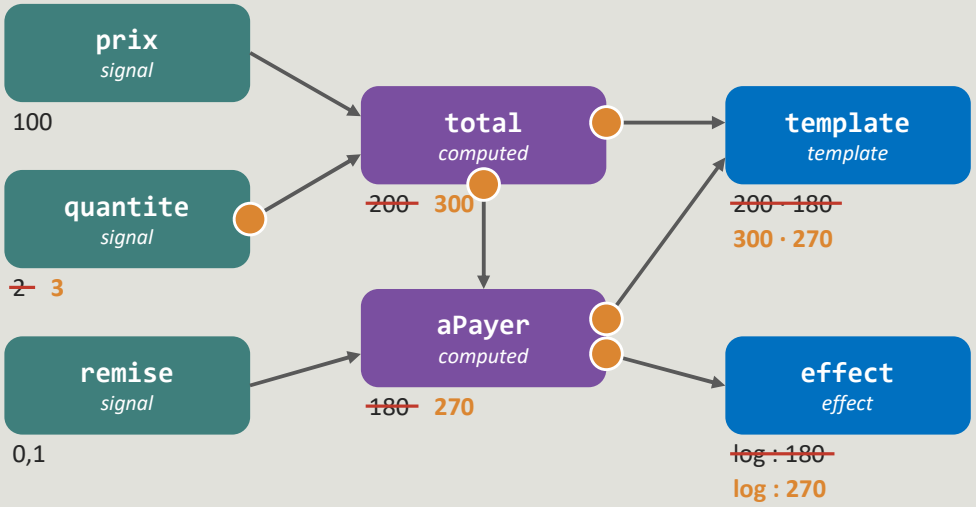
Le graphe réactif : producteurs et consommateurs

```
prix = signal(100);
quantite = signal(2);
remise = signal(0.1);

total = computed(() =>
  this.prix() * this.quantite());
aPayer = computed(() =>
  this.total() * (1 - this.remise()));

constructor() {
  effect(() => console.log(this.aPayer()));
}
// template : {{ total() }} {{ aPayer() }}

// plus tard : this.quantite.set(3);
```



producteur pur : signal()

producteur et consommateur : computed(), linkedSignal()

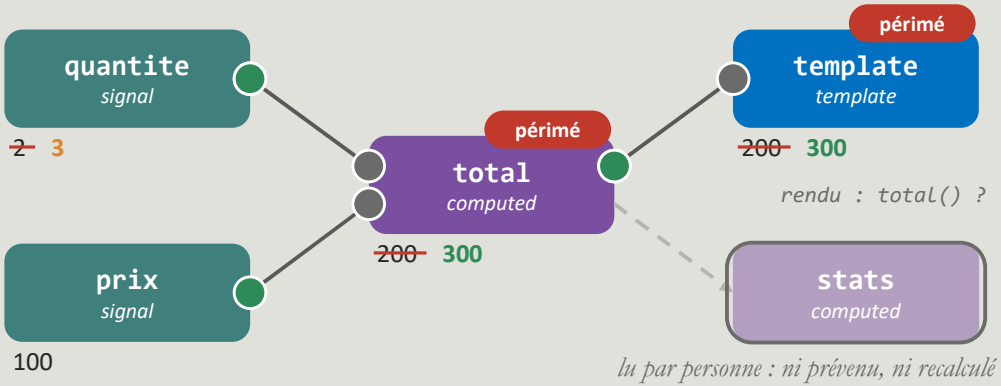
consommateur pur : template, effect()

Push puis pull : comment un changement se propage

```
prix = signal(100);
quantite = signal(2);
total = computed(() =>
  this.prix() * this.quantite());
stats = computed(() =>
  calculLourd(this.total()));

// template : {{ total() }}
// stats n'est lu par personne

this.quantite.set(3);
```



1. push : on prévient (périmé), on ne calcule rien
2. pull : à la lecture, on recalcule ce qui est lu

producteur pur : signal() producteur et consommateur : computed(), linkedSignal() consommateur pur : template, effect()

Même valeur, pas de propagation (Object.is)

```
age = signal(20);
majeur = computed(() => this.age() >= 18);

// template :
// @if (majeur()) { Bienvenue }
// @else { Accès refusé }

this.age.set(21);   // majeur reste true
this.age.set(16);   // majeur passe à false
```



true === true (Object.is) : rien ne change, le template n'est pas recalculé

Un signal ou un computed dont la nouvelle valeur est identique (`Object.is`) ne réveille pas ses consommateurs. Pour un objet, préciser l'égalité : `computed(fn, { equal: (a, b) => a.id === b.id })`.

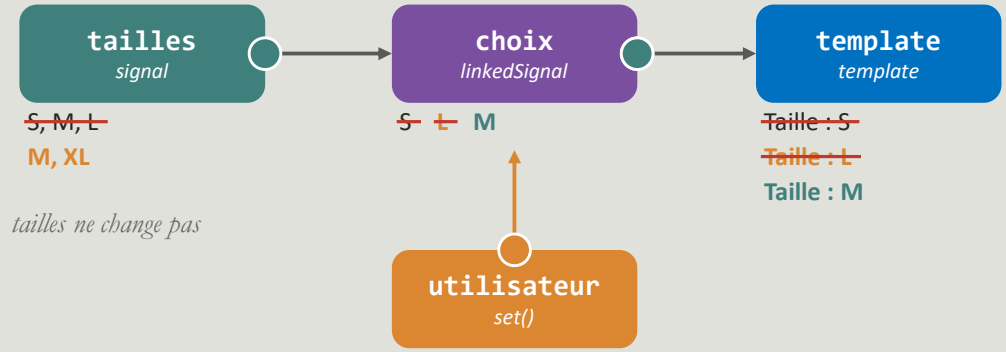
producteur pur : signal() — producteur et consommateur : computed(), linkedSignal() — consommateur pur : template, effect()

linkedSignal : dérivé, mais modifiable

```
tailles = signal(['S', 'M', 'L']);
choix = linkedSignal(() => this.tailles()[0]);

// l'utilisateur choisit une taille
this.choix.set('L');

// un autre produit : nouvelle liste
this.tailles.set(['M', 'XL']);   // choix : 'M'
```



la source change : le calcul reprend la main

`signal` : écrit, sans dépendance · `computed` : dérivé, lecture seule · `linkedSignal` : dérivé et modifiable

producteur pur : signal() · producteur et consommateur : computed(), linkedSignal() · consommateur pur : template, effect()

Angular 17
Le nouveau flux de control

- Afin de continuer sur la logique de simplification de l'apprentissage d'Angular, l'équipe Angular a proposé de **nouveaux flux de contrôle**.
- Les flux suivants ont été proposés :
  - @If
  - @for
  - @switch

Control flow
@if

- **@if** est associé à une expression booléenne. Si cette expression est fausse (false) alors l'élément et son contenu sont retirés du DOM, ou jamais ajoutés.
- **@if(condition)**
  - Si le booléen est true alors l'élément host est visible.
  - Si le booléen est false alors l'élément host est caché.

```
@if (authService.isAuthenticated()) {
<button
  (click)="deleteCv(cv)"
  class="btn btn-danger">
  Delete
</button>
}
```

Control flow
@if, @else if et @else

- **@if** peut également être utilisé avec **@else if** et/ou **@else** selon le besoin.
- La **syntaxe est très intuitive**, demandé vous comment vous aurez fait en Javasript et **ajoutez un @ :D.**

```
@if (connectedUser) {
  Hello {{ connectedUser.name}}
} @else {
<div>Merci de vous connectez</div>
}
```

Control flow
@if, @else if et @else

```
@if (!connectedUser) {
<div class="alert alert-danger">Merci de vous connectez</div>
} @else if (!connectedUser.activated) {
<div class="alert alert-warning">
  Hello {{ connectedUser.name }}, merci d'activer votre compte
</div>
} @else {
<div class="alert alert-success">Hello {{ connectedUser.name }}</div>
}
```

Control flow
@for

- La directive structurelle **@for** permet de boucler sur un itérable et d'injecter les éléments dans le DOM.

```
<ul>
  @for (episode of episodes; track episode.id) {
    <li>{{ episode.title }}</li>
  }
</ul>
```

Control flow
@For

- **@for** fournit certaines informations sur la boucle en cours :
  - **$index**: position de l'élément.
  - **$odd**: true si l'élément est à une position impaire.
  - **$even**: true si l'élément est à une position paire.
  - **$first**: true si l'élément est à la première position.
  - **$last**: true si l'élément est à la dernière position.

```
<ul>
  @for (episode of episodes; track episode.id) {
  <li>
    Episode {{ $index + 1 }} : {{ episode.title }}
  </li>
  }
</ul>
```

Control flow
@For
track

- Avec @For la **fonction de tracking est devenue obligatoire**
- La fonction de suivi créée via l'instruction **track** est **utilisée pour permettre au mécanisme de détection des changements d'Angular de savoir exactement quels éléments mettre à jour dans le DOM** après les modifications de l'itérable d'entrée.
- La fonction de suivi **indique à Angular comment identifier de manière unique un élément de la liste**.

```
@for (episode of episodes; track episode.id) {
  <li>{{ episode.title }}</li>
}
```

Control flow
@For
track

- En principe, **il devrait toujours y avoir quelque chose d'unique** dans les **éléments sur lesquels vous itérer**.
- Dans le **pire des cas**, s'il n'y a rien d'unique dans les éléments du tableau, vous pouvez **utiliser $index de l'élément**, c'est-à-dire la **position de l'élément dans le tableau**.

```
@for (episode of episodes; track $index) {
  <li>{{ episode.title }}</li>
}
```

Control flow
@For, la gestion d'un itérable vide

- Afin de gérer le cas ou votre **itérable est vide**, nous avons le block **@empty**.
- Ce bloque **n'est activé** que si **l'itérable** sur lequel vous bouclez est **vide**.

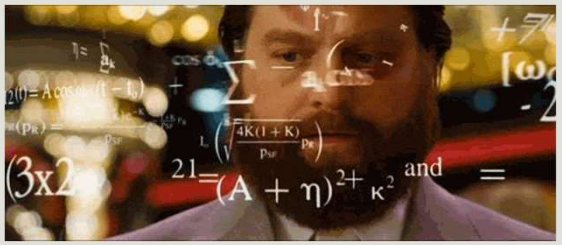
```
<ol class="list-group">
  @for (player of players; track player.id) {
    <li class="list-group-item">{{player.name}}</li>
  }
  @empty {
    <li class="list-group-item list-group-item-danger">La liste ne nous est
pas encore parvenue</li>
  }
</ol>
```

Control flow @switch

- Avec **@switch**, vous pouvez créer des switch très simplement.
- Vous avez les trois opérateurs : **@switch, @case, @default**
  - **@switch** : définir l'élément sur lequel switcher
  - **@case** : pour identifier le cas
  - **@default** : pour les valeurs par défaut

```
@switch(streamingService) {
  @case ('Disney+') {
    <div>'Mandalorian'</div>
  } @case ('AppleTV') {
    <div>'Ted Lasso'</div>
  } @default {
    <div>'Peaky Blinders'</div>
  }
}
```

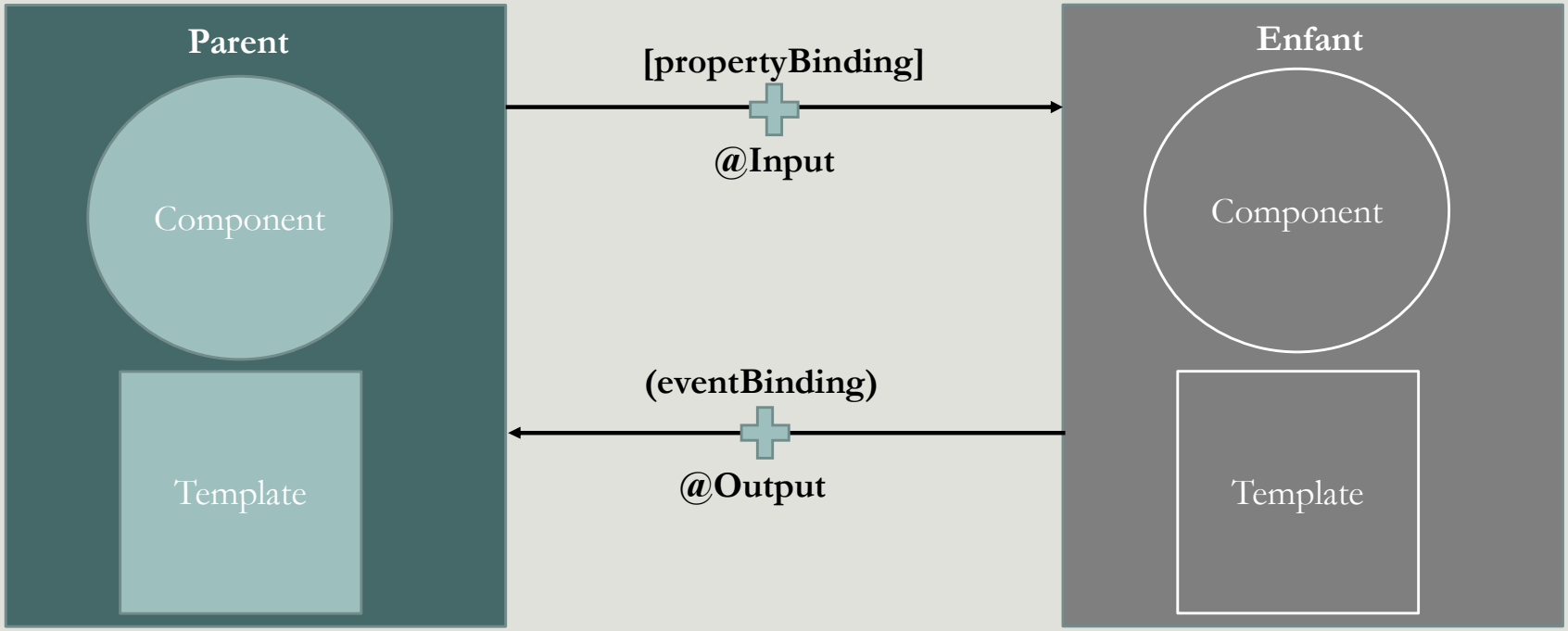
Exercice



- Créer un nouveau composant names.
- Définissez un tableau de noms
- Affichez la liste des noms et faites en sorte que les noms, dont la taille dépasse 10, soit suivie d'une icone de votre choix

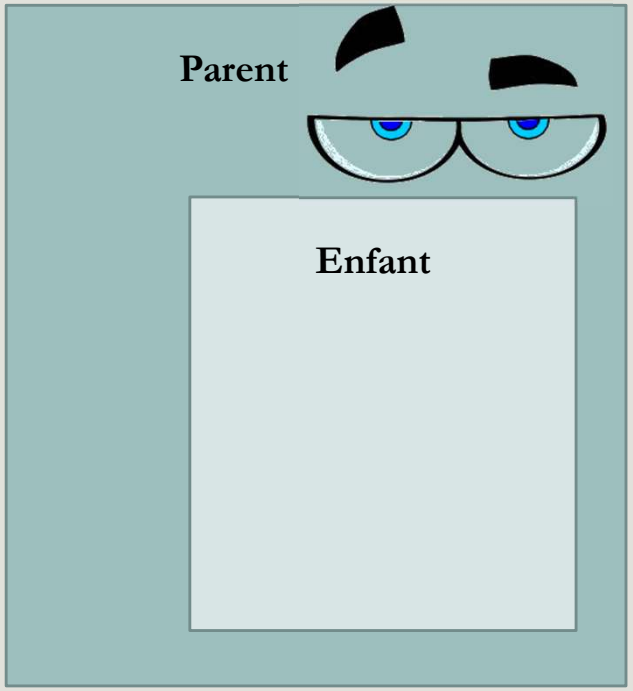
sellaouti
lewandowski ♡
dupont

Interaction entre composants



Pourquoi ?

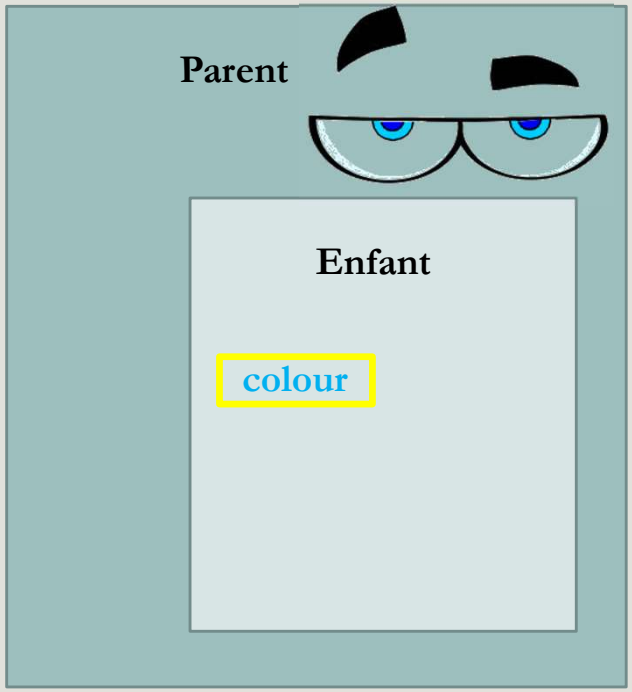
Le père voit le fils, le fils ne voit pas le père




```
import { Component } from '@angular/core';

@Component({
  selector: 'app-pere',
  template: `
    <p>Je suis le composant père </p>
    <forma-fils></forma-fils>
`,
  styles:
})
export class PereComponent {
  color = 'green';
}
```

Interaction du père vers le fils



- Le parent **voit l'enfant** mais **ne peut pas voir ses propriétés**.
- Solution : Faire du **property binding avec @input**, qui peut prendre un objet en paramètre (pour spécifier que l'envoie d'une valeur est required par exemple).

```ts
import { Component, OnInit, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-child-first',
  templateUrl: './child-first.component.html',
  styleUrls: ['./child-first.component.css']
})
export class ChildFirstComponent implements OnInit {

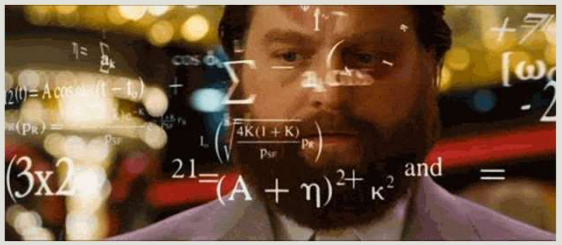
colour:string;
```

Ts du fils

```html
<app-fils [colour]="color"/>
```

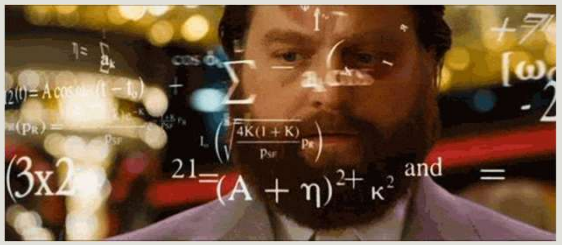
Template du pere

Exercice



- Créer un composant fils
- Récupérer la couleur du père dans le composant fils
- Faite en sorte que le composant fils affiche la couleur du background de son père

Signal Input



- Commençons cette partie avec un petit exercice. Créer un composant isEven qui récupère un entier en input et qui affiche s'il est paire ou impaire.
- Créer un autre composant qui contient un champ de texte de type number.
- Ce composant doit appeler le composant isEven et lui passer en paramètre la valeur de l'input.



Signal Input

- Pour que l'exemple précédent fonctionne, on avait deux méthodes :
  - Utiliser un setter
  - Utiliser le OnChange hook
- La version 17.1 a vu Angular introduire le concept de *Signal Input* pour améliorer l'interaction entre les composants.
- Les Signal Input permettent de lier les valeurs des composants parents. Ces valeurs sont exposées à l'aide d'un signal et peuvent changer au cours du cycle de vie de votre composant.

```
import { Input, input } from '@angular/core';
  isEvenn!: boolean;
  // Sans Signal Input
  @Input() isEven: number;
  // Avec les signal Input
  counter = input<number>();
```

Signal Input

- **Comme les @Input**, le Signal Input peut être optionnel ou **required**
- Il peut avoir une **valeur par défaut**
- Il peut avoir une **Alias**
- Et vous pouvez le **transformer**

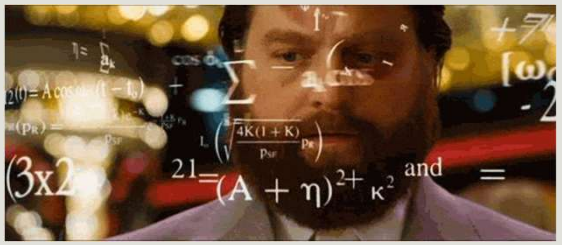
```
// optional
counter = input<number>();
// Valeur par défaut = 0
counter = input<number>(0);
// Required
counter = input.required<number>();
```

```
counter = input(0,{
    alias: 'counter',
    transform: (value: number) =>  value * 100,
  });
```

```
  counter = input.required({
    alias: 'counter',
    transform: (value: number) => value * 100,
  });
```

```
// avant
@Input({required: true})
Counter!:number;
// après
counter = input.required<number>();
```

Signal Input



- Reprenez cet exemple en utilisant les Signal Input



Signal Input Migration

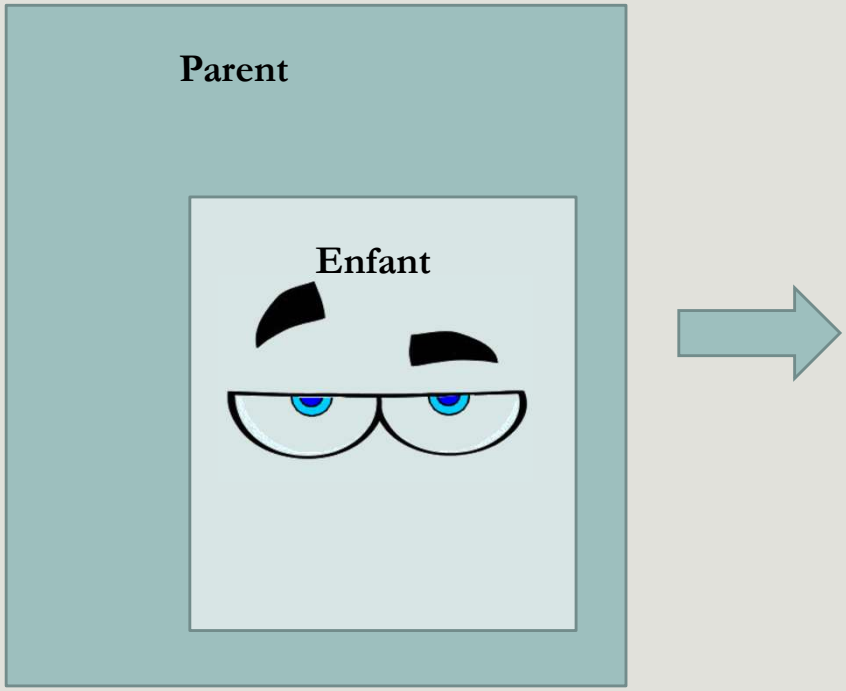
- Vous pouvez migrer toute votre application vers les signals input via cette commande :

ng generate @angular/core:signal-input-migration

- Vous avez plusieurs options de migrations :
  - **--best-effort-mode** : Par défaut, la **migration ignore les entrées qui ne peuvent pas être migrées en toute sécurité.** Elle tente de refactoriser le code de la manière la plus sûre possible.
  - Lorsque l'option **--best-effort-mode est activée**, la migration **s'efforce de migrer autant que possible, même si cela risque d'endommager votre build**.
  - **--insert-todos** Lorsque cette option est activée, la **migration ajoute des tâches à effectuer aux entrées qui n'ont pas pu être migrées**. Ces tâches incluent les raisons pour lesquelles les entrées ont été ignorées.
  - **--analysis-dir** : Dans les projets volumineux, vous pouvez utiliser cette option pour **réduire le nombre de fichiers analysés**.

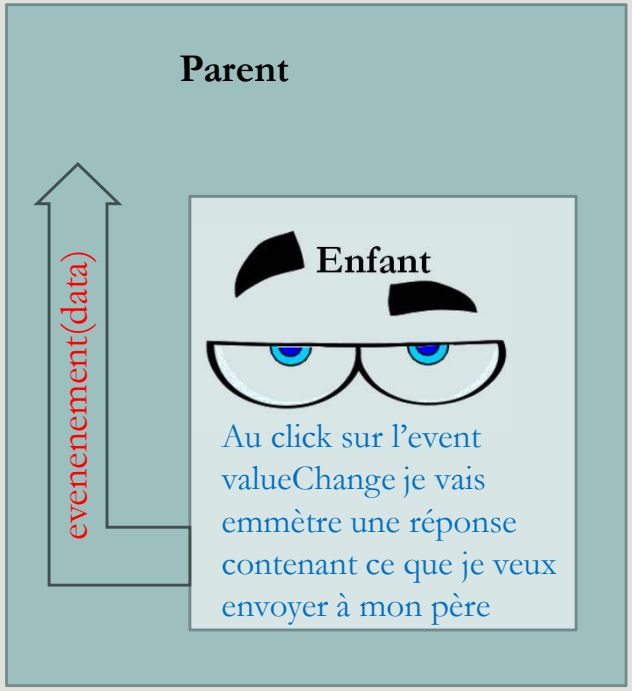
Interaction du fils vers le père

L'enfant ne peut pas voir le parent. Appel de l'enfant vers le parent. Que faire ?




```
import {Component} from '@angular/core';
@Component({
  selector: 'bind-output',
  template: `Bonjour je suis le fils`,
  styleUrls: ['./output.component.css']
})
export class OutputComponent {
  valeur:number=0;
  }
}
```

Interaction du fils vers le père



- L'enfant ne voit pas le parent car il ne sait simplement pas par quel composant il a été appelé.
- Solution : 1. Faire du event binding avec @output

```
import { Component, OnInit, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-child-first',
  templateUrl: './child-first.component.html',
  styleUrls: ['./child-first.component.css']
})
export class ChildFirstComponent implements OnInit {
 @Output() sendRequest = new EventEmitter();
```

Template du fils

```
<app-fils [colour]="color"/>
```

Template du pere

Interaction du fils vers le père

2. Configurer l'événement

```
sendEvent() {
  this.sendRequest.emit('Accuse la réception de la couleur '+ this.color);
}
```

3. Récupérer l'évènement dans le composant parent

```
<app-child-first (sendRequest)="ReceivedEvent($event)"></app-child-first>
```

4. Traiter le message reçu de la part du composant enfant

```
ReceivedEvent(msg : any)
{
  alert(msg);
}
}
```

Signal Output

- L'API output() remplace directement le décorateur @Output() traditionnel.
- Le décorateur @Output n'est pas obsolète
- Angular a donc ajouté output() comme nouvelle façon de définir les sorties des composants dans Angular, d'une manière plus sûre et mieux intégrée à RxJs que l'approche traditionnelle @Output et EventEmitter.
- La syntaxe a été simplifiée.

```
<app-item (selectCv)="onSelectCv($event)" [cv]="cv"  />
```

Pere

```
/* Sans les signal Output */
 @Output() oldSelectCv = new EventEmitter<Cv>();
 /* Avec les signal Output */
 selectCv = output<Cv>();
 onClick() {
  if (this.cv) {
   /* Cette partie du code reste la même */
   this.selectCv.emit(this.cv);
  }
```

Fils

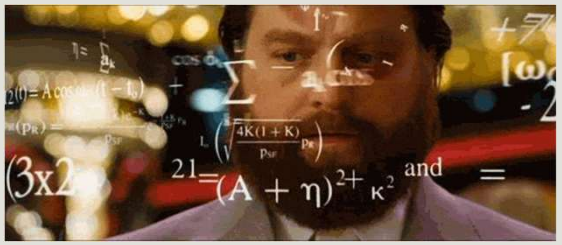
Signal Output Migration

- Vous pouvez migrer toute votre application vers les signals input via cette commande :

```
ng generate @angular/core:signal-output-migration
```

- Comme pour Input,Vous avez plusieurs options de migrations

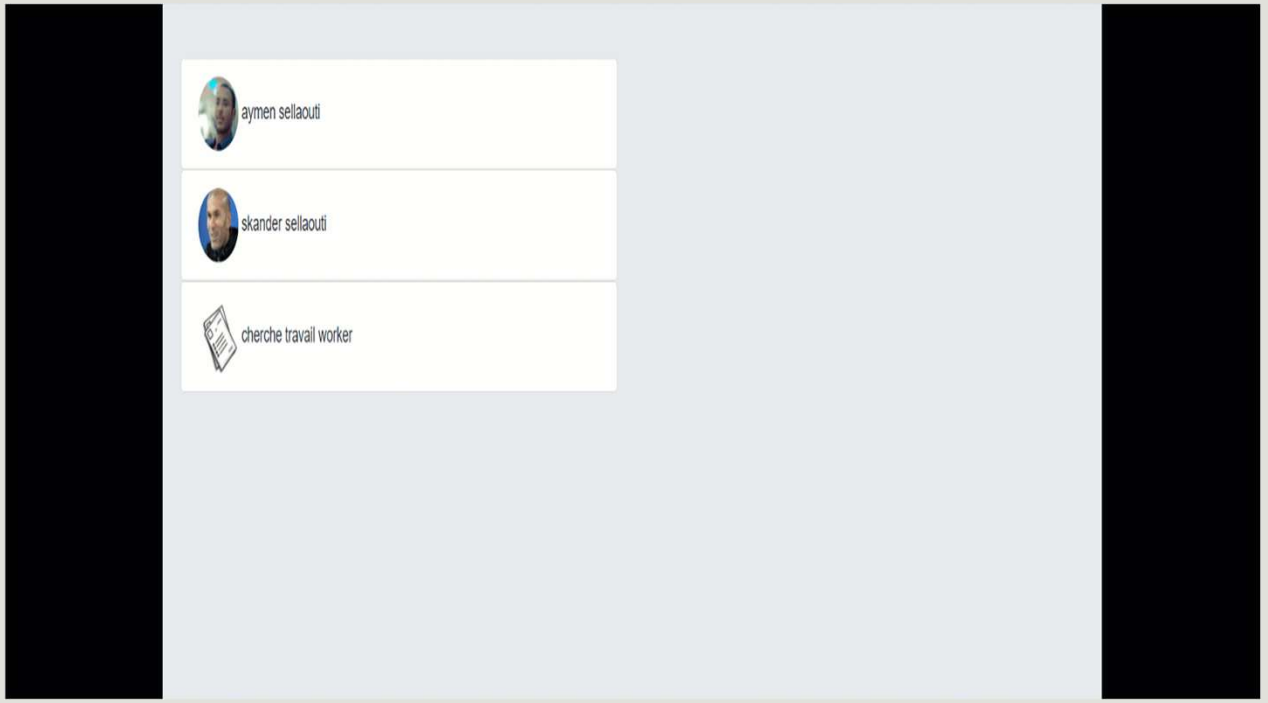
Exercice



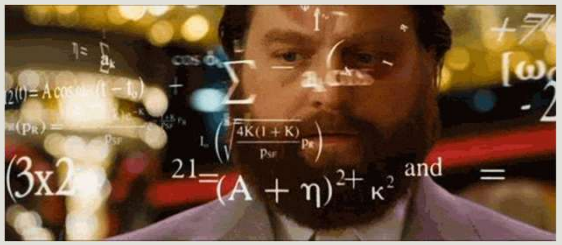
- Le but de cet exercice est de créer une mini plateforme de recrutement.

- La première étape est de créer la structure suivante avec une vue contenant deux parties :
  - Liste des Cvs inscrits
  - Détail du Cv qui apparaitra au click

- Il vous est demandé juste d'afficher un seul Cv et de lui afficher les détails au click.

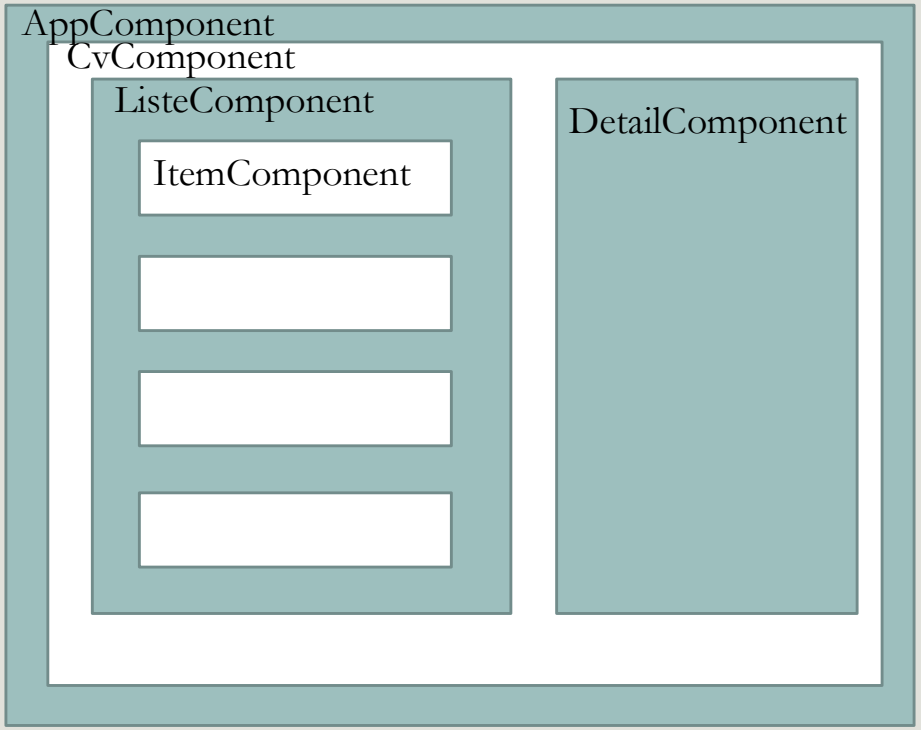


Exercice



- Voici la décomposition de l'interface en composant

Un cv est caractérisé par :
id
name
firstname
Age
Cin
Job
path



CvTech : qui stocke, qui affiche



L'état vit dans la page, plus proche parent commun de la liste et de la carte. Les données descendent par `input()`, les événements remontent par `output()` : Liste, Item et Detail affichent et préviennent.

CvTech : le même trajet dans le code

CvComponent (la page)

```
// cv.ts
cvs = signal<Cv[]>([...]);
selectedCv = signal<Cv | null>(null);
// cv.html
<app-liste [cvs]="cvs()"
  (selectCv)="selectedCv.set($event)" />
<app-detail [cv]="selectedCv()" />
```

ListeComponent

```
// liste.ts
cvs = input.required<Cv[]>();
selectCv = output<Cv>();
// liste.html
@for (cv of cvs(); track cv.id) {
  <app-item [cv]="cv"
    (selectCv)="selectCv.emit($event)" />
}
```

DetailComponent (la carte)

```
// detail.ts
cv = input<Cv | null>(null);
// detail.html
@if (cv(); as c) {
  <h3>{{ c.firstname }} {{ c.name }}</h3>
  <p>{{ c.job }}</p>
} @else { <p>Sélectionnez un CV</p> }
```

ItemComponent

```
// item.ts
cv = input.required<Cv>();
selectCv = output<Cv>();
// item.html
<div (click)="selectCv.emit(cv())">
  {{ cv().firstname }} {{ cv().name }}
</div>
```

`signal()` : l'état, dans la page seulement · `input()` : recevoir, en lecture seule · `output()` et `emit()` : prévenir le parent, qui écoute avec `(selectCv)` et lit la valeur dans `$event`.

Liste smart ou présentationnelle ?

Présentationnel : la liste arrive par input()

```
// liste.ts
cvs = input.required<Cv[]>()
```

```
// cv.html
<app-liste [cvs]="seniors()" />
<app-liste [cvs]="juniors()" />
```



onglet seniors

Aymen Sellaouti
Sami Trabelsi

onglet juniors

Skander Sellaouti
Ines Gharbi

✓ même composant, deux listes différentes

Smart : la liste va chercher les CV elle-même

```
// liste.ts
cvs = inject(CvService).getCvs()
```

```
// cv.html
<app-liste />  <!-- onglet seniors -->
<app-liste />  <!-- onglet juniors -->
```



onglet seniors

Aymen Sellaouti
Skander Sellaouti
Sami Trabelsi
Ines Gharbi

onglet juniors

Aymen Sellaouti
Skander Sellaouti
Sami Trabelsi
Ines Gharbi

✗ les deux onglets affichent tous les CV

Un seul composant smart, la page `CvComponent` : elle injecte `CvService`, stocke les CV et le CV sélectionné, calcule `seniors()` et `juniors()`. Liste, Item et Detail restent présentationnels, donc réutilisables.

Angular
Les directives

AYMEN SELLAOUTI

Objectifs

1. Comprendre la définition et l'intérêt des directives.

2. Voir quelques directives d'attributs offertes par angular et savoir les utiliser

3. Créer votre propre directive d'attributs

4. Voir quelques directives structurelles offertes par angular et savoir les utiliser

Qu'est ce qu'une directive

- Une directive est une **classe annotée avec le décorateur @Directive()** qui permet à Angular **d'attacher un comportement spécifique à un élément HTML**.
- Les directives permettent :
  - **Réutilisabilité du code : en centralisant** un comportement ou un style **réutilisable** dans toute l'application.
  - **Séparation des préoccupations (Separation of concerns)** :
    La logique **métier est séparée du design** ou de l'affichage.
  - **Clarté et lisibilité :**
    Plutôt qu'un code lourd dans le composant, la directive gère une fonctionnalité spécifique (exemple : gestion de permissions d'un bouton).
  - **Personnalisation facile :**
    Tu peux créer des directives sur mesure pour enrichir le comportement de tes composants

Qu'est ce qu'une directive

- La documentation officielle d'Angular identifie trois types de directives :
  - Les **composants** qui sont des directives avec des templates.
  - Les **directives structurelles** qui changent **l'apparence** du **DOM** en ajoutant et supprimant des éléments.
  - Les **directives d'attributs** (attribute directives) qui permettent de changer **l'apparence** ou le **comportement** d'un **élément.**
- Il existe des **directives fournies par Angular** mais vous pouvez **créer vos propres directives**

Les directives structurelles

- Une **directive** structurelle permet de modifier le DOM.
- Elles sont appliquées sur **l'élément HOST**.
- Elles sont généralement précédées par le **préfix *.**
- Les directives les plus connues **(dépréciées)** sont :
  - *ngIf
  - *ngFor

Les directives structurelles *ngIf

- Prend un booléen en paramètre.
- Si le booléen est true alors l'élément host est visible
- Si le booléen est false alors l'élément host est caché

Exemple

```
<p *ngIf="true">
  Je suis visible :D</p>
<p *ngIf="false">
  Le *ngIf c'est faché contre
moi et m'a caché :(
</p>
```

Les directives structurelles *ngFor

- Permet de répéter un élément plusieurs fois dans le DOM.
- Prend en paramètre les entités à reproduire.
- Fournit certaines valeurs :
  - index : position de l'élément courant
  - first : vrai si premier élément
  - last vrai si dernier élément
  - even : vrai si l'indice est paire
  - odd : vrai si l'indice est impaire

```
<ul>
  <li *ngFor="let episode of episodes">
  {{episode.title}}
  </li>
</ul>
```

```
<ul>
  <li *ngFor="let episode of episodes; let i = index;>
    Episode {{i+1}}{{episode.title}}
  </li>
</ul>
```

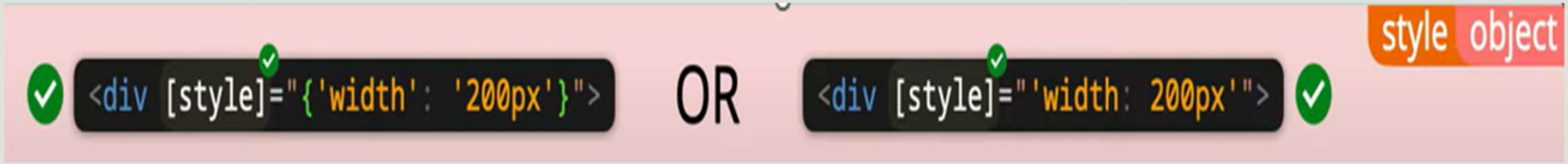
Les directives d'attribut (ngStyle) soft deprecation à partir d'NG19

- Cette directive permet de modifier **l'apparence** de **l'élément cible**.
- Elle est placée entre [ ] **[ngStyle] qu'on ajoute dans la balise cible.**
- Elle prend en paramètre un **attribut** représentant un **objet** décrivant le **style** à appliquer.
- Dans cet objet, la **clé** va représenter **l'attribut de style que vous voulez gérer**, e.g. color, backgroundColor, fontSize, fontFamily, border.
- La **valeur** représente **la valeur associée** à cette propriété de style et qui peut être une constante, dans ce cas elle ne change pas ou une variable et donc elle suivra toujours la valeur de cette variable.
- Dans cet exemple, l'attribut de style color prend sa valeur de la variable color, il sera donc 'lightblue'
- Pour l'attribut de style fontFamily on lui a associé une constante qui est 'garamond'.

```
@Component({
  selector: 'app-ngstyle-exemple',
  templateUrl: './ngstyle-exemple.component.html',
  styleUrls: ['./ngstyle-exemple.component.css']
})
export class NgstyleExempleComponent {
  color = 'lightblue';
}
```

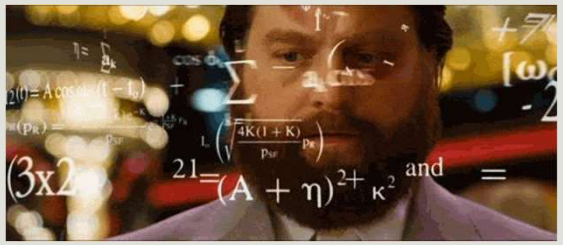
```
<p
  [ngStyle]="{
    backgroundColor: 'red',
    color: color,
    fontFamily: 'garamond'
  }"
>ngstyle-exemple works!</p>
```

Les directives d'attribut (ngStyle) soft deprecation à partir d'NG19

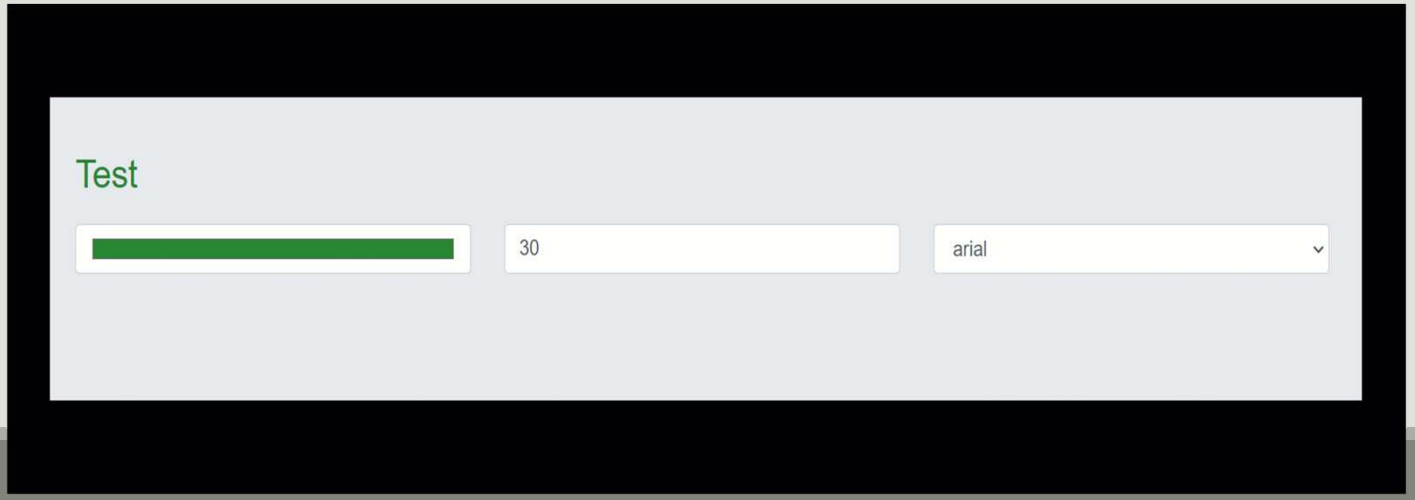


https://www.youtube.com/watch?v=x5tHq5zOh3c&ab_channel=IgorSedov

Exercice



- Nous voulons simuler un Mini Word pour gérer un paragraphe en utilisant ngStyle.
- Préparer un input de type color, un input de type number, et un select box.
- Faite en sorte que lorsqu'on change la couleur du color input, ca devienne la couleur du paragraphe. Et que lorsque on change le nombre dans le number input la taille de l'écriture.
- Finalement ajouter une liste et mettez-y un ensemble de police. Lorsque le user sélectionne une police dans la liste, la police dans le paragraphe change.



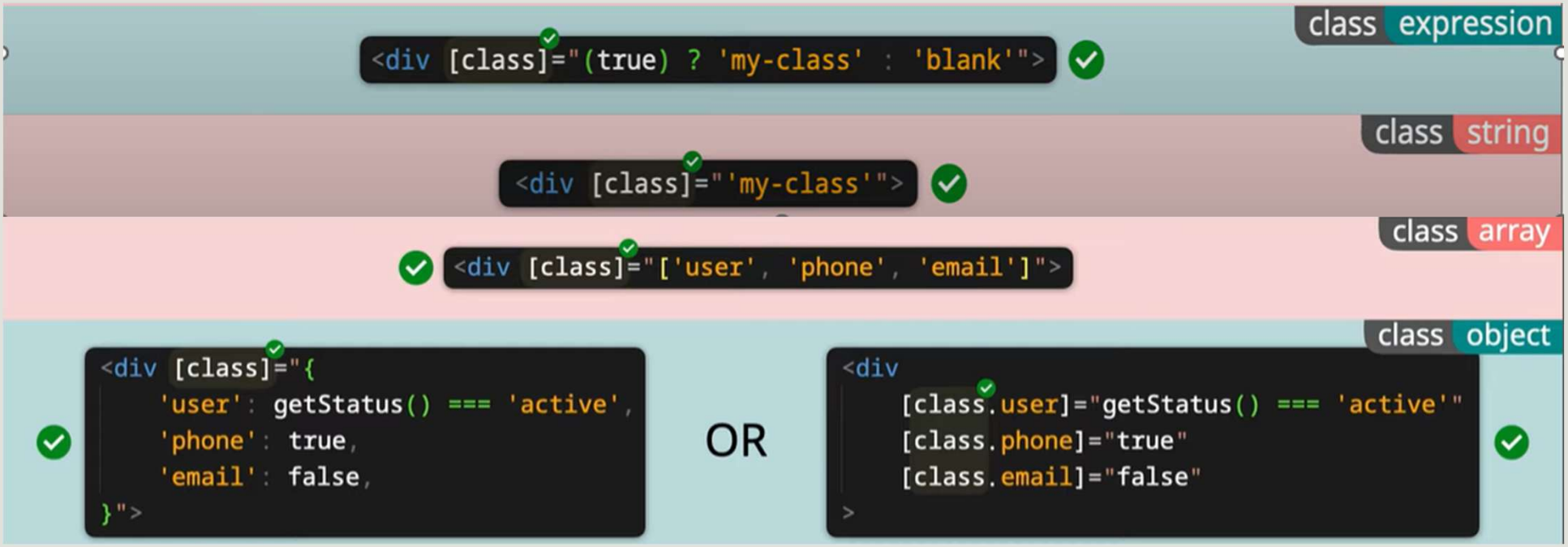
Les directives d'attribut (ngClass) soft deprecation à partir d'NG19

- Cette directive permet de modifier **l'attribut class**.
- Elle cohabite avec l'attribut **class**.
- Elle prend en paramètre
  - Une chaine (string)
  - Un tableau (dans ce cas il faut ajouter les [ ] donc [ngClass]
  - Un objet (dans ce cas il faut ajouter les [ ] donc [ngClass]
- Elle utilise le **property Binding.**
- Dans cet exemple la classe CSS off sera appliqué à la Div

```
@Component({
  selector: 'app-ngclass-exemple',
  templateUrl: './ngclass-exemple.component.html',
  styleUrls: ['./ngclass-exemple.component.css'],
})
export class NgclassExempleComponent {
  isOn = false;
}
```

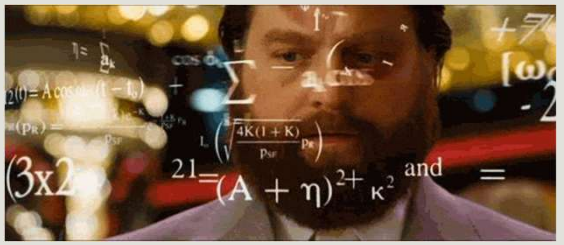
```
<div
  [ngClass]="{
    on: isOn,
    off: !isOn
  }"
  class="ampoule">lampe</div>
```

Les directives d'attribut (ngClass) soft deprecation à partir d'NG19

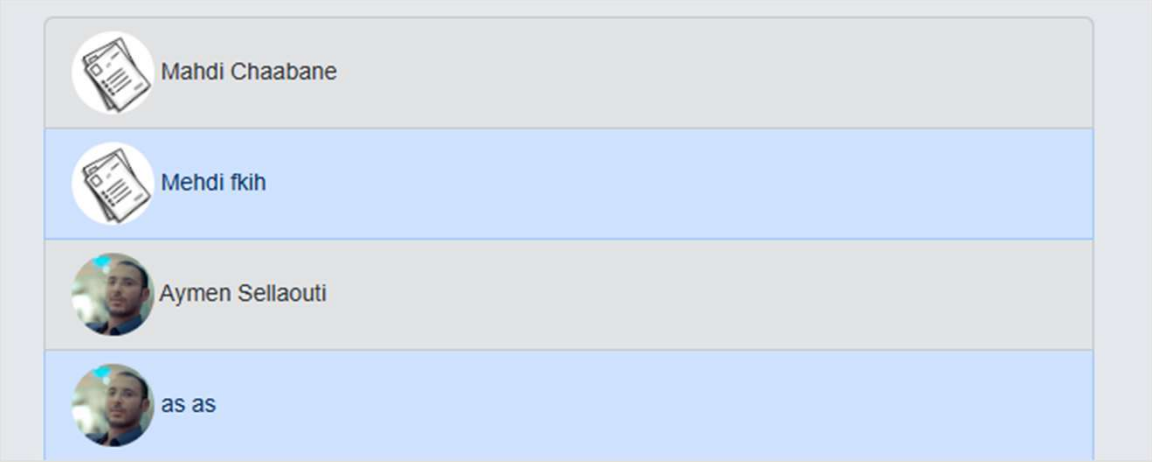


https://www.youtube.com/watch?v=x5tHq5zOh3c&ab_channel=IgorSedov

Exercice



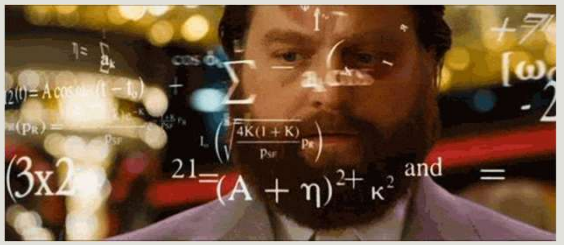
- Afin d'améliorer l'affichage de la liste des cvs dans votre CvTech, reprenez le composant cvList et faites-en sortes d'avoir les éléments successifs colorés d'une manière permutée entre deux couleurs.



Customiser une directive d'attribut

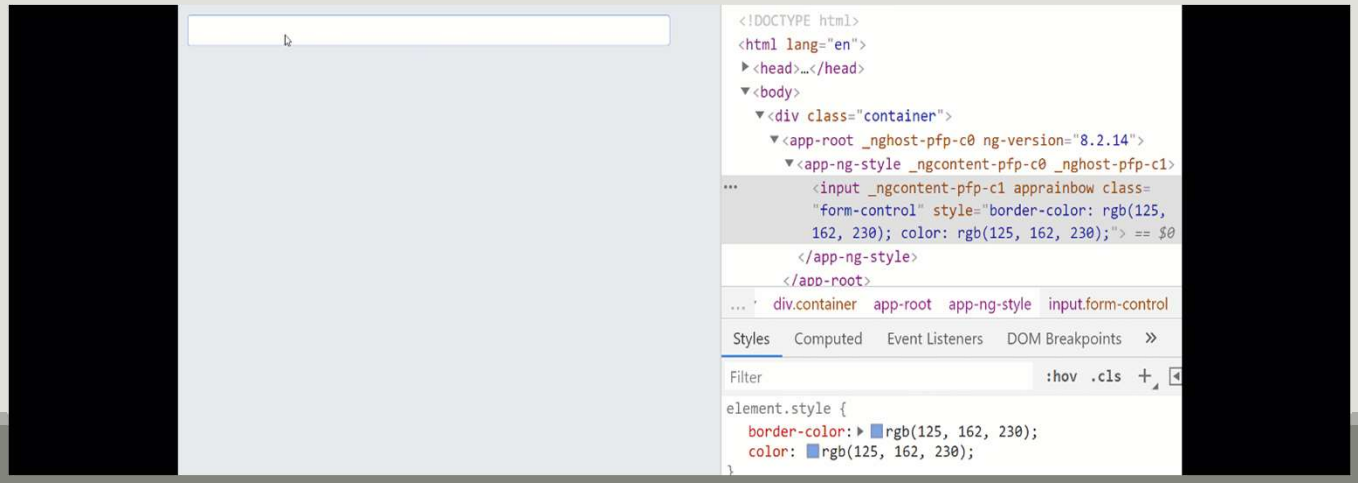
- Afin de créer sa propre « attribut directive » il faut utiliser un `HostBinding` sur la **propriété** que vous voulez **binder**.
  - **Exemple :**
```
@HostBinding('style.backgroundColor')
bg:string="red";
```
- Dans cet exemple on lie (bind) l'attribut **bg** de la directive à l'attribut **style.backgroundColor** de l'élément hôte (celui qui est tagué) de la directive. A chaque fois qu'on change **bg** le **style.backgroundColor** de l'élément hôte change.
- Si on veut associer un **événement** à notre directive on utilise un `HostListner` qu'on associe à un **événement** déclenchant une `méthode.`
  - **Exemple :**
```
@HostListener('mouseenter') mouseover(){
    this.bg =this.highlightColor;
}
```
- Afin d'utiliser le HostBinding et le HostListner il faut les importer du core d'angular

Exercice



Un truc plus sympa, on va créer un simulateur d'écriture arc en ciel.

- Créer une directive
- Créer un hostbinding sur la couleur et la couleur de la bordure.
- Préparer une méthode qui permet de générer aléatoirement une couleur dans votre directive.
- Faite en sorte qu'en appliquant votre directive à un input, à chaque écriture d'une lettre (event keyup) la couleur change en prenant aléatoirement l'une des couleurs de votre tableau. Pensez à utiliser Math.random() qui vous retourne une valeur entre 0 et 1.



Customiser une attribut directive

- Nous pouvons aussi utiliser le @Input afin de rendre notre directive paramétrable
- Tous les paramètres de la directive peuvent être mises en @Input puis récupérer à partir de la cible.
- Exemple
  - Dans la directive `@Input()` `private myColor:string="red";`
  - `<direct-direct [myColor]="gray">`

HostBinding et HostListener

- La solution la plus appropriée est de ne plus utiliser les décorateur @HostBinding et @HostListener et de plutôt **passer par l'attribut host de votre décorateur @Directive ou @Componet**.
- Cet attribut prend en paramètre un objet dans lequel vous pouvez utilisez le binding standard

```
@Directive({
  selector: '[appHighlight]',
  standalone: true,
  host: {
    '[style.backgroundColor]': 'this.color()',
    '(mouseenter)':'this.onMouseEnter()',
    '(mouseleave)':'this.onMouseLeave()',
  }
})
export class HighlightDirective {//…}
```

Angular
Les pipes

AYMEN SELLAOUTI

Objectifs

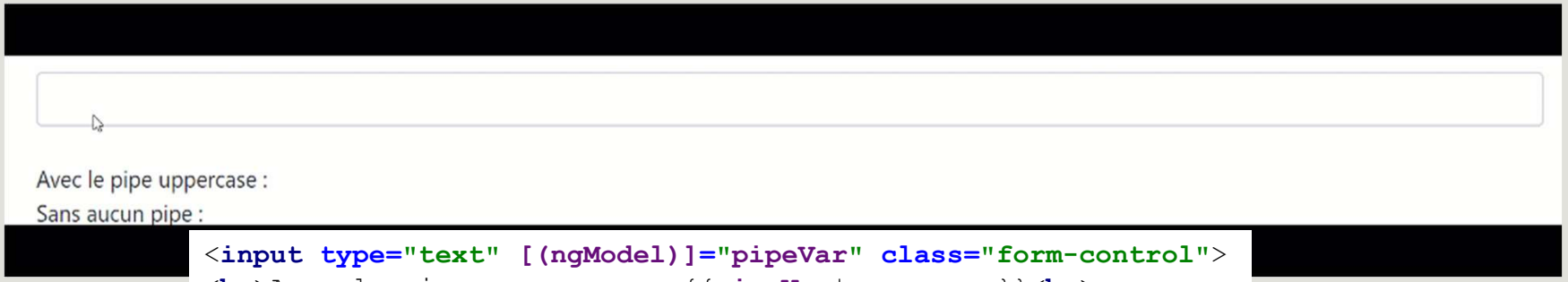
1. Définir les pipes et l'intérêt de les utiliser

2. Vue globale des pipes prédéfinies

3. Créer un pipe personnalisé

Qu'est ce qu'un pipe

- Un pipe est une fonctionnalité qui permet de formater et de transformer vos données avant de les afficher dans vos Templates.
- Exemple l'affichage d'une date selon un certain format.
- Il existe des pipes offerts par Angular et prêt à l'emploi.
- Vous pouvez créer vos propres pipes.




```
<input type="text" [(ngModel)]="pipeVar" class="form-control">
<br>Avec le pipe uppercase : {{pipeVar|uppercase}}<br>
Sans aucun pipe : {{pipeVar}}
```

Syntaxe

- Afin d'utiliser un pipe vous utilisez la syntaxe suivante :
  - {{ variable | **nomDuPipe** }}
- Exemple : {{ maDate | **date** }}
- Afin d'utiliser plusieurs pipes combinés vous utilisez la syntaxe suivante :
  - {{ variable | **nomDuPipe1** | **nomDuPipe2** | **nomDuPipe3** }}
- Exemple : {{ maDate | **date** | **uppercase** }}

Les pipes disponibles par défaut (Built-in pipes)

- La documentation d'angular vous offre la liste des pipes prêt à l'emploi.

https://angular.io/api?type=pipe

- uppercase
- lowercase
- titlecase
- currency
- date
- json
- percent
- …

Paramétrer un pipe

- Afin de paramétrer les pipes ajouter ‘:’ après le pipe suivi de votre paramètre.
  - {{ maDate | date:"MM/dd/yy" }}
- Si vous avez plusieurs paramètres c’est une suite de ‘:’
  - {{ nom | slice:1:4 }}

Pipe personnalisé

- Un pipe personnalisé est une classe décoré avec le décorateur @Pipe.
- Elle implémente l'interface PipeTransform
- Elle doit implémenter la méthode transform qui prend en paramètre la valeur cible ainsi qu'un ensemble d'options.
- La méthode transform doit retourner la valeur transformée
- Le pipe doit être déclaré au niveau de votre module de la même manière qu'une directive ou un composant.
- Pout créer un pipe avec le cli : ng g p nomPipe

Exemple de pipe

```ts
import { Pipe, PipeTransform } from
'@angular/core';

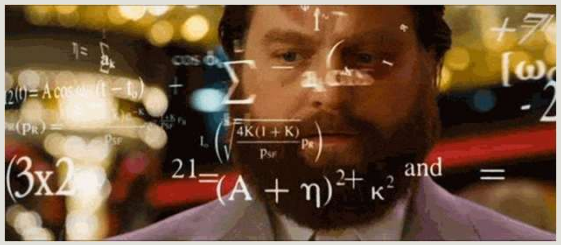
@Pipe({
  name: 'team',
})
export class TeamPipe implements PipeTransform
{
  transform(value: any, args?: any): any {
    switch (value) {
      case 'barca':
        return ' blaugrana';
      case 'roma':
        return ' giallorossa';
      case 'milan':
        return ' rossoneri';
    }
  }
}
```

```ts
<ol>
  @for ( team of teams() ; track team ) {
    <li>
      {{team | team}}
    </li>
  }
</ol>
```

```ts
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, TeamPipe],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {
  teams = signal(['milan', 'barca', 'roma']);
}
```

1. rossoneri
2. blaugrana
3. giallorossa

Exercice



Créer un pipe appelé defaultImage qui retourne le nom d'une image par défaut que vous stockerez dans vos assets au cas ou la valeur fournie au pipe est une chaine vide ou ne contient que des espaces.

Angular
Service et injection de dépendances



AYMEN SELLAOUTI

Objectifs

1. Définir un service
2. Définir ce qu'est l'injection de dépendance
3. Injecter un service
4. Définir la portée d'un service
5. Réordonner son code en utilisant les services

Comment fonctionne un restaurant ?



Injection de dépendance (DI)



- L'injection de dépendance est un patron de conception.

```
Classe A1{
 ClasseB b;
 ClasseC c;
 …
}
```

```
Classe A2{
 ClasseB b;
…
}
```

```
Classe A3{
ClasseC c;
 …
}
```

Que se passera t-il si on change quelque chose dans le constructeur de B ou C ?
Qui va modifier l'instanciation de ces classes dans les différentes classes qui en dépendent?

Injection de dépendance (DI)



- Déléguer cette tache à une entité tierce.

```
Classe A1{
 Constructor(B b, C c)
 …
}
```

```
Classe A2{
Constructor(B b)
…
}
```

```
Classe A3{
Constructor(C c)
 …
}
```

INJECTOR

Comment fonctionne un restaurant (système d'injection de dépendances) ?

Providers

Injector

IOC

Injection



Injection de dépendance (DI)

- Comment les injecter ?
- Comment spécifier à l'injecteur quel service et ou est-il visible ?

Injection de dépendance (DI)

- L'injection de dépendance utilise les étapes suivantes :
  - **Provider les dépendances (Préparer notre menu, définir quels plats nous pouvons fournir)** via **l'annotation @Injectable et son attribut providedIn** (ici les services), dans le **provider du module** ou du **composant** (**C'est la préparation de votre menu**).
  - **Injecter le service (commander le plat souhaité)** en le passant comme **paramètre du constructeur** de la **classe (composant ou service) qui en a besoin ou à partir de la version 14 utiliser la fonction inject (c'est la bonne pratique)**.

Injection de dépendance (DI)

Provider les dépendances et les injecter (Préparer notre menu, et commander)

ancienne

```
@NgModule({
  providers: [CvService],
  bootstrap: [AppComponent],
})
export class AppModule {}
```

```
@Injectable({
  providedIn: 'root',
})
export class CvService {}
```

Provider

Injecter

```
@Component({
  selector: 'app-cv',
  templateUrl: './cv.component.html',
  styleUrls: ['./cv.component.css'],
  providers: [CvService]
})
export class CvComponent {
```

```
export class CvComponent {
  constructor(
    private cvService: CvService
  ) {}
}
```

ancienne

```
export class CvComponent {
  private cvService = inject(CvService);
}
```

nouvelle

```
export const appConfig: ApplicationConfig = {
  providers: [ CvService ],
};
```

nouvelle

Les providers personnalisés
la fonction inject (après Angular 14)

- La fonction inject vous permet d'injecter un injectable.
- **Avant Angular 14** et à partir **d'Angular 9**, la fonction **inject** pouvait être utilisé uniquement dans la **factory** de **l'InjectionToken** ou dans le factory du **@Injectable**.
- A partir d'Angular 14, vous pouvez **l'utiliser dans tout le context d'injection de dépendances comme vos composants, directives et pipes**.
- Le premier intérêt est le **type safety** avec le décorateur @Inject.
- Il **facilite aussi l'héritage** en externalisant la dépendance du composant.
- **Faites attention quand vous utilisez inject elle doit être dans un contexte d'injection**

```
export class CvComponent {
  //  Donne moi le sayHelloService
  sayHelloService = inject(SayHelloService);
  cvService = inject(CvService);
}
```

Chargement automatique du service

- A partir de Angular 6 vous pouvez ne plus utiliser le provider du module afin de charger votre service mais le faire directement au niveau du service à travers l'annotation **@Injectable** et sa propriété **providedIn**. Vous pouvez charger le service dans toute l'application via le mot clé root. Ceci permet **d'optimiser votre code** via :

- **Lazy loading** : N'instancie un service que s'il est injecté au moins une fois

- Permettre le **Tree-Shaking** des services non utilisés : Si le service n'est jamais utilisé, son **code sera entièrement retiré du build final**.

```
@Injectable({
  providedIn: 'root',
})
export class CvService {}
```

@Injectable

- C'est un décorateur permettant de rendre une classe injectable
- Une classe est dite injectable si on peut y injecter des dépendances
- @Component, @Pipe, et @Directive sont des sous classes de @Injectable(), ceci explique le fait qu'on peut y injecter directement des dépendances.
- Si vous n'aller injecter aucun service dans votre service, cette annotation n'est plus nécessaire.
- **Remarque** : Angular conseille de toujours mettre cette annotation.

Injection de dépendance (DI)
Avantages

- L'intérêt de l'injection de dépendance est donc :
  - **Couplage lâche**
  - Facilement **remplacer une implémentation** d'une dépendance à des fins de test
  - Prendre en charge **plusieurs environnements** d'exécution
  - Fournir de **nouvelles versions d'un service** à un tiers qui utilise votre service dans sa base de code, etc.

Standalone Component
Les composants autonomes
Configurez l'injection de dépendance

- Afin de fournir un **provider** pour **toute l'application**, vous pouvez ajouter en **deuxième paramètre de la fonction bootstrapApplication**, un **objet d'options**.
- Cet objet contient une **clé providers** qui prend en paramètre un **tableau de providers**.
- Vous pouvez aussi **récupérer des providers** offerts par un **module** avec la fonction **importProvidersFrom(moduleCible)**.
- A partir de la **version 15**, vous avez **plusieurs fonctions spécifiques** pour les **modules** les **plus utilisés** comme le **http** avec sa fonction **provideHttpClient(), ou provideRouter(APP_ROUTES).**

Standalone Component
Les composants autonomes
Configurez l'injection de dépendance

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(
      routes,
      withComponentInputBinding()
    ),
    provideHttpClient(
      withInterceptorsFromDi()
    ),
    importProvidersFrom(
      TranslateModule.forRoot({
        loader: {
          provide: TranslateLoader, useFactory: HttpLoaderFactory,deps: [HttpClient],
        },
      })
    ),
  ],
};
```

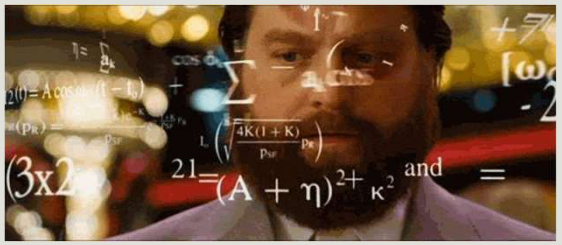
Standalone Component
Les composants autonomes
Créer un service

- Afin de créer un service, vous pouvez passer par la commande

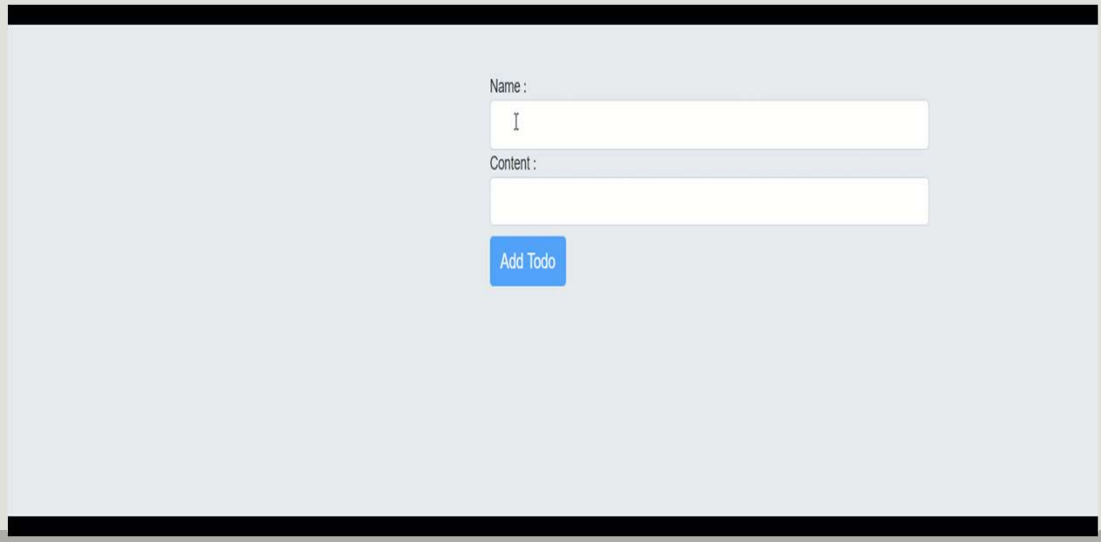
ng g s nomService

- Créer manuellement une classe et de la fournir
  - en ajoutant @Injectable({'providedIn': 'root'})
  - ou en la providant via un provider

Exercice



- Créons un service de Todo, le Model et le composant qui va avec. Un Todo est caractérisé par un nom et un contenu.
- Ce service permettra de faire les fonctionnalités suivantes :
  - Logger les todos
  - Ajouter un Todo
  - Récupérer la liste des Todos
  - Supprimer un Todo



DI Hiérarchique

- Dans Angular vous disposez de plusieurs endroits où vous pouvez définir les fournisseurs pour vos dépendances :
  - Module
  - Composant
  - Directive !

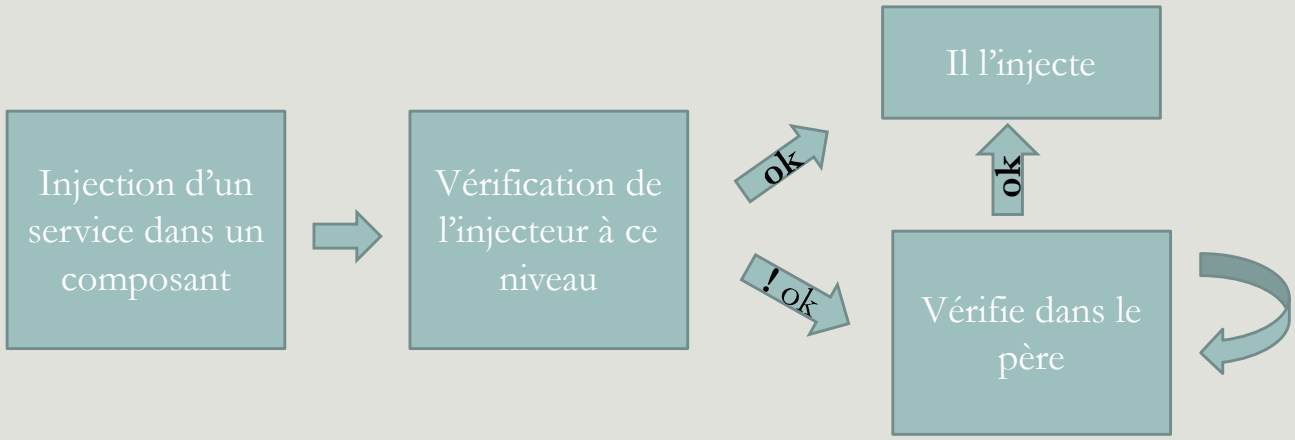
DI Hiérarchique

- Le système d'injection de dépendance d'Angular est **hiérarchique**
- Il possède **deux hiérarchies d'injecteur**
  - Une **hiérarchie d'injecteur niveau composant** (**element Injector Hierarchy**)
  - Une hiérarchie d'injecteur **niveau module**.

- La **hiérarchie composant** (appelé aussi **Node Injector Hierarchy**) est la plus prioritaire

DI Hiérarchique
Hiérarchie d'injecteur niveau composant

- Le système d'injection de dépendance d'Angular est hiérarchique.
- Un arbre d'injecteur est créé. Cet arbre est // à l'arbre de composant.
- L'algorithme suivant permet la détection de l'injecteur adéquat :



Tant que il ne le trouve pas et qu'on est pas arrivé dans la racine

DI Hiérarchique
Hiérarchie d'injecteur niveau Module

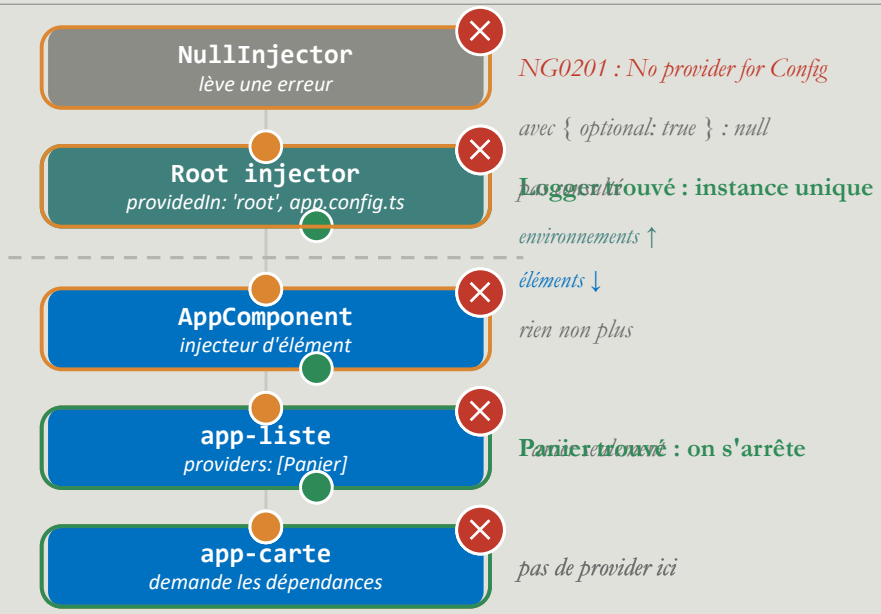
- Si Angular **ne trouve pas de provider** au niveau de la **hiérarchie des composants**, il va aller voir dans la **hiérarchie de module**.
- Le ModuleInjector peut être configuré de deux manières en utilisant :
  - La propriété @Injectable pour référencer un **NgModule**, ou '**root**'
  - Le tableau des providers dans @NgModule()
- Le moduleInjector **identifie les providers disponibles** en effectuant un **aplatissement de tous les tableaux de fournisseurs** qui peuvent être atteints en suivant **les NgModule.imports de manière récursive**.

Résolution : inject() remonte l'arbre des injecteurs

```ts
@Injectable({ providedIn: 'root' })
export class Logger {}

@Component({
  selector: 'app-liste',
  providers: [Panier],
})
export class Liste {}

@Component({ selector: 'app-carte' })
export class Carte {
  logger = inject(Logger);
  panier = inject(Panier);
  config = inject(Config);
}
```



Angular part de l'injecteur du composant qui demande, puis remonte de parent en parent : le premier provider trouvé gagne. Après les composants viennent les injecteurs d'environnement, puis `NullInjector` et son erreur.

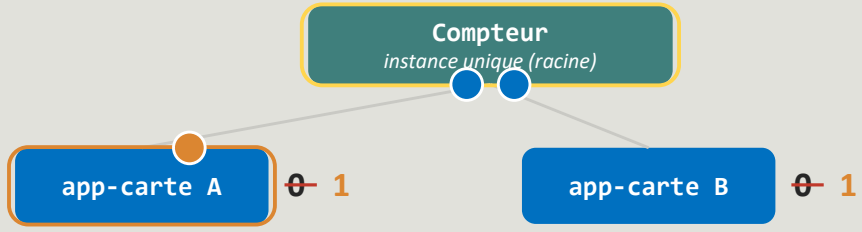
Une instance par injecteur : partagée ou isolée

```ts
@Injectable({ providedIn: 'root' })
export class Compteur {
  valeur = signal(0);
  plus() {
    this.valeur.update(v => v + 1);
  }
}

// cas 1 : rien dans le composant
@Component({ selector: 'app-carte' })

// cas 2 : une instance par carte
@Component({
  selector: 'app-carte',
  providers: [Compteur],
})
```

cas 1 : providedIn: 'root', une seule instance



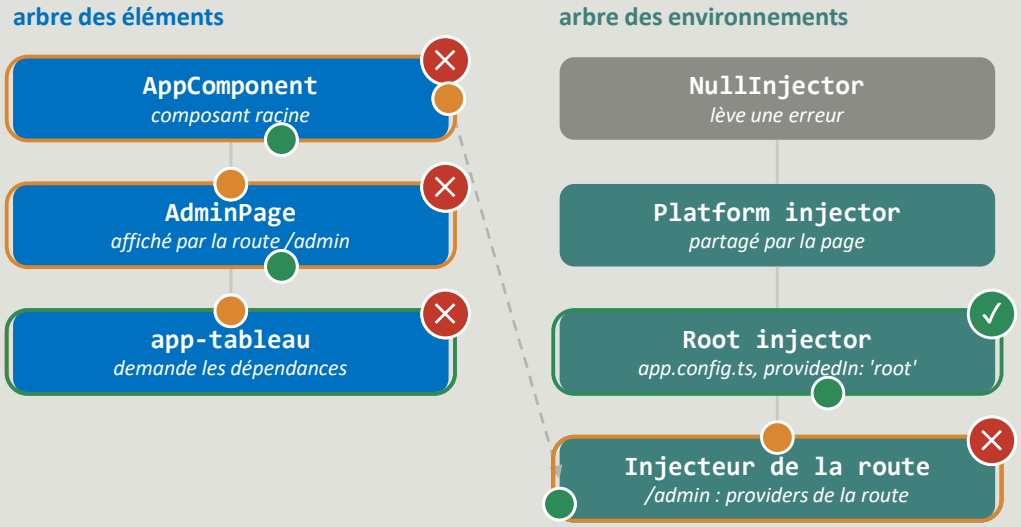
les deux cartes lisent la même instance

cas 2 : providers: [Compteur], une instance par carte



Une instance par injecteur qui fournit le service : à la racine, une seule pour toute l'application ; dans `providers` d'un composant, une par instance du composant, créée et détruite avec lui.

Deux hiérarchies : les éléments, puis les environnements



le code

```
// app.config.ts
providers: [provideHttpClient()]

// app.routes.ts
{
  path: 'admin',
  providers: [AdminApi],
  loadComponent: () =>
    import('./admin-page'),
}

// tableau.ts
api = inject(AdminApi);
http = inject(HttpClient);
```

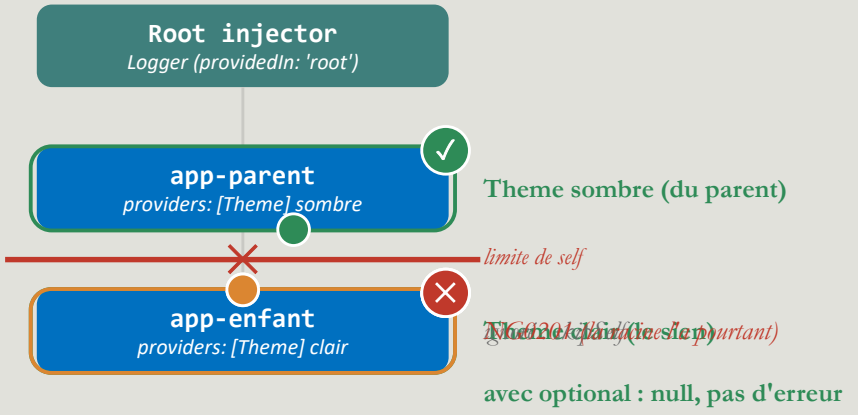
AdminApi : la route admin AppComponent ; route pour la racine, trouvé via le provider HttpClient de la route / admin

L'ordre : 1. les injecteurs d'éléments, du composant jusqu'au composant racine ; 2. les injecteurs d'environnement : route, racine, plateforme ; 3. `NullInjector` : erreur `NG0201`.

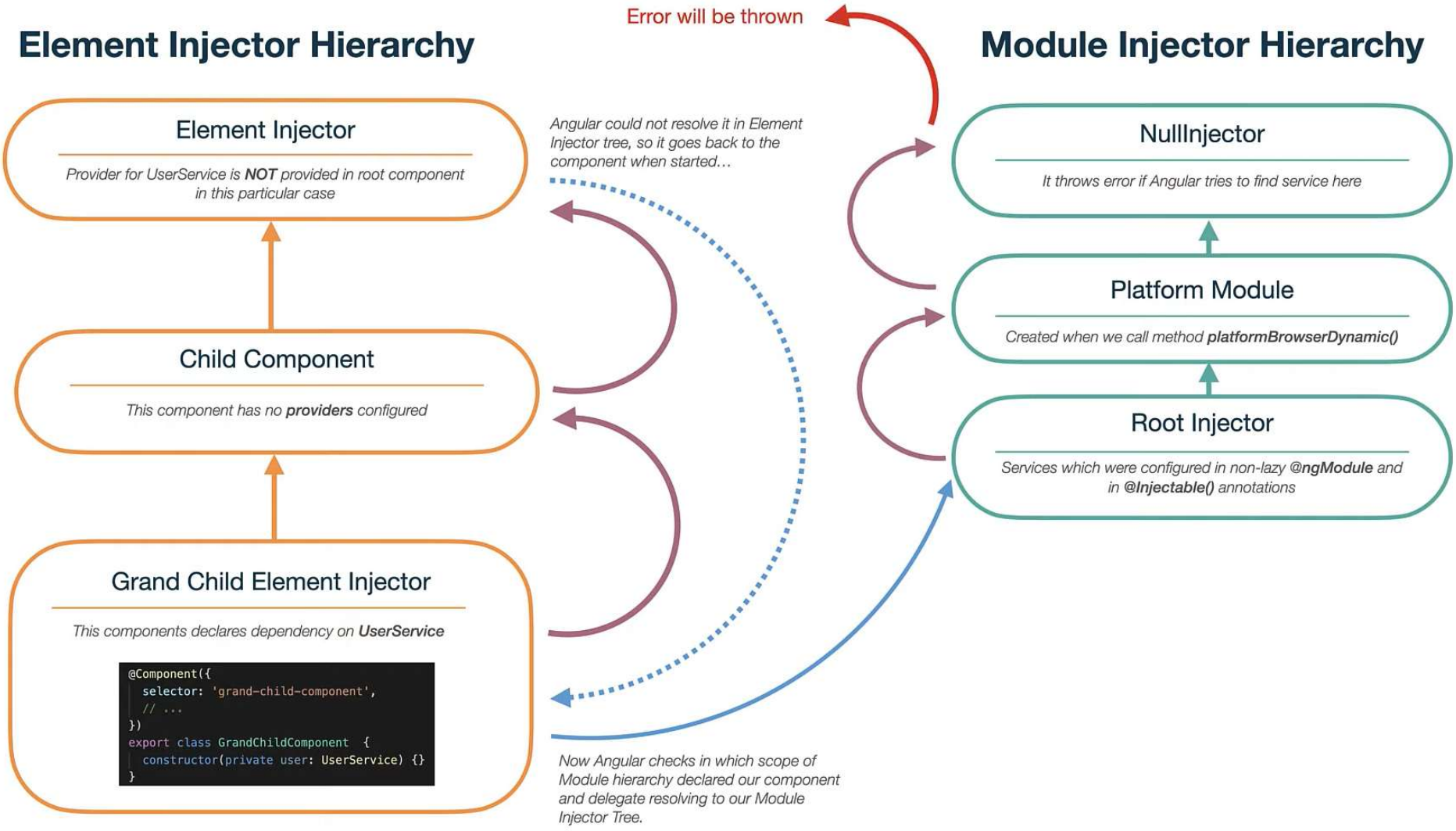
Les options de inject() : où commencer, où s'arrêter

```
// app-parent : providers: [Theme]  (sombre)
// app-enfant : providers: [Theme]  (clair)
// Logger : providedIn: 'root'

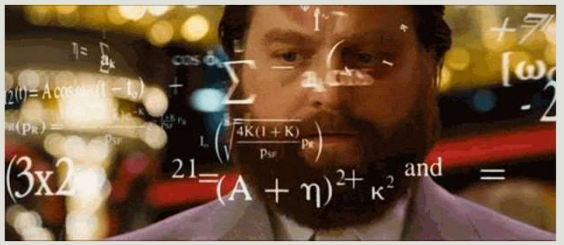
// dans app-enfant :
inject(Theme);
inject(Theme, { skipSelf: true });
inject(Logger, { self: true });
inject(Logger, { self: true, optional: true });
```



`self` : seulement l'injecteur du composant · `skipSelf` : commencer au parent · `host` : s'arrêter au composant hôte · `optional` : `null` au lieu d'une erreur. Sans option : du composant jusqu'à `NullInjector`.

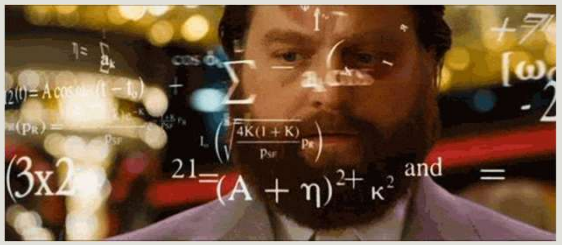

Element Injector Hierarchy
Error will be thrown
Module Injector Hierarchy
Element Injector
Provider for UserService is NOT provided in root component in this particular case
Angular could not resolve it in Element Injector tree, so it goes back to the component when started…
NullInjector
It throws error if Angular tries to find service here
Child Component
This component has no providers configured
Platform Module
Created when we call method platformBrowserDynamic()
Root Injector
Services which were configured in non-lazy @ngModule and in @Injectable() annotations
Grand Child Element Injector
This components declares dependency on UserService
@Component({
selector: 'grand-child-component',
// ...
})
export class GrandChildComponent {
constructor(private user: UserService) {}
}
Now Angular checks in which scope of Module hierarchy declared our component and delegate resolving to our Module Injector Tree.

Exercice



- Ajouter les services suivants afin d'améliorer la gestion de notre plateforme d'embauche.
  - Un premier service CvService qui gérera les Cvs. Pour le moment c'est lui qui contiendra la liste des cvs que nous avons.
  - Ajouter aussi un composant pour afficher la liste des cvs embauchées ainsi qu'un service EmbaucheService qui gérer les embauches.
  - Au click sur le bouton embaucher d'un Cv, le cv est ajoutés à la liste des personnes embauchées et une liste des embauchées apparait.

Exercice



CvTech Home Cv Add Cv Todo Mini word Color Rxjs Logout Francais English

Mahdi Chaabane

Mehdi fkih

Aymen Sellaouti

"To be or not to be, this is my awesome motto!"

Job Description

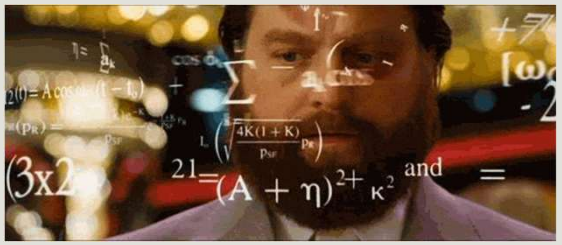
Web design, Adobe Photoshop, HTML5, CSS3, Corel and many others...

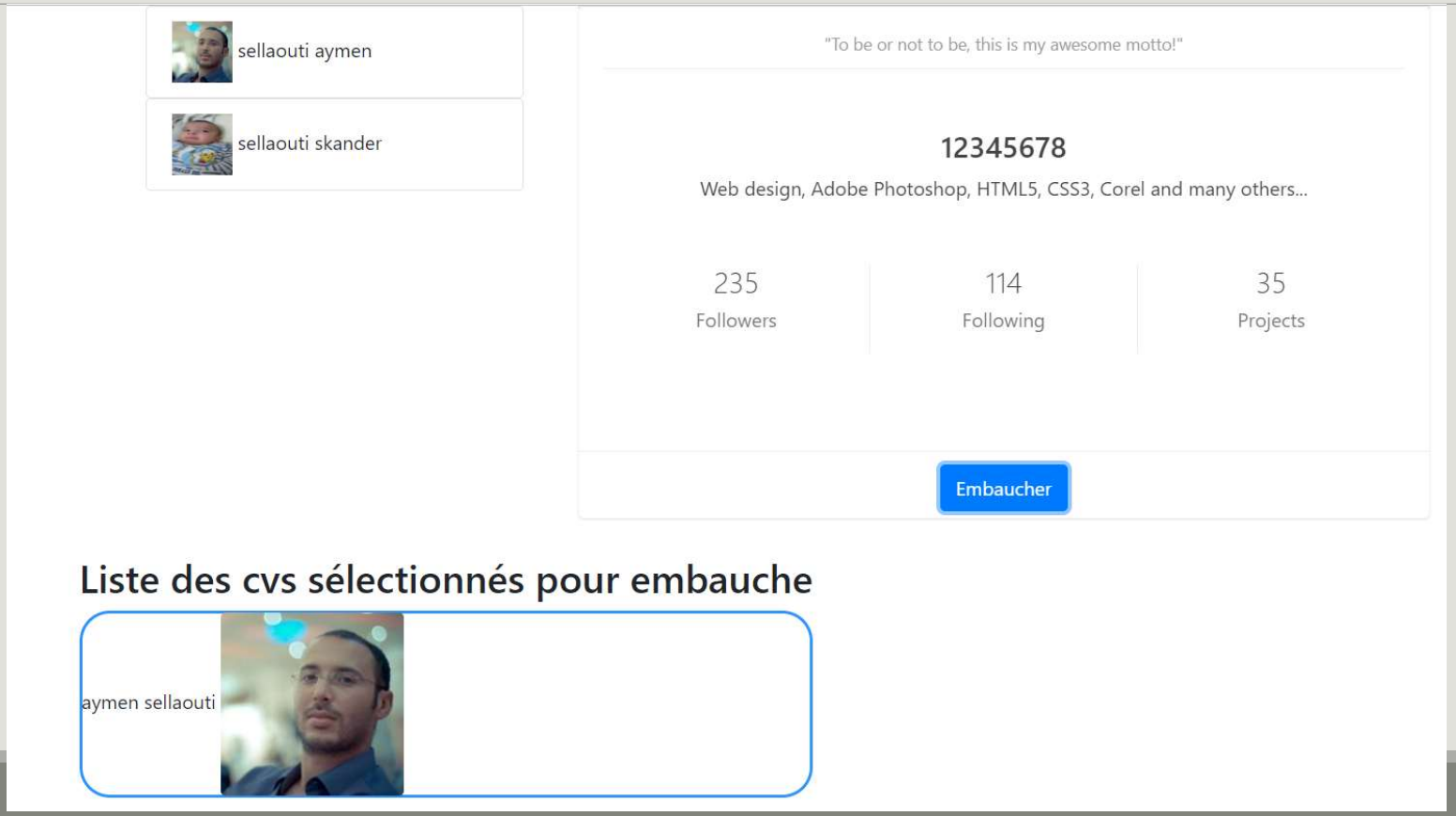
235 Followers | 114 Following | 35 Projects

embaucher détails

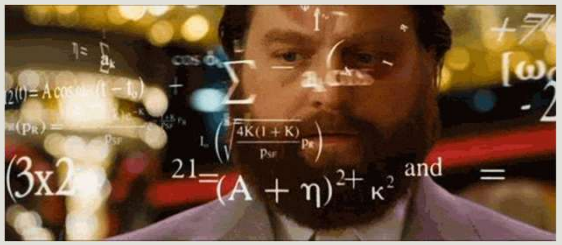
Footer

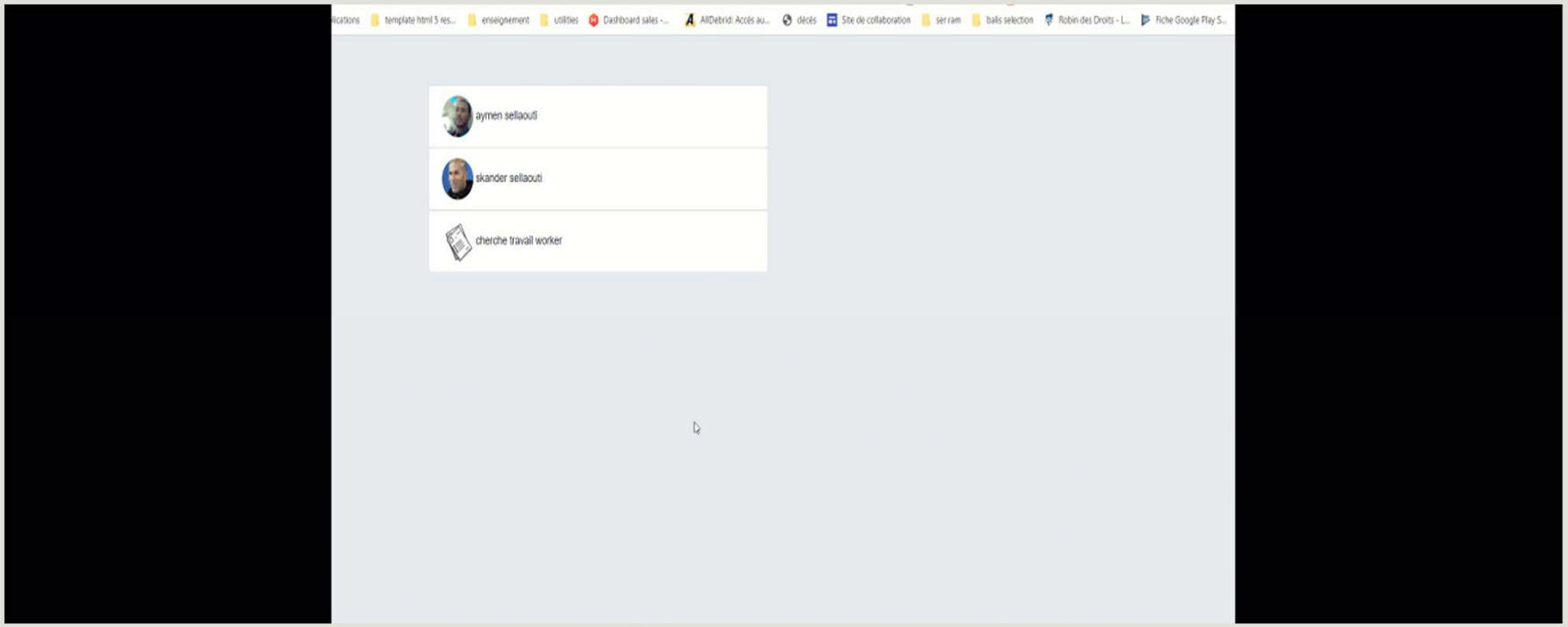
Exercice




sellaouti aymen
sellaouti skander
"To be or not to be, this is my awesome motto!"
12345678
Web design, Adobe Photoshop, HTML5, CSS3, Corel and many others...
235
Followers
114
Following
35
Projects
Embaucher
Liste des cvs sélectionnés pour embauche
aymen sellaouti

Exercice




aymen sellaouti
skander sellaouti
cherche travail worker

Angular Routing

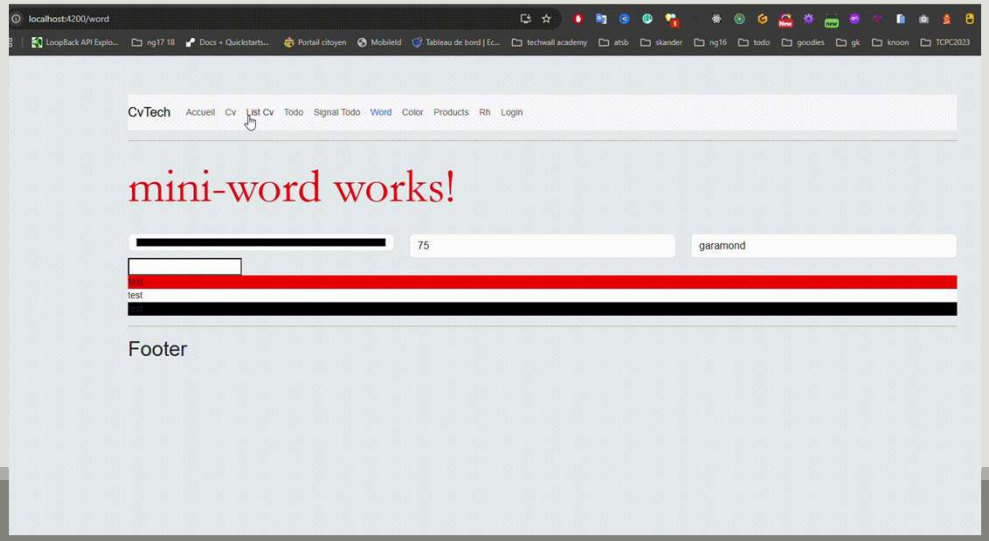
AYMEN SELLAOUTI

Objectifs

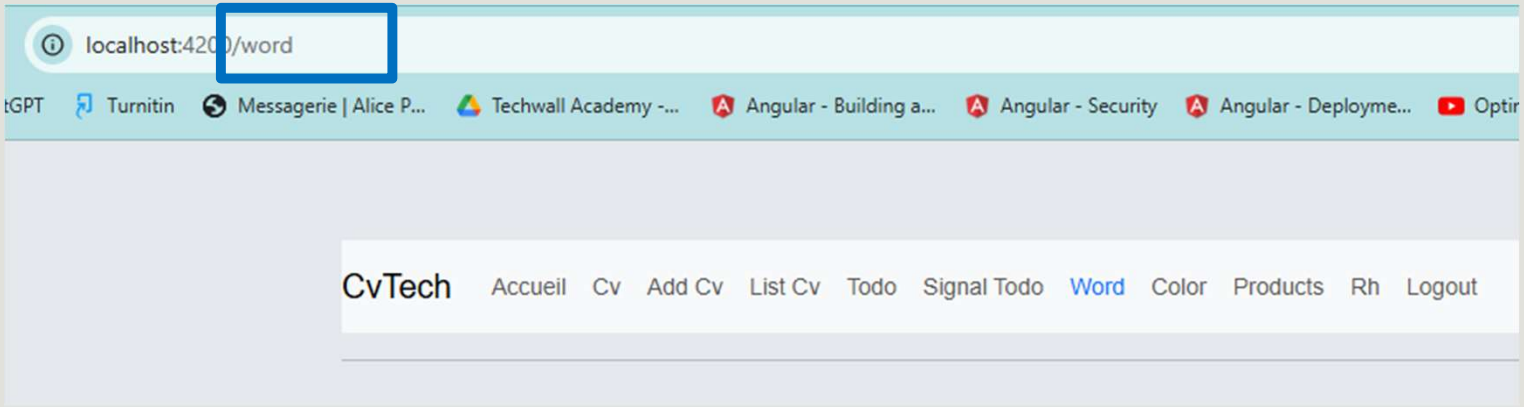
1. Définir le routeur d'Angular
2. Définir une route
3. Déclencher une route à partir d'un composant
4. Ajouter des paramètres à une route
5. Récupérer les paramètres d'une route à partir du composant.
6. Préfixer un ensemble de routes
7. Gérer les routes inexistantes

Comment fonctionne le système de routing d'Angular et quels sont ses composantes

- Quand vous **naviguez dans un site web**, c'est votre URI qui vous permet de demander **quelle page (ressource) afficher**.
- A chaque fois que vous changer d'URI, Angular détecte ce changement et essaye de détecter à quoi correspond l'URI demandée. Pour ce faire nous avons besoin de deux choses :
  - Un **responsable** de cette tâche : c'est le **RouterModule**
  - De définir cette **relation URI, Composant** pour que le RouterModule sache quoi faire : c'est votre **route**.



Création d'un système de Routing Application modulaire




```
@NgModule({
  imports: [
    RouterModule.forRoot(routes),
  ],
  exports: [RouterModule],
})
export class AppRoutingModule {}
```

```
const routes: Routes = [
  { path: '', component: FirstComponent },
  { path: 'word', component: MiniWordComponent },
  { path: 'color', component: ColorComponent },
];
```

Syntaxe minimaliste d'une route

- Une route est un objet.
- Les deux propriétés essentielles sont path et component.
- Path représente l'URI
- component permet de spécifier le composant à exécuter.

```
const routes: Routes = [
  { path: '', component: FirstComponent },
  { path: 'word', component: MiniWordComponent },
  { path: 'color', component: ColorComponent },
];
```

Création d'un système de Routing Application Standalone

- Afin de créer un système de routing, il suffit de provider vos routes avec la fonction **provideRouter** au niveau de la fonction **bootstrapApplication.**
- **Vous ne devez plus importez le RouterModule, c'est la fonction provideRouter qui s'en charge.**

```
bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

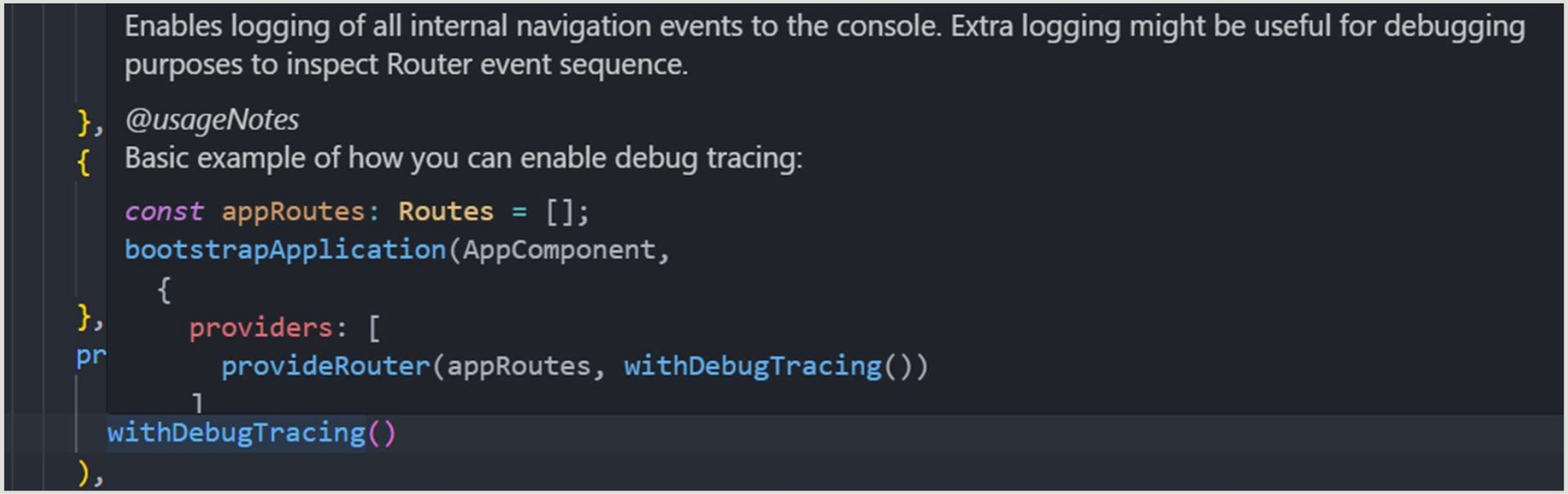
```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes)
 ]
}
```

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
]
```

Création d'un système de Routing Application Standalone provideRouter

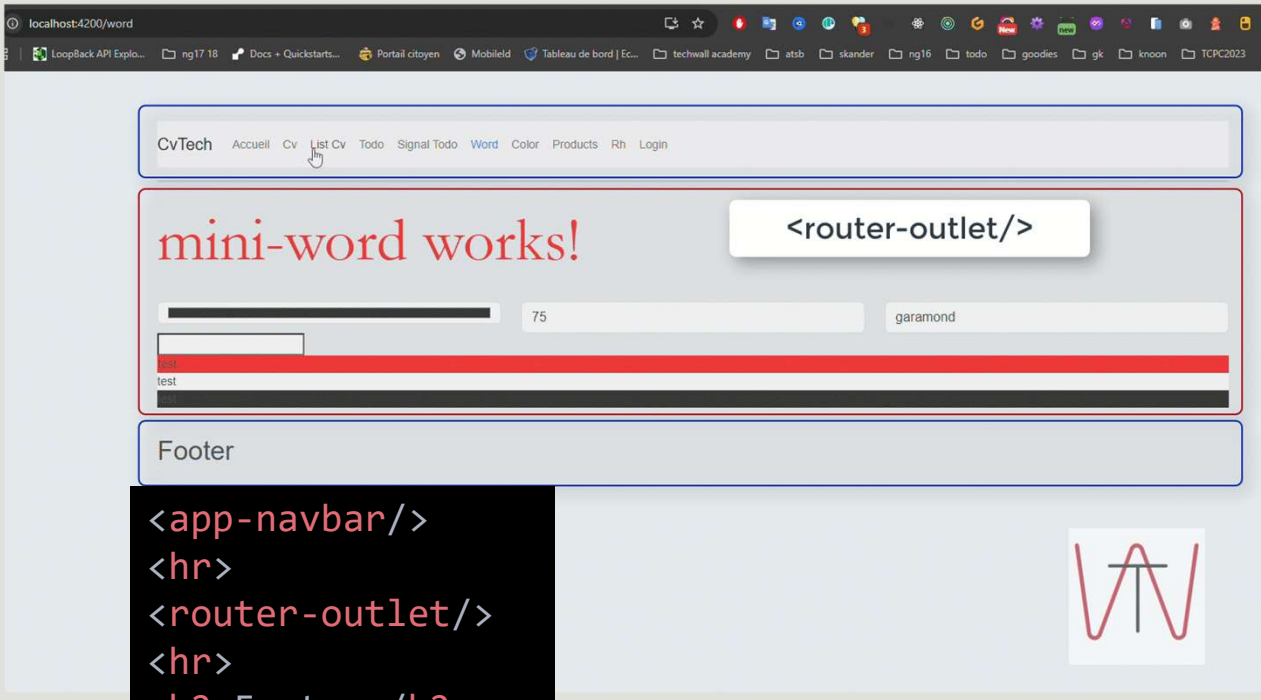
- La fonction **provideRouter ne prend pas uniquement en charge les routes** mais elle permet aussi **l'implémentation des fonctionnalités additionnelles du routeur** et qui étais définies auparavant dans le **RouterModule** via la méthode **forRoot**.
- Les options introduites dans les fonctions **provide**QuelqueChose suivent aussi un pattern de nommage : **with**QuelqueChose
- **with**DebugTracing(), permet par exemple de déclencher le **débogage des évènements du routeur.**
- Un autre avantage de ce pattern est de **faciliter le tree shaking** et donc **l'élagage du code mort** qui est plus simple avec les fonctions

Création d'un système de Routing Application Standalone provideRouter



Préparer l'emplacement d'affichage des vues correspondantes aux routes

- Généralement dans vos sites ou applications Web vous avez un template que vous suivez.
- Ce template contient une **partie statique** et une **partie variable**.
- La partie variable sera celle ou Angular devra affiché le composant associé à la route.
- Pour indiquer cette partie variable vous devez utilisez la directive <router-outlet/>.




```
<app-navbar/>
<hr>
<router-outlet/>
<hr>
<h2>Footer</h2>
```

Création d'un système de Routing Application Standalone

- Maintenant et dans les **différents composants** qui vont utiliser les **éléments du module de routing**, vous avez deux choix.
- **Importer le RouterModule**
- Ceci fonctionne **vous pouvez utiliser le router-outlet**, mais **ce n'est pas la meilleure façon de faire**, ca n'explique pas de quoi dépend exactement votre AppComponent.

```
@Component({
  selector: 'app-root',
  template: `
    <router-outlet></router-outlet>
  `,
  styleUrls: ['./app.component.css'],
  imports: [RouteModule],
  standalone: true,
})
export class AppComponent {
```

Création d'un système de Routing Application Standalone

- **Importer uniquement ce dont vous dépendez**, **ce que vous avez besoin d'utiliser**, n'oubliez pas il n'y a pas que les Standalone Components, on parle aussi de **Standalone Directive et Pipe** et c'est le même modèle mental.

```
@Component({
  selector: 'app-root',
  template: `
    <router-outlet></router-outlet>
  `,
  styleUrls: ['./app.component.css'],
  imports: [RouterOutlet],
  standalone: true,
})
export class AppComponent {
```

Déclencher une route routerLink
routerLink : DIRECTIVE VS PROPERTY

- **RouterLink** a deux variantes : une directive et une property de la directive.
  1. On opte pour la directive quand la route cible est statique (sans paramètres).
  2. On fait du property binding quand la route cible est dynamique.

```
@Component({
  selector: 'app-root',
  template: `
    <a [routerLink]="['cv', cv.id]">route dynamique</a>
    <a routerLink="cv">route statique</a>
    <router-outlet></router-outlet>`,
  styleUrls: ['./app.component.css'],
  imports: [RouterOutlet, RouterLink],
  standalone: true,
})
export class AppComponent {
```

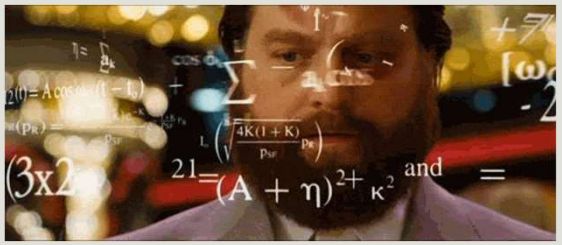
Créer des slices de routes

- Si vous utilisez les standalones components et afin **d'organisez vos routes par fonctionnalité (feature),** vous pouvez créer des **slices de routes** et les **appeler.**

```
export const FEATURE_ROUTES = [
  { path: 'featurehome', component: FeatureHomeComponent},
  { path: 'otherfeaturehome', component: OtherFeatureHomeComponent },
]
```

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  ...FEATURE_ROUTES,
]
```

Exercice
slices de routes



- Faites ceci pour les routes de votre feature Cv

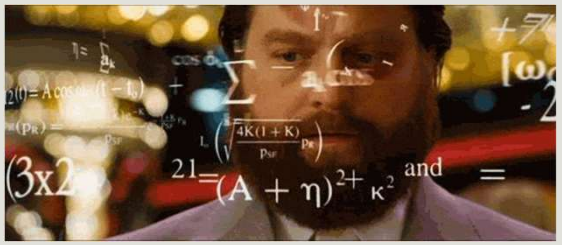
Déclencher une route routerLink

- Afin de **mettre en avant la route sélectionnée par l'utilisateur** dans votre menu, vous pouvez utilisez la directive **routerLinkActive**

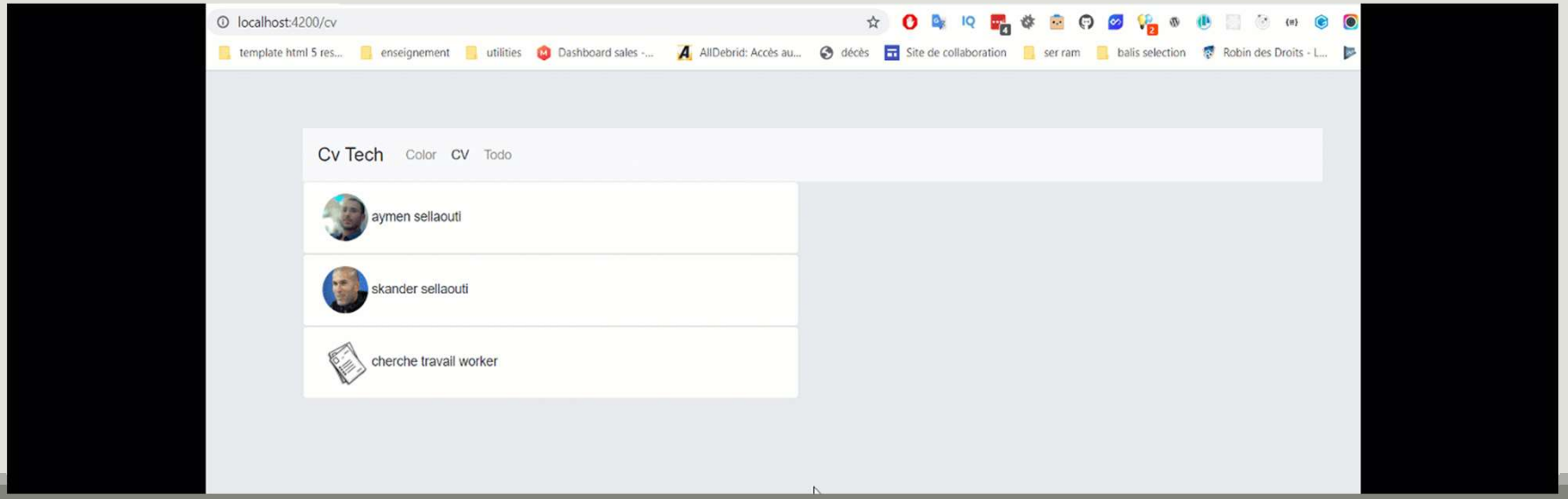
- **routerLinkActive** prend en paramètre la **liste des classes CSS** que vous voulez **appliquez** à la **route sélectionnée ainsi qu'à tous ces ses ancêtres**.
- Par exemple si on a l'URI **/cv/liste** les classes de **routerLinkActive** seront ajoutées à cet URI ainsi qu'à l'URI **/cv** et **/**.
- Pour identifier uniquement l'uri cible, ajouter la directive **routerLinkActive** avec la valeur **{exact: true}**

```
<a class="nav-item nav-link"
routerLinkActive="active text-primary"
[routerLinkActiveOptions]="{exact: true}"
[routerLink]="['']">Accueil</a>
```

Exercice



- Faites en sorte d'avoir un composant Header dans votre application qui permet d'afficher l'ensemble de vos liens.
- En cliquant sur un lien, le composant qui lui est associé doit être affiché.



Déclencher une route à partir du composant

- Afin de déclencher une route à travers le composant on utilise le service **Router** et sa méthode **navigate**.
- Cette méthode prend le **même paramètre** que le routerLink, à savoir un tableau contenant la description de la route.
- Afin d'utiliser le Router, il faut l'importer de l'@angular/router et l'injecter dans votre composant.

```
import { ActivatedRoute, Router } from "@angular/router";
router = inject(Router);
onLoadCv(id: number) {
  // méthode conseillée
  this.router.navigate(['cv', id]);
  //alternative
  this.router.navigate([`cv/${id}`]);
}
```

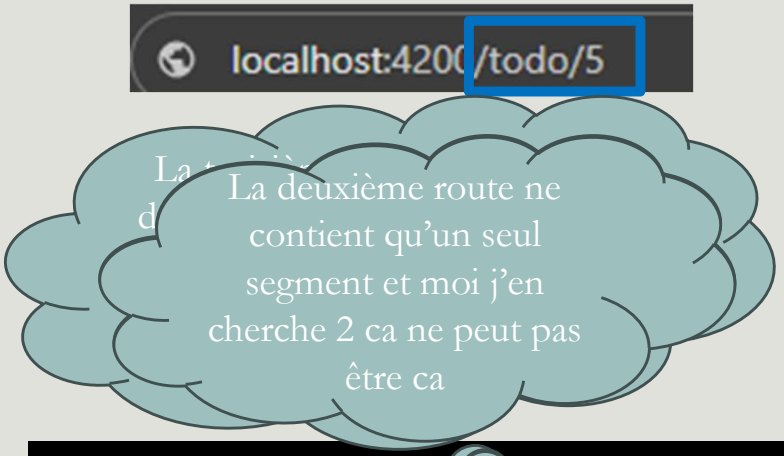
Les paramètres d'une route

- Afin de spécifier à notre router qu'un segment d'une route est un paramètre, il suffit d'y ajouter '**:**' devant le nom de ce segment.

- Exemple
  - **/cv/:id** permet de dire que la root contient **au début cv** ensuite un **paramètre de root appelé id**.

```
const routes: Routes = [
  { path: '', component: FirstComponent },
  { path: 'todo/:id', component:TodoDetailsComponent },
  { path: 'color', component: ColorComponent },
];
```

Création d'un système de Routing



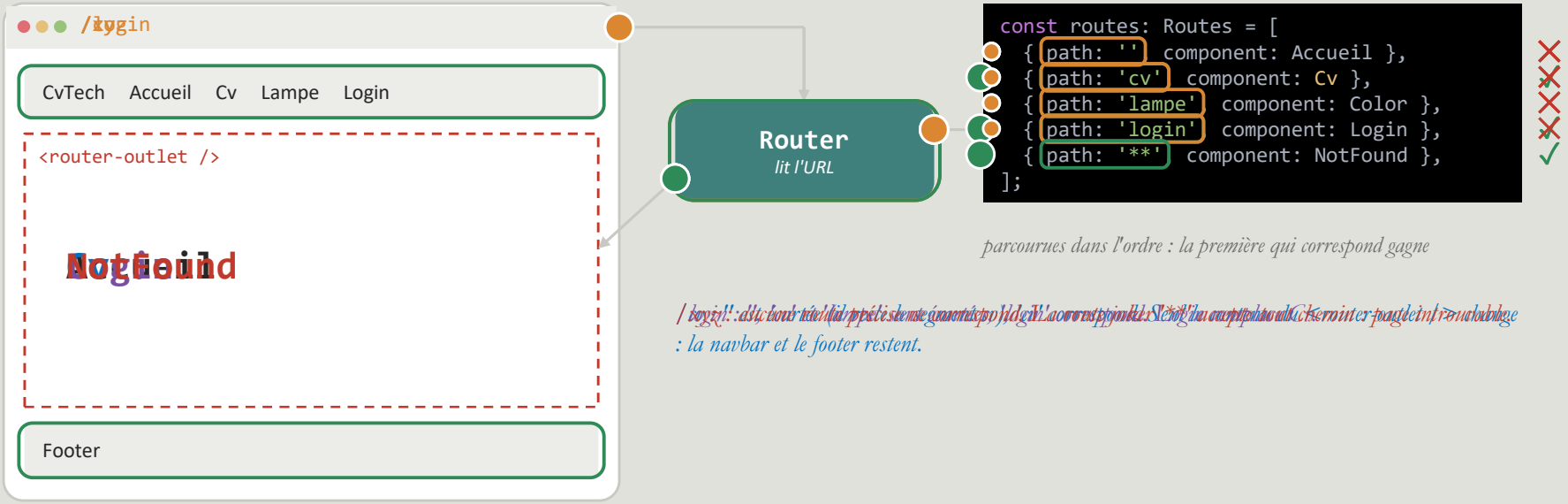
Je cherche une route avec deux segments
1- Le premier segment si c'est une constante il doit être todo sinon si c'est une variable peut contenir n'importe quoi
2- Le deuxième segment si c'est une constante il doit être 5, sinon si c'est une variable, il peut contenir n'importe quoi

```
@NgModule({
  imports: [
    RouterModule.forRoot(routes),
  ],
  exports: [RouterModule],
})
export class AppRoutingModule {}
```

```
const routes: Routes = [
  { path: 'todo/5'    , component: FirstComponent },
  { path: 'color'     , component: ColorComponent },
  { path: 'todo/:id', component:TodoDetailsComponent },

];
```

De l'URL au composant affiché



Le `Router` lit l'URL, essaie les routes dans l'ordre et retient la première qui correspond. Son composant s'affiche dans `<router-outlet />` ; le reste du template ne change pas. `'**'` accepte tout : elle se place en dernier.

Correspondance segment par segment

```ts
const routes: Routes = [
  { path: 'cv', component: CvList },
  { path: 'cv/add', component: AddCv },
  { path: 'cv/:id', component: DetailCv },
  { path: '**', component: NotFound },
];
```

segments de l'URL	cv	42	
`'cv'`	cv	–	✗ il reste le segment 42
`'cv/add'`	cv	add	✗ add ≠ 42
`'cv/:id'`	cv	:id	✓ :id accepte 42 : id = '42'

cv/42

```
<router-outlet />
```

DetailCv

```
id = '42'
```

Le RouterLink développe en interne un segment Routes (splits 42 pour chaque parent Map) pour créer un ActivatedComponentInputBinding().

Les routes sont essayées dans l'ordre, segment par segment ; une route sans enfants doit consommer toute l'URL. Un segment `:id` accepte toute valeur et la capture. Placez `'cv/add'` avant `'cv/:id'`, sinon add serait lu comme un id.

Récupérer les paramètres d'une route

- Afin de récupérer les paramètres d'une route au niveau d'un composant on doit procéder comme suit :
  1. Importer **ActivatedRoute** qui nous permettra de récupérer les paramètres de la route de "@angular/router".
  2. **Injecter** ActivatedRoute au niveau du composant.
  3. Utilisez l'objet **snapshot**

```
import { ActivatedRoute } from "@angular/router";

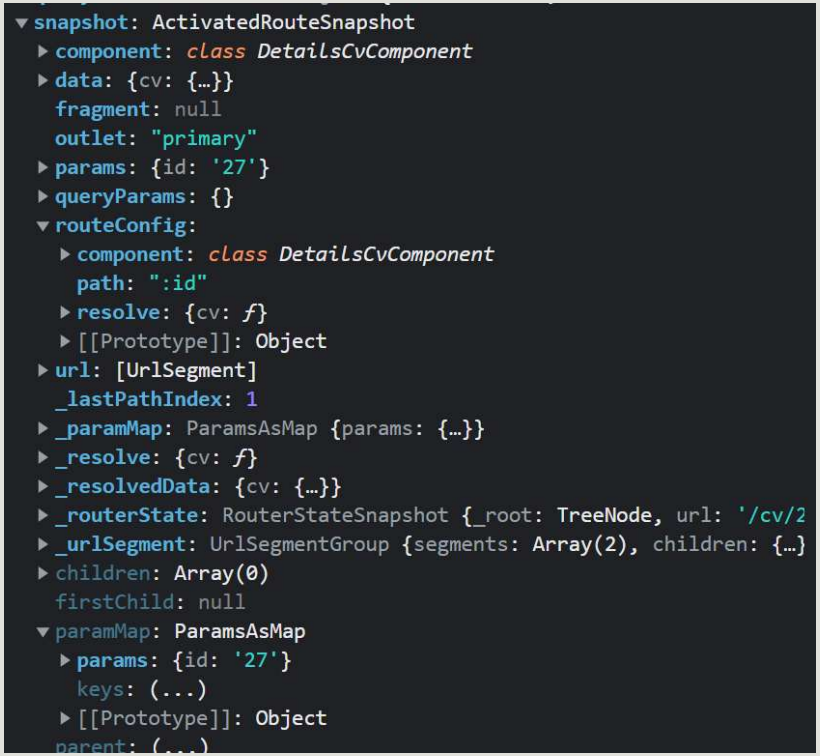
export class DetailsCvComponent {
  acr = inject(ActivatedRoute);
constructor() {
    this.acr.snapshot.params;
}
```

Récupérer les paramètres d'une route
ActivatedRoute / snapshot

- La propriété **snapshot** de l'objet **ActivatedRoute** contient un instantané de l'état de la route actuelle.
- Elle contient des **informations** telles que **l'URL actuelle**, **les paramètres de route actuels,**…
- Elle est généralement utilisée pour **accéder aux informations de route** dans un composant **lors de son initialisation**.
- Il est important de noter que **l'instantané de la route ne change pas lorsque la route change**. Il représente un **état figé** de la route lors de son instanciation.

Récupérer les paramètres d'une route
ActivatedRoute / snapshot

- Voici quelques propriétés courantes de l'API snapshot :
  - **url**: Retourne l'URL de la route actuelle sous forme de tableau de segments d'URL.
  - **params**: Retourne un objet qui contient les paramètres de route actuels.
  - **queryParams**: Retourne un objet qui contient les paramètres de requête actuels.
  - **fragment**: Retourne la partie de l'URL après le symbole "#".
  - **data**: Retourne les données de route associées à la route actuelle.
  - **component**: Retourne le composant de route actuel.
  - **routeConfig**: Retourne la configuration de la route actuelle.



Récupérer les paramètres d'une route
ActivatedRoute / snapshot

- Donc pour accéder à votre propriété, passez par l'objet snapshot
- Avec snapshot, vous avez deux méthodes pour récupérer les paramètres:
- Via la **propriété params** qui retourne un tableau d'objet des paramètres
- Via la propriété **paramMap**
  - Appeler sa méthode **get**
  - Passez lui le nom de la propriété souhaitée.




```
import { ActivatedRoute } from "@angular/router";

export class DetailsCvComponent {
  acr = inject(ActivatedRoute);
  constructor() {
    console.log({'snapshot.params':this.acr.snapshot.params});
}
```

Récupérer les datas d'un resolver via l'Input withComponentInputBinding

- A partir d'angular 16, vous pouvez récupérer les informations sur vos paramètres de route sans passer par l'injection du service ActivatedRoute.
- Vous devez tout d'abord activer cette option

```
// Application Modulaire
imports: [RouterModule.forRoot(routes, { bindToComponentInputs: true })],
```

```
// Application Standalone
provideRouter(routes, withComponentInputBinding()),
```

- Ensuite, vous récupérer le paramètre comme un input via son nom ou alias comme vous le faite que ce soit pour @Input ou le signal input.

```
cv = input.required<Cv>();
```

L'ordre de priorité avec withComponentInputBinding

- Les informations de bining du routeur proviennent des sources suivantes :
  - QueryParams
  - Params
  - Données de route statique
  - Données des résolveurs

- Les clés en double sont résolues dans cet ordre, de la plus petite à la plus grande, ce qui signifie que **les résolveurs ont la priorité la plus élevée** et remplacent toutes les autres informations de la route.
- Il est important de noter que lorsqu'une entrée ne contient aucun élément de données de route avec une clé correspondante, cette entrée est définie comme undefined.

Passer le paramètre à travers le tableau de routerLink

- Une autre méthode permet de passer le paramètre de la route est en l'ajoutant comme un autre attribut du tableau associé au routerLink

```
import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
@Component({
  selector: 'app-home',
  templateUrl: './home.component.html',
  styleUrls: ['./home.component.css'],
})
export class HomeComponent {
  constructor(private router: Router) {}
  id: number = 10;
  onNaviger() {
    this.router.navigate(['/about', this.id]);
  }
}
```

Déclencher une route routerLink
Route relative Vs Route absolue

- Quand vous définissez la route vers laquelle vous voulez naviguer, si on **préfixe la valeur passé au routerLink** avec :
  - **'/'**, la route sera **absolue**.
  - **Rien ou './',** le Router **regardera dans les enfants du ActivatedRoute**.
  - **'../'**, le Router regardera dans le niveau au-dessus dans l'arbre de routes.
- Quand on désire **passer un path relativement à la route dans laquelle on est**, il est possible de passer à la **méthode navigate**, en argument, **un objet dont la clé relativeTo** avec comme **valeur** est une **instance de ActivatedRoute**, informe sur le fait **qu'on veut une route relative à la route active.**

```
this.router.navigate(['edit'], {relativeTo: this.activatedRoute});
```

Les queryParameters

- Les **queryParameters** sont les paramètres envoyés à travers une requête **GET**.
- Identifié avec le **?**.



- Afin d'insérer un **queryParameters** on dispose de deux méthodes
- On ajoute dans la méthode **navigate** du **Router** un second paramètre de type objet.
- L'une des propriétés de cet objet est aussi un objet dont la clé est **queryParams** dont le contenu est aussi un objet content les identifiants des queryParams et leurs valeurs.

```
this.router.navigate(['/about', this.id], {
  queryParams: { qpVar: 'je suis un qp' },
});
```

Les queryParameters

- La deuxième méthode est en l'intégrant à notre routerLink de la manière suivante :

```
<a
  [routerLink]="['/about/10']«
  [queryParams]="{qpVar:'je suis un qp bindé avec le routerLink'}"
>
About
</a>
```

Récupérer Les queryParameters

- Les **queryParameters** sont récupérable de la même façon que les paramètres. Soit d'une façon statique avec **snapshot via la propriété queryParams** ou sa propriété **queryParamMap et sa méthode get.**
- **Soit dynamiquement via l'observable queryParams**

```
this.activatedRoute.snapshot.queryParams['page']
```

```
this.activatedRoute.snapshot.queryParamMap.get('page')
```

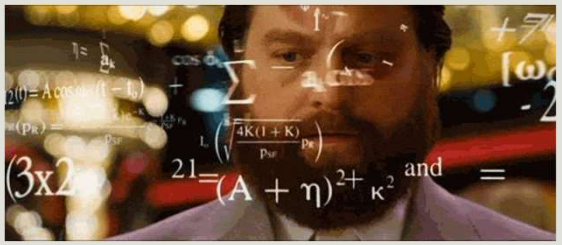
La route joker

- Il existe une route **joker** qui **matche n'importe quelle autre route**. C'est la route **'**'**.

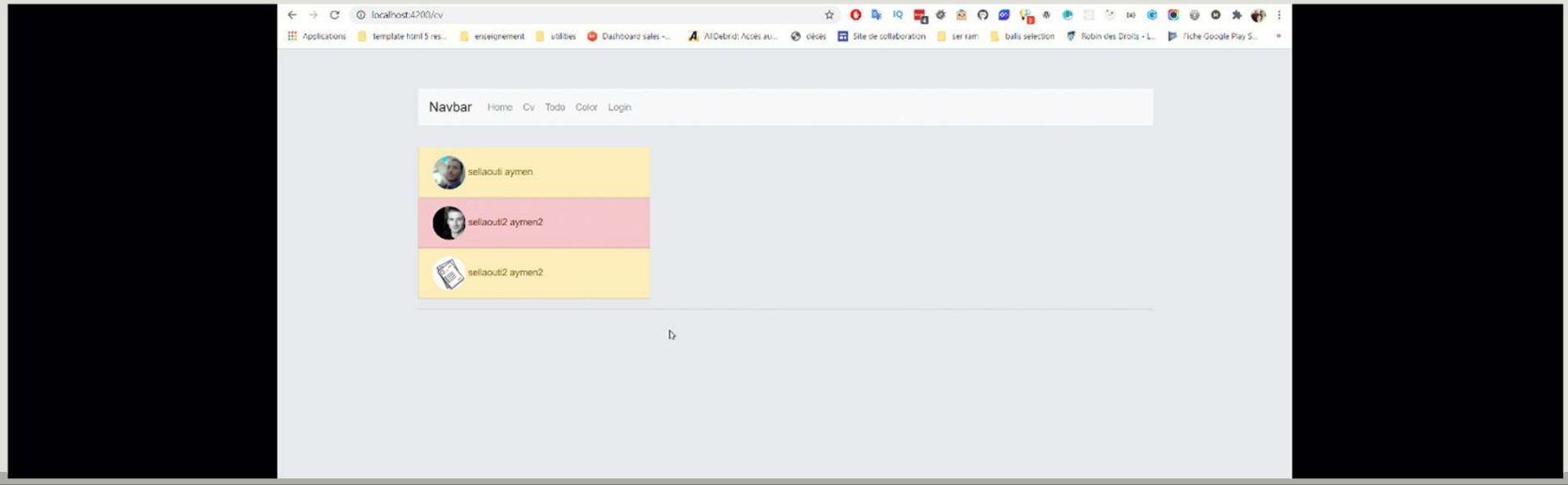
Exemple

```
const APP_ROUTE: Routes = [
  { path: 'cv', component: CvComponent },
  { path: 'lampe', component: ColorComponent },
  { path: 'login', component: LoginComponent },
  { path: '**', component: NFComponent },
];
```

Exercice



- Ajouter les fonctionnalités suivantes à votre cvTech:
  - Une page détail qui va afficher les détails d'un cv.
  - Un bouton dans chaque cv qui au click vous envoi vers la page détails.
  - Dans la page détail, un bouton delete qui au click supprime ce cv et vous renvoi à la liste des cvs.



Route Fils

- Certains composants ne sont visible qu'à l'intérieur d'autres composants.

- Prenons l'exemple d'un objet Personne. En accédant à la route /personne/:id nous avons l'affichage de la personne et nous aimerions avoir deux boutons. Un pour éditer la Personne (route /personne/:id/editer). L'autre pour afficher ces détails (route /personne/:id/apercu).

- L'idée est de préfixer nos routes.

Route Fils

- Afin de mettre en place ce processus nous procédons comme suit :
  - Nous définissons le préfixe avec la propriété **path**.
  - Nous y ajoutons la propriété children qui contiendra le tableau des routes. Chaque route de ce tableau sera préfixé avec la route définie dans **path**.

Route Fils

```
const CV_ROUTE: Routes = [
  {
    path: 'cv',
    children: [
      {path: '', component: CvComponent },
      {path: 'detail/:id', component: DetailCvComponent },
      {path: 'addPersonne', component: FormPersonneComponent },
    ]
  }
];
```

Route fils / définition dans un parent

- Supposons que nous voulons avoir un Template central avec des données fixe et des parties variables dans le même template.

- En changeant les routes, le contenu principal doit rester le même et la partie variable doit changer selon la route.

Route Fils

- Afin de mettre en place ce processus nous procédons comme suit :
  - Nous définissons le préfixe avec la propriété **path**. On lui associe le composant Père.
  - Nous y ajoutons la propriété children qui contiendra le tableau des routes. Chaque route de ce tableau sera préfixée avec la route définie dans **path**.
  - Nous ajoutons la balise <router-outlet></router-outlet> dans le Template père.

Route Fils

```
const CV_ROUTE: Routes = [
  {
    path: 'cv',
    component: CvComponent,
    children: [
      {path: 'detail/:id', component: DetailCvComponent },
      {path: 'addPersonne', component: FormPersonneComponent },
    ]
  }
];
```

Angular Form

AYMEN SELLAOUTI

Approche de gestion de FORM

1. Approche basée Template
2. Approche réactive

Objetctifs

1. Créer un formulaire
2. Ajouter des validateurs
3. Appréhender les classes Css générées par le formulaire
4. Manipuler l'objet ngForm
5. Manipuler les controles du formulaire

Approche basée Template/ Template Driven Approach

1. **Importer le module FormsModule** dans **app.module**.ts si vous travaillez avec un **application modulaire**, sinon **dans votre composant** si vous travaillez avec les **standalone component**
2. Angular détecte automatiquement un objet form à l'aide de la balise FORM. Cependant, il ne détecte aucun des éléments (inputs).
3. Spécifier à Angular quel sont les éléments (contrôles) à gérer.
   - Pour chaque élément ajouter la directive angular **ngModel.**
   - Identifier l'élément avec un nom permettant de le détecter et de l'identifier dans le composant.

```
<input
  type="text"
  id="username"
  class="form-
control"
  ngModel
  name="username"
>
```

4. Associer l'objet représentant le formulaire à une variable et la passer à votre fonction en utilisant le référencement interne # et la directive **ngForm**

Approche basée Template/ Template Driven Approach

```
<form
(ngSubmit)="onSubmit(formula
ire)" #formulaire="ngForm">
```

Template

```
export class
TmeplateDrivenComponent{
   onSubmit(formulaire:
NgForm){

console.log(formulaire);
   }
}
```

Component.ts

Approche basée Template Validation

Afin de valider les propriétés des différents contrôles, Angular utilise des attributs et des directives

- required
- email

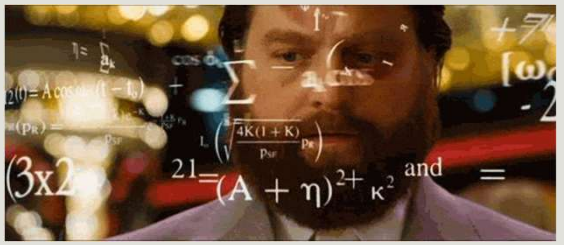
La propriété valid de ngForm permet de vérifier si le formulaire est valid ou non en se basant sur les validateurs qu'ils contient.

Approche basée Template
NgForm

En détectant le formulaire, Angular décore les différents éléments du formulaire avec des classes qui informe sur leur état :

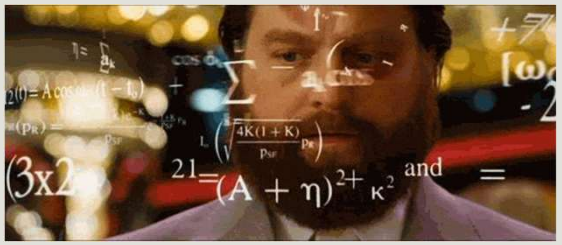
- **dirty** : informe sur le fait que l'une des propriétés du formulaire a été modifié ou non
- **Valid / invalid** : informe si le formulaire est valide ou non
- **Untouched / touched** : informe si le formulaire est touché ou non
- **pristine** : le formulaire n'a pas été touché, c'est l'opposé du dirty

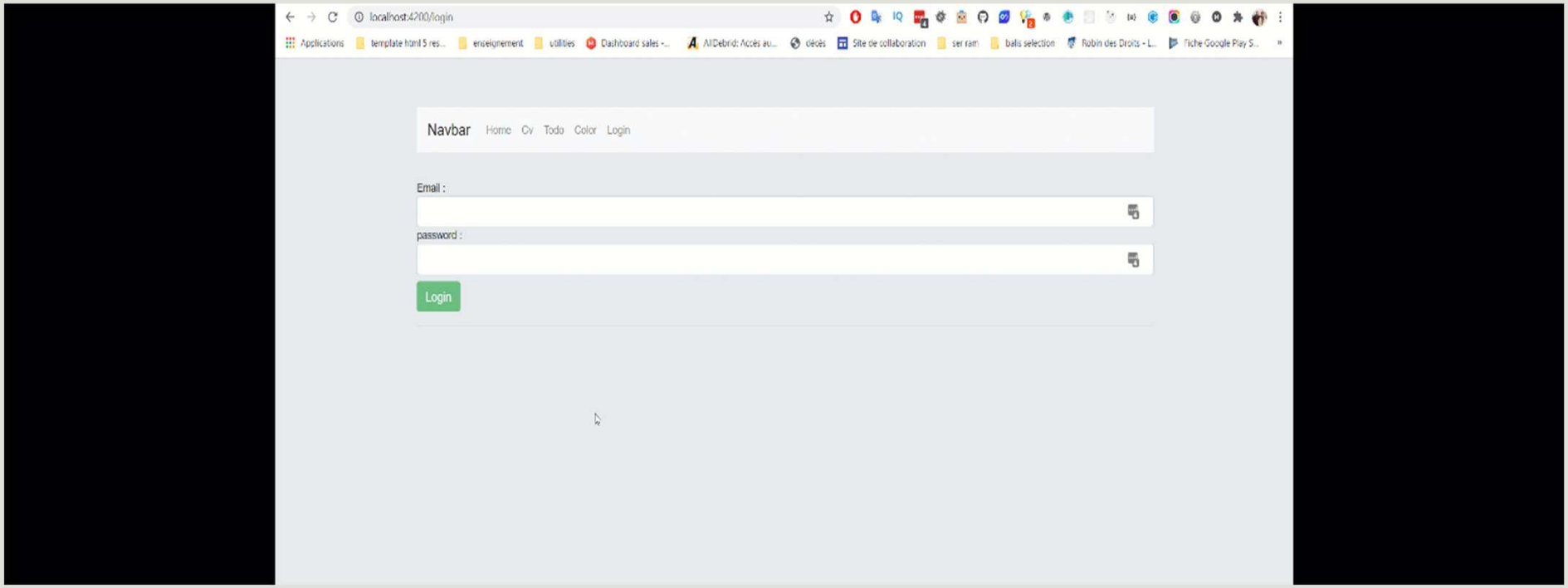
Exercice



- Créer un formulaire d'authentification contenant les champs suivants :
  - Email
  - Password
  - Envoyer
- Si un champ est invalide alors il devra avoir une bordure rouge.
- Les deux champs sont obligatoires
- Le password doit avoir au moins 4 caractères.
- Un champ vide et non encore modifié ne peut avoir de bordure rouge que s'il a été touché.
- Le bouton « envoyer » ne doit être cliquable que si le formulaire est valide.
- Utiliser le binding sur la propriété *disabled*.

Exercice



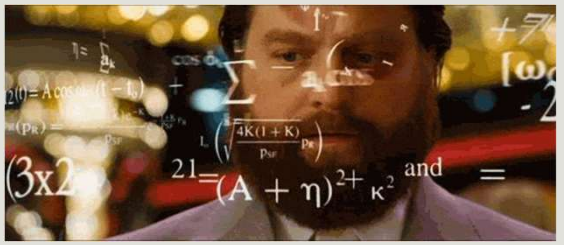

localhost:4200/login
Navbar
Home
Cv
Todo
Color
Login
Email :
password :
Login

Approche basée Template
Accéder aux propriétés d'un champ (contrôle) du formulaire

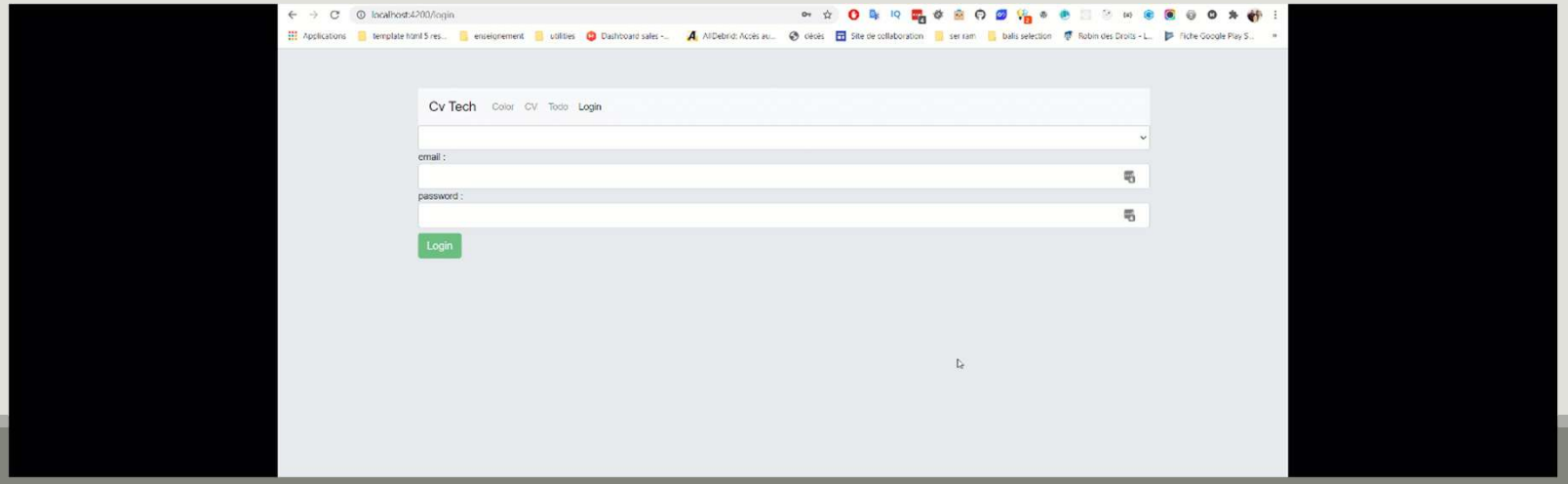
- Pour accéder à l'objet form et ces propriétés nous avons utilisé #notreForm=«ngForm »

- Pour les champs du formulaire c'est la même chose mais au lieu du ngForm c'est un **ngMdoel**
  #notreChamp=« ngModel »

Exercice



- Ajouter un petit message d'erreur qui devra s'afficher sous le champs de l'email s'il est invalide. Ce champ ne devra apparaitre que si l'utilisateur accède ou modifie le champ email.
- Ajouter un champ d'erreur pour signaler l'erreur à l'utilisateur.



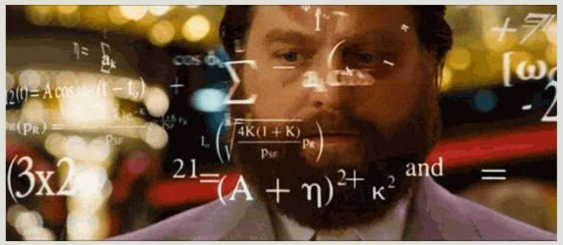
Approche basée Template
Associer des valeurs par défaut aux champs

- Pour associer des valeurs par défaut aux champs d'un formulaire associé à Angular il faut le faire à partir du composant.

- Afin de gérer les valeur du formulaire à partir du composant il faut du binding.

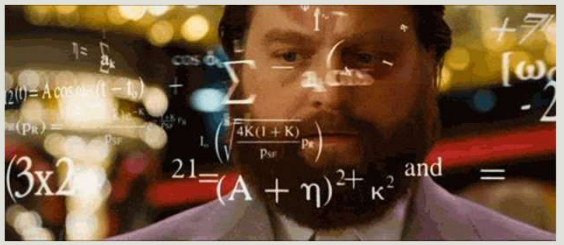
- Au lieu d'avoir juste la primitive ngModel associée au contrôle d'un élément on ajoute le property binding avec [ngModel]

Exercice

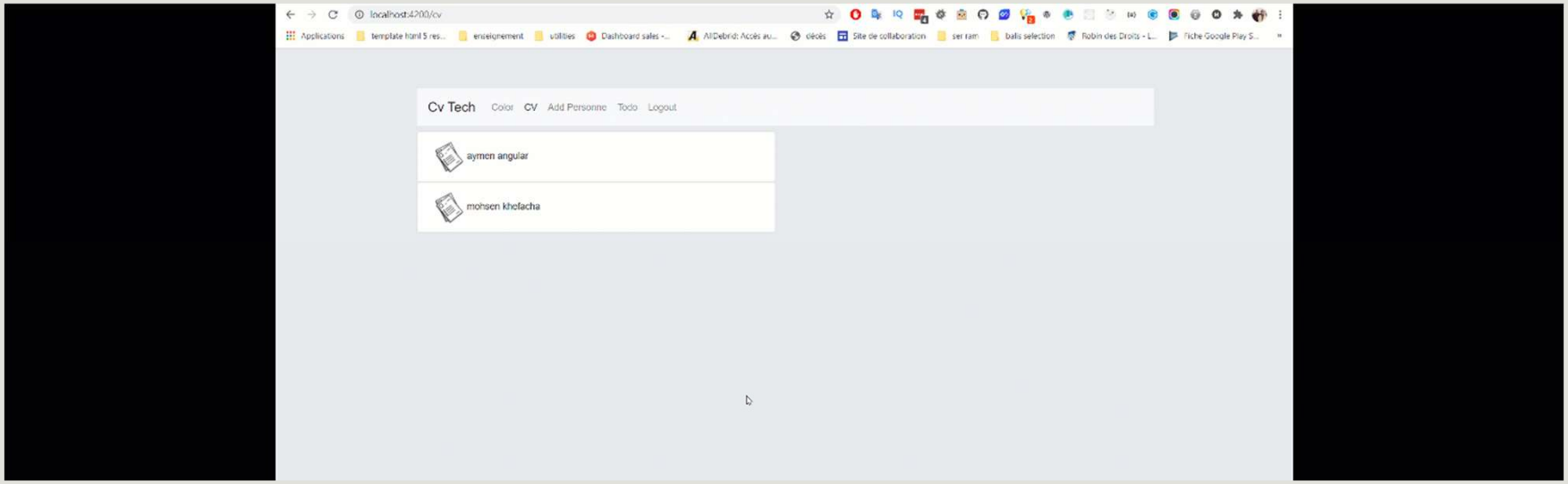


- Ajouter la valeur par défaut « myUserName » au champ username.

Exercice



- Ajouter dans votre cvTech un composant contenant un formulaire. Ce formulaire devra vous permettre d'ajouter un utilisateur.
- Après l'ajout fowarder le user vers la liste des cvs.



Angular HTTP

AYMEN SELLAOUTI

Objectifs

1. Comprendre le design pattern Observable et son implémentation avec RxJs
2. Appréhender le Module HTTPClientModule d'Angular
3. Utiliser les différents services du module HTTPClientModule
4. Comprendre le principe d'authentification via les tokens
5. Utiliser les protecteurs de routes (les guards)
6. Utiliser les Interceptors afin d'intercepter les requêtes Http

HTTP

- Angular est un Framework FrontEnd
- Pas d'accès à la BD
- Pas de possibilité de persistance des données
- Pas de puissance permettant des traitements lourds.

Le HTTPClient

Programmation Asynchrone

Programmation non bloquante.

Les promesses

- Ce sont des objets qui représentent une complétion ou l'échec d'une opération asynchrone.

(https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Utiliser_les_promesses)

- Le fonctionnement des promesses est le suivant :
  - On crée une promesse.
  - La promesse va toujours retourner deux résultats :
    - resolve en cas de succès
    - reject en cas d'erreur
  - Vous devrez donc gérer les deux cas afin de créer votre traitement

Promesse

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve(3);
  }, 5000);
});
promise.then(function (x) {
  console.log('resolved with value :', x);
});
```

Les API JavaScript sont déjà des flux

API JavaScript	Ce qui arrive dans le temps
`setTimeout(cb, 1000)` *minuteur unique*	
`setInterval(cb, 1000)` *minuteur répété*	
`el.addEventListener('click', cb)` *événements du DOM*	
`fetch(url).then(cb)` *requête HTTP, une Promise*	
`ws.onmessage = cb` *WebSocket, messages du serveur*	

v next : une valeur | complete : fin du flux | ••• le flux continue

Un seul contrat : `next, error, complete`

Qu'est ce que la programmation réactive

1. Nouvelle manière d'appréhender les appels asynchrones
2. Programmation avec des flux de données asynchrones

Programmation reactive =
Flux de données (observable) + écouteurs d'événements(observer).

Qu'est ce que la programmation réactive

- La programmation réactive nous permet donc de **réagir à divers événements** tel qu'une écoute sur un **click, un timer ou un interval** qui déclenche un traitement, une **requête http**.
- Elle va nous permettre **d'uniformiser la gestion de ses événements**.
- Actuellement pour une requête **http** c'est un **fetch**, pour un **event** dans le navigateur c'est un **addEventListener**, …
- L'idée est donc de **normaliser tout ca.**
- Ceci nous permettra de les **fusionner**, les **combiner** …

Les API JavaScript sont déjà des flux

API JavaScript	Ce qui arrive dans le temps
`setTimeout(cb, 1000)` *minuteur unique*	
`setInterval(cb, 1000)` *minuteur répété*	
`el.addEventListener('click', cb)` *événements du DOM*	
`fetch(url).then(cb)` *requête HTTP, une Promise*	
`ws.onmessage = cb` *WebSocket, messages du serveur*	

v next : une valeur | complete : fin du flux | ••• le flux continue

Un seul contrat : `next, error, complete`

Les API JavaScript sont déjà des flux

API JavaScript	Ce qui arrive dans le temps	En RxJS
`setTimeout(cb, 1000)` *minuteur unique*		`timer(1000)` 1 valeur, puis complete
`setInterval(cb, 1000)` *minuteur répété*		`interval(1000)` infini : jamais de complete
`el.addEventListener('click', cb)` *événements du DOM*		`fromEvent(el, 'click')` infini, au rythme de l'utilisateur
`fetch(url).then(cb)` *requête HTTP, une Promise*		`fromFetch(url)` 1 valeur puis complete, ou error
`ws.onmessage = cb` *WebSocket, messages du serveur*		`webSocket(url)` N valeurs, complete à la fermeture

v next : une valeur | complete : fin du flux | ••• le flux continue

Un seul contrat : `next, error, complete`

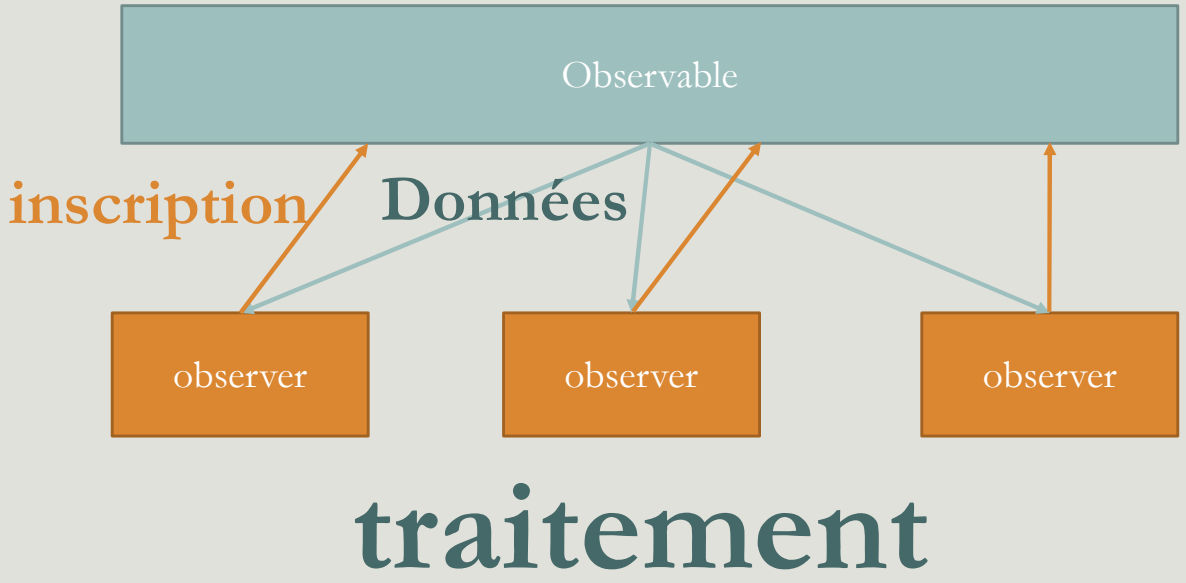
Le pattern « Observer »

- Le patron de conception **Observateur** permet à un objet de garder la trace d'autres objets, intéressés par l'état de ce dernier.

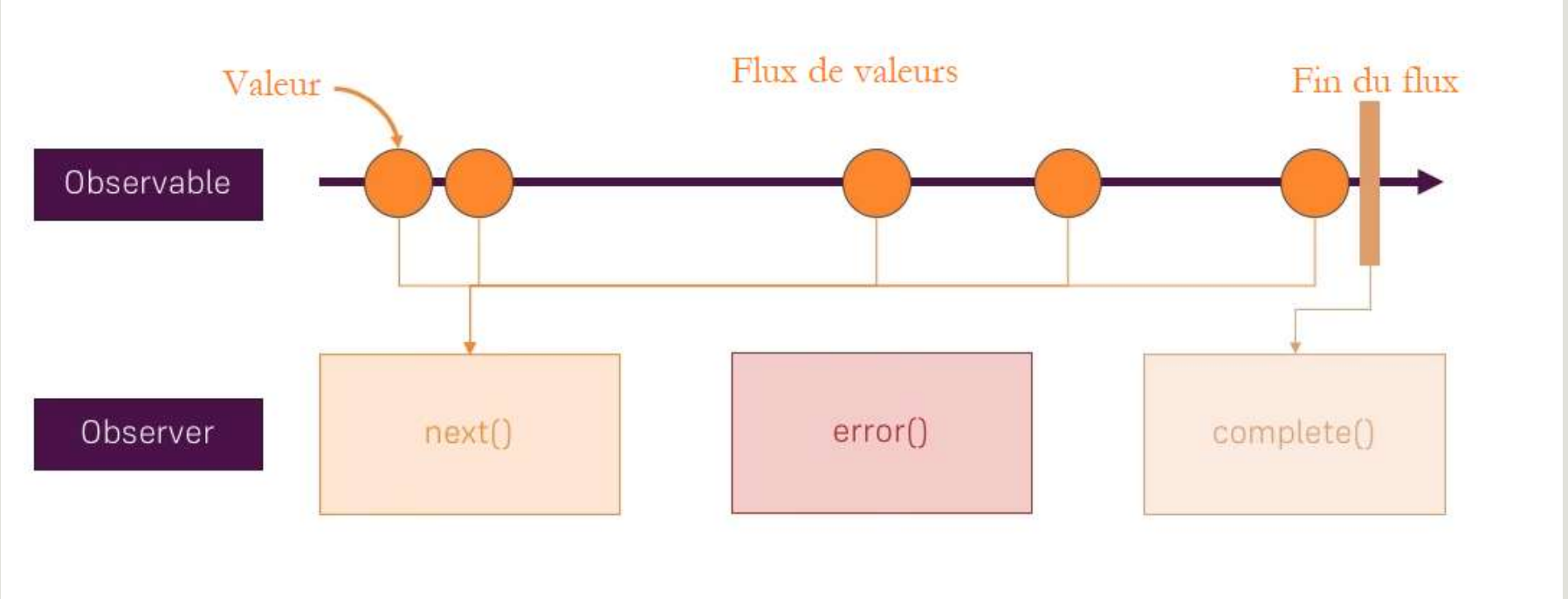
- **Il définit une relation entre objets de type un-à-plusieurs.**

- Lorsque l'état de cet objet **change**, il **notifie** ces **observateurs**.

Observables, Observers et subscriptions



Fonctionnement



Promesse Vs Observable

Promesse	Observable
Un promesse gère un seul événement	Un observable gère un « flux » d'événements.
Non annulable.	Annulable.
Traitement immédiat.	Lazy : le traitement n'est déclenché qu'à la première utilisation du résultat.
Deux méthodes uniquement (then/catch).	Une centaine d'opérateurs de transformation natifs (map, reduce, merge, filter, …).
	Operateurs tels que retry, replay

S'inscrire à un observable

- Afin de **montrer votre intérêt à un flux Observable**, vous pouvez utilisez la méthode **subscribe** qu'il vous offre.
- En vous inscrivant à cet observable, vous pouvez **passez à cette méthode** un **objet qui a 3 fonctions** :
  - **next,** qui sera **appelé à chaque fois qu'une nouvelle valeur arrive dans le flux.** Elle prend en **paramètre cette valeur** et vous permet d'implémenter ce que vous voulez faire avec.
  - **Error, déclenché une fois en cas d'erreur** qui vous permet d'implémenter le comportement en réaction à cette erreur
  - **Complete, déclenché dès la fin du flux** qui vous permet d'implémenter le comportement en réaction à la fin du flux.

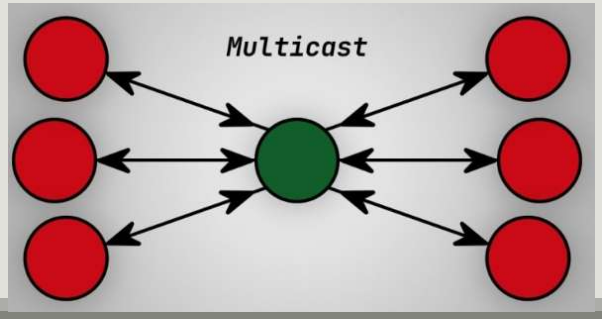
S'inscrire à un observable

```
myObservable$.subscribe({
      next: (data) => {
        //TODO: implementez le comportement quand vous recevez les données
      },
      error: (error) => {
        //TODO: implementez le comportement quand vous avez une erreur
       },
      complete: () => {
        //TODO: implementez le comportement à la fin du flux
      },
    });
```

Hot Vs Cold Observable

- Les ***Cold Observables*** commencent à émettre des valeurs uniquement quand on s'y inscrit. Les ***Hot observables***, par contre émettent toujours.
- Les ***Cold Observables*** diffusent un flux par inscrit, ils sont ***unicast***. Chaque nouvelle inscription crée un ***nouveau contexte d'exécution***.
- Les ***Hot observables***, sont ***multicast***, le ***même flux est partagé par tous les inscrits***.
- Dans les ***Cold Observables,*** la ***source de données est à l'intérieur*** de l'observable.
- Dans les ***HotObservables,*** la ***source de données est à l'extérieur*** de l'observable.





asyncPipe

- asyncPipe est un pipe qui permet d'afficher directement un observable.

- {{ valeurSourceAsynchrone | async }}

- L'asyncPipe s'inscrit automatiquement à l'observable et affiche le dernier résultat envoyé.

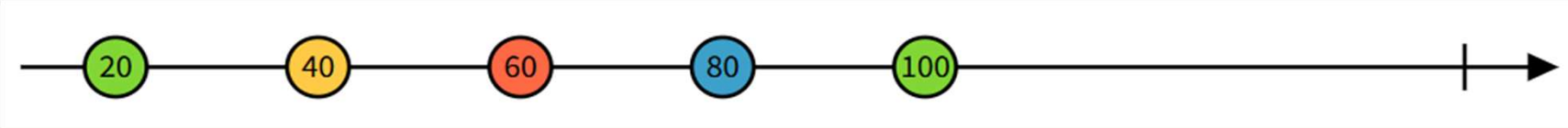
- Quand le composant est détruit l'asyncPipe se désinscrit automatiquement de l'observable.

Les operateurs de l'observable

- Les opérateurs sont des fonctions. Il y a deux types d'opérateurs :
- **Un opérateur pipeable** est une fonction qui prend un observable comme entrée et renvoie un autre observable. C'est une opération pure : le précédent Observable reste inchangé.
  - Syntaxe : monObservable.pipe(opertaeur1(), operateur2(), …).
- **Les opérateurs de création** sont l'autre type d'opérateur, qui peut être appelé comme fonctions autonomes pour créer un nouvel Observable. Par exemple : of(1, 2, 3) crée un observable qui va émettre 1, 2, et 3, l'un après l'autre.

Lire un marble diagram

- Un **marble diagram** représente un flux dans le temps : l'axe va de gauche à droite
- Chaque **bille** est une valeur émise (**`next`**)
- Un **trait vertical** | marque la complétion, une **croix X** marque une erreur



https://rxmarbles.com

Les operateurs de création

- Les opérateurs de création sont utilisés pour créer de nouveaux observables.
- Ils sont divisés en **opérateurs de création** et en **opérateurs de création de jointure**.
- La principale différence entre eux réside dans le fait que les opérateurs de création de **jointure créent des observables à partir d'autres observables**, alors que les **opérateurs de création** créent des observables **à partir d'objets qui diffèrent des observables**.

of, from et timer : exemples

`of` *émet ses arguments, puis termine*

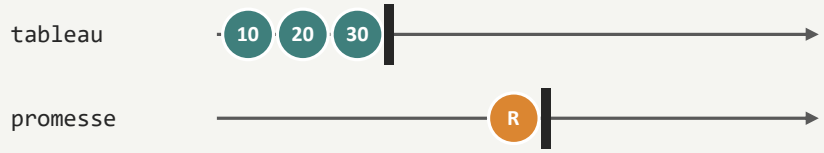
```
of('Angular', 'RxJS', 'NestJS')
  .subscribe(console.log);
// Angular, RxJS, NestJS, puis complete :
// tout est émis d'un coup, à l'abonnement
```



Exemple : *des données de test, une valeur de repli dans catchError*

`from` *déroule un tableau, attend une promesse*

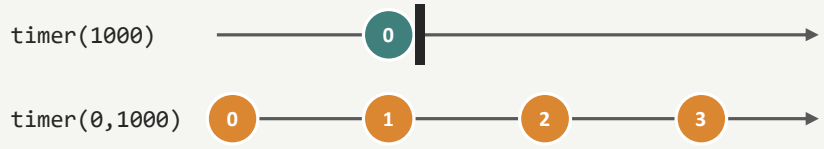
```
from([10, 20, 30]).subscribe(console.log);
// 10, 20, 30, puis complete
from(fetch('/api/cvs')).subscribe(console.log);
// la réponse, une seule fois, puis complete
```



Exemple : *convertir une promesse (fetch, une bibliothèque tierce) en Observable*

`timer` *attend, puis émet 0 (et continue avec une période)*

```
timer(1000).subscribe(console.log);
// 0 au bout d'1 s, puis complete
timer(0, 1000).subscribe(console.log);
// 0, 1, 2… toutes les secondes, sans fin
```

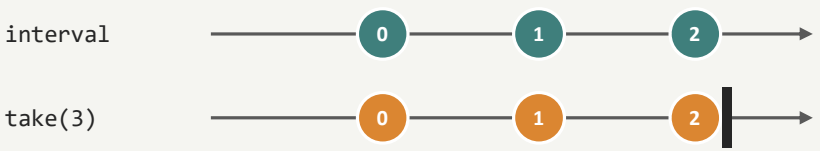


Exemple : *afficher un message après 3 s, démarrer un polling*

interval, fromEvent et defer : exemples

`interval` *émet 0, 1, 2… sans fin*

```
interval(1000).subscribe(console.log);
// 0, 1, 2… sans fin : penser au désabonnement
interval(1000).pipe(take(3)).subscribe(console.log);
// 0, 1, 2, puis complete
```



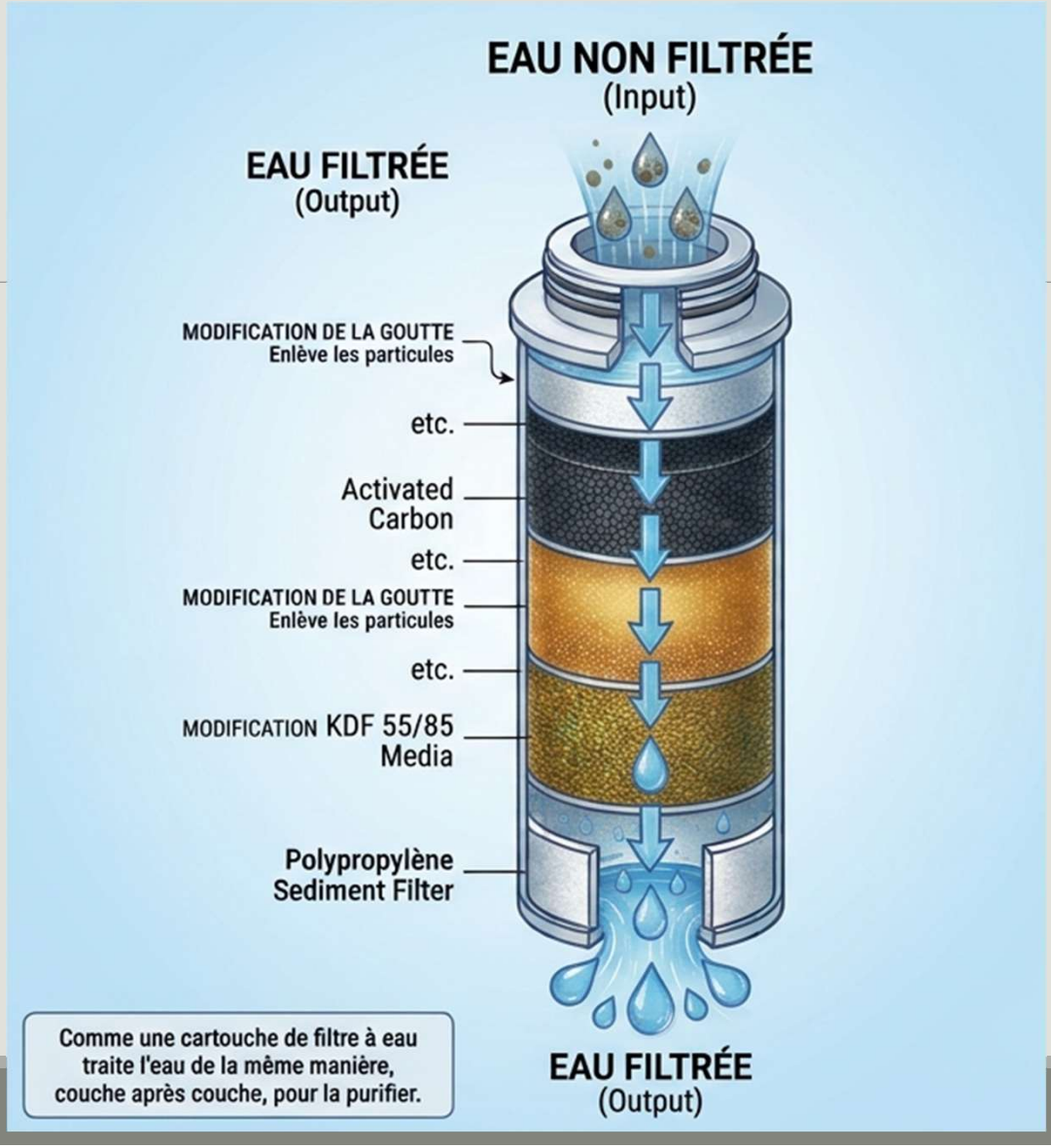
Exemple : *une horloge, un polling, un carrousel*

`fromEvent` *un événement du DOM devient un flux*

```
fromEvent(bouton, 'click')
  .subscribe(e => console.log('clic', e));
// un événement par clic, sans fin :
// le listener est retiré au désabonnement
```



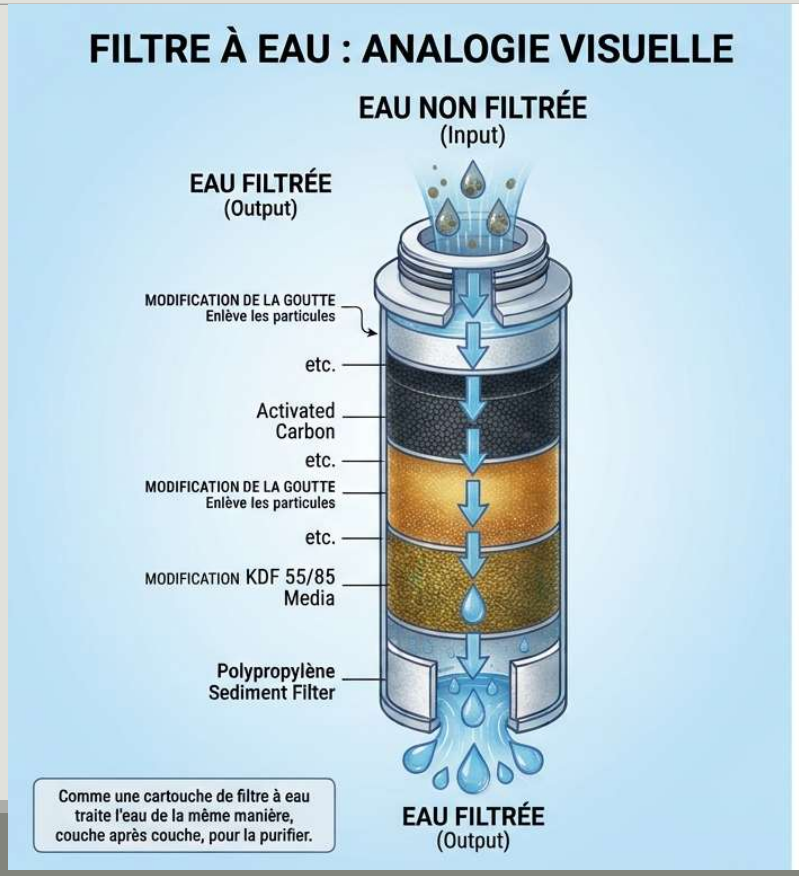
Exemple : *clics, saisie, scroll, redimensionnement de la fenêtre*

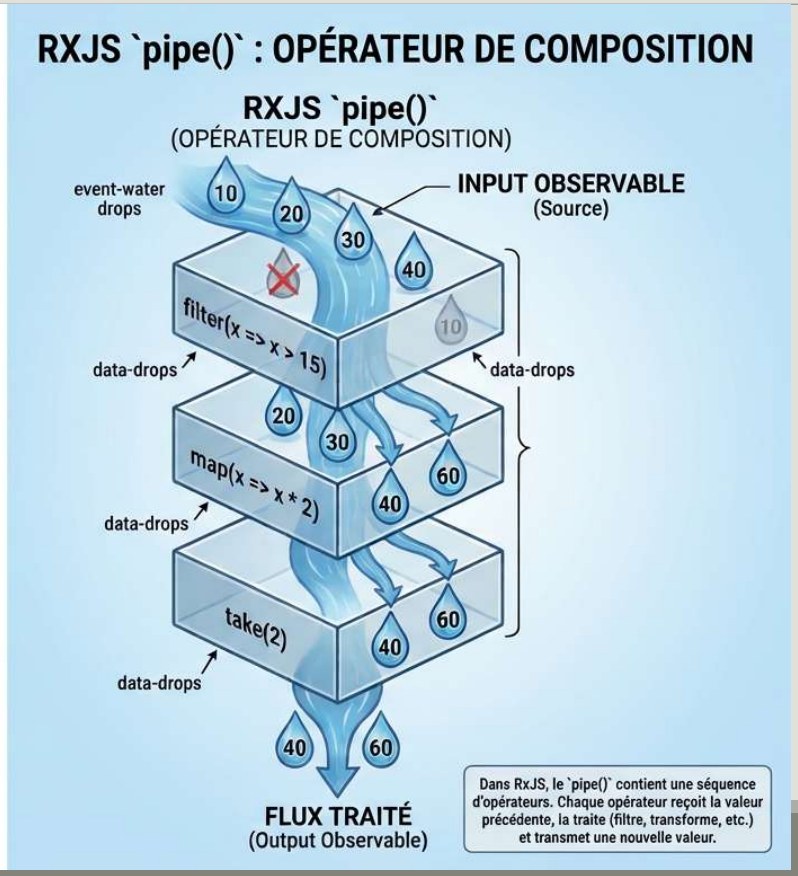

EAU NON FILTRÉE
(Input)
EAU FILTRÉE
(Output)
MODIFICATION DE LA GOUTTE
Enlève les particules
etc.
Activated
Carbon
etc.
MODIFICATION DE LA GOUTTE
Enlève les particules
etc.
MODIFICATION KDF 55/85
Media
Polypropylène
Sediment Filter
Comme une cartouche de filtre à eau
traite l'eau de la même manière,
couche après couche, pour la purifier.
EAU FILTRÉE
(Output)

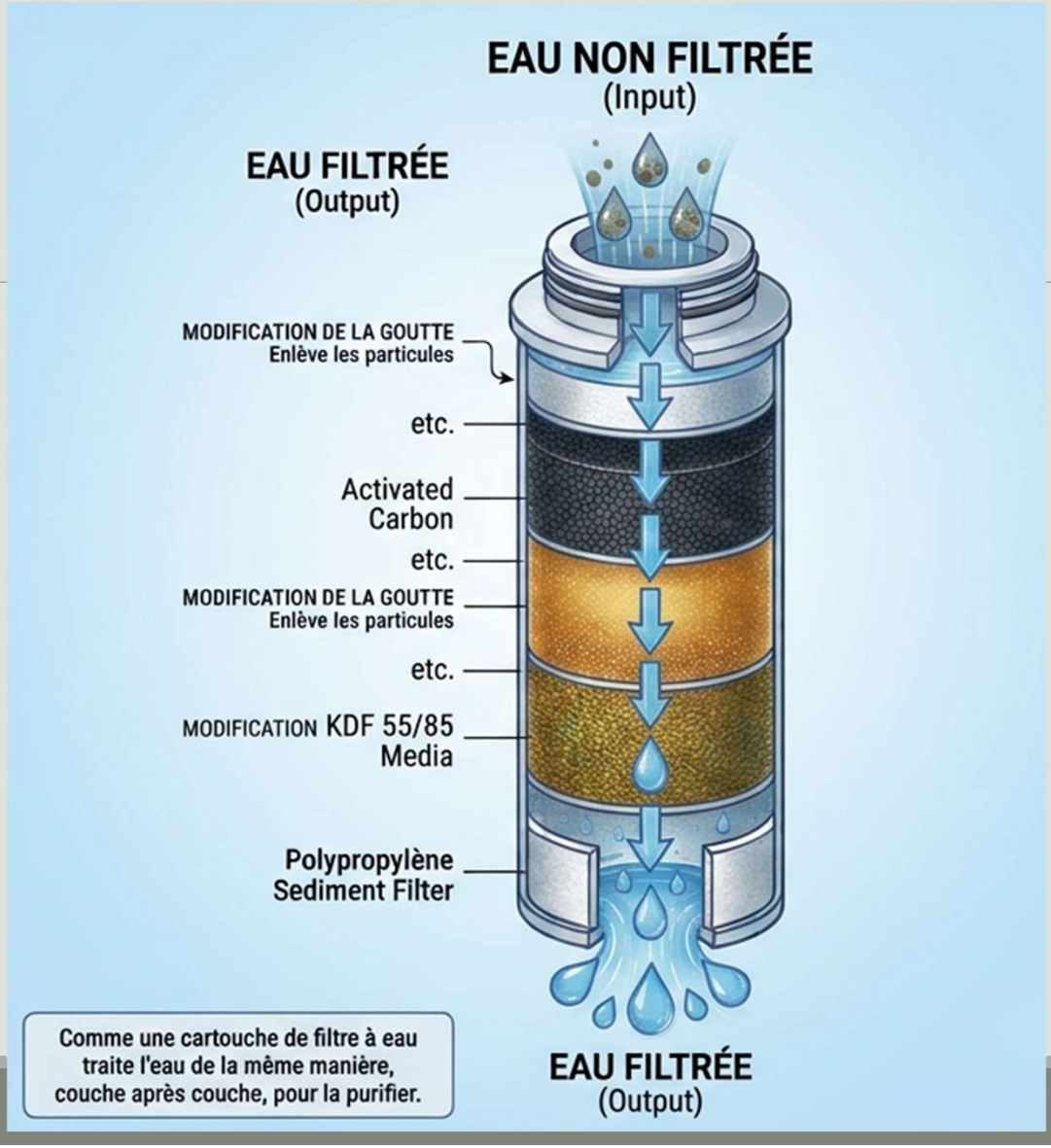
Les operateurs de trannsformation Opérateur pipable

- **Un opérateur pipeable** est une fonction qui prend un observable comme entrée et renvoie un autre observable. C'est **une opération pure** : le précédent Observable reste inchangé.
  - Syntaxe : monObservable.pipe(opertaeur1(), operateur2(), …).

Les operateurs de trannsformation Opérateur pipable



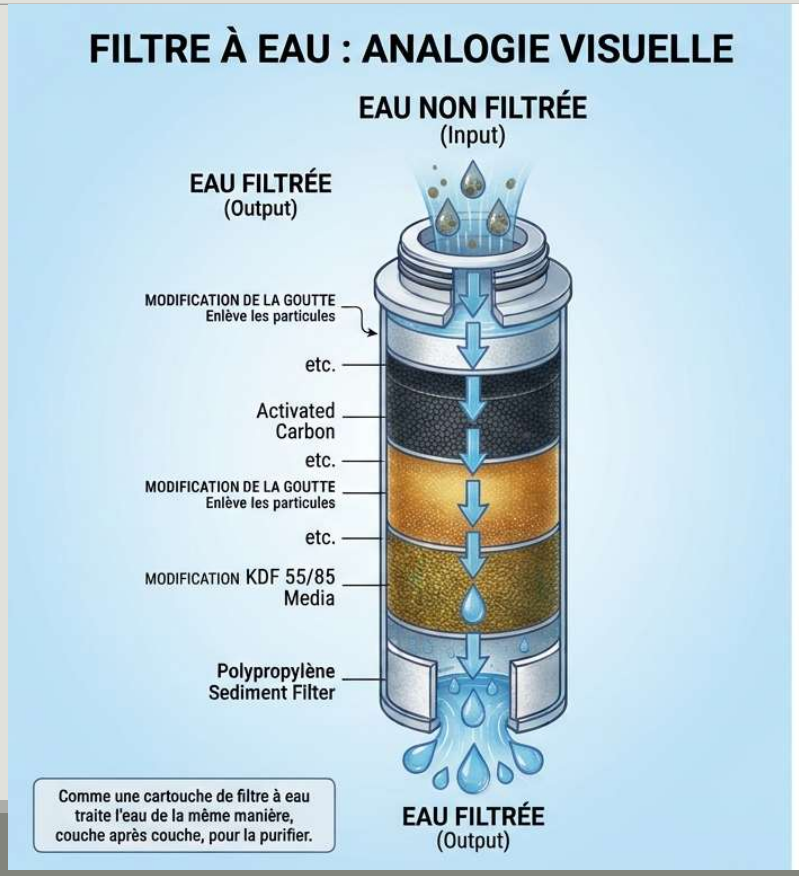


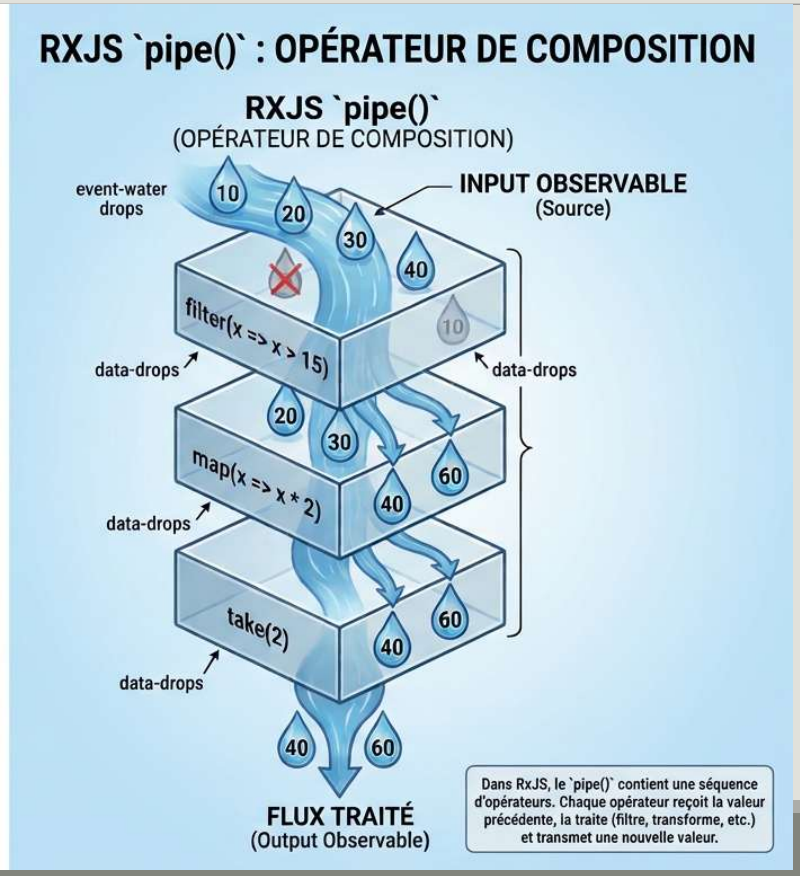

EAU NON FILTRÉE
(Input)
EAU FILTRÉE
(Output)
MODIFICATION DE LA GOUTTE
Enlève les particules
etc.
Activated
Carbon
etc.
MODIFICATION DE LA GOUTTE
Enlève les particules
etc.
MODIFICATION KDF 55/85
Media
Polypropylène
Sediment Filter
Comme une cartouche de filtre à eau
traite l'eau de la même manière,
couche après couche, pour la purifier.
EAU FILTRÉE
(Output)

Les operateurs de trannsformation Opérateur pipable

- **Un opérateur pipeable** est une fonction qui prend un observable comme entrée et renvoie un autre observable. C'est **une opération pure** : le précédent Observable reste inchangé.
  - Syntaxe : monObservable.pipe(opertaeur1(), operateur2(), …).

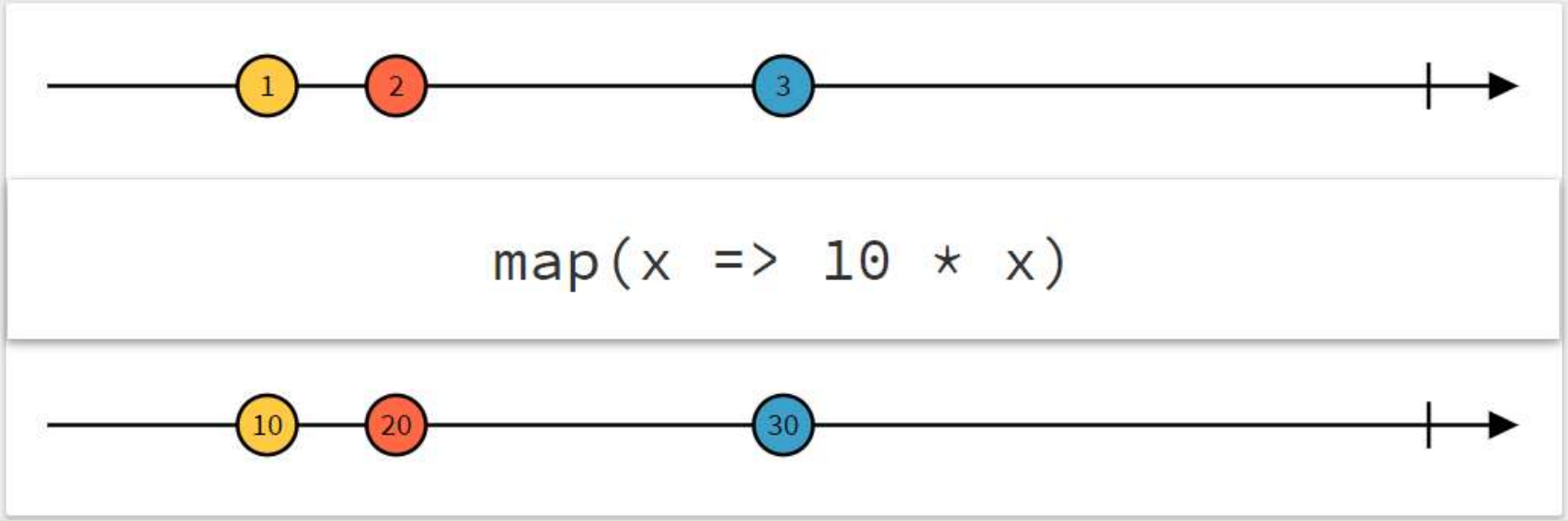
Les operateurs de trannsformation Opérateur pipable





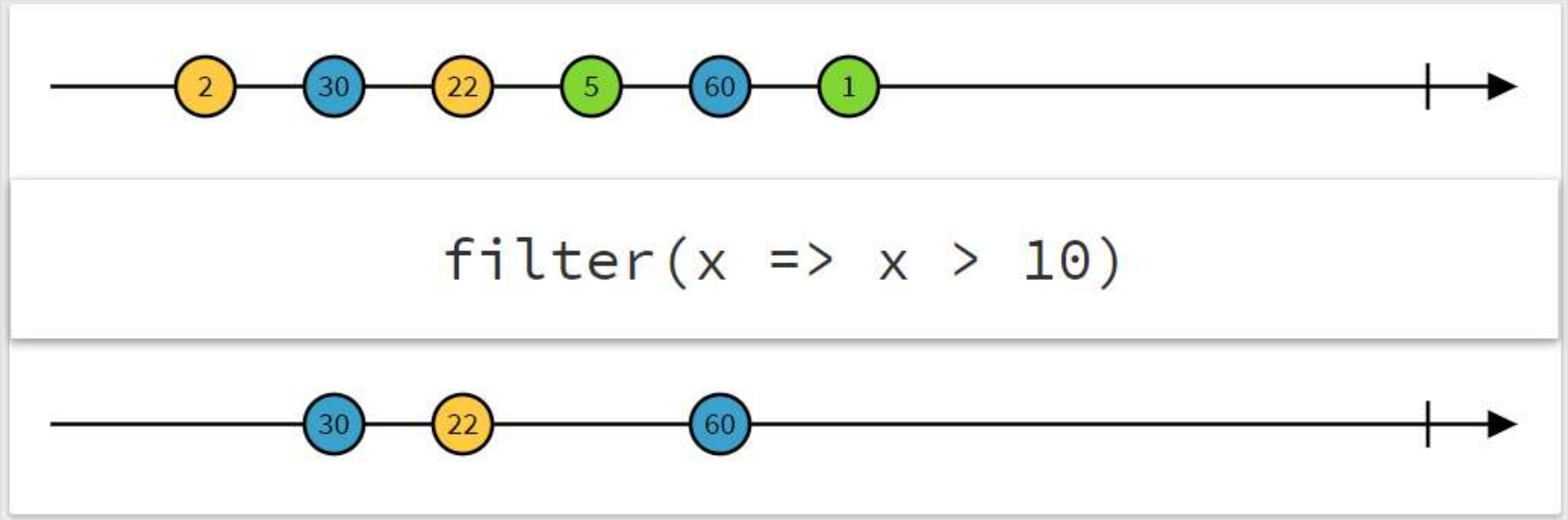
Quelques opérateurs utiles de l'Observable

map



Quelques opérateurs utiles de l'Observable

filter



Transformation et effets de bord
map, tap, scan, startWith

- **`map`** : transforme chaque valeur
- **`tap`** : exécute un **effet de bord** (log, debug) sans modifier le flux
- **`scan`** : accumule et **émet à chaque valeur** (comme un reducer)
- **`startWith`** : émet d'abord une valeur initiale, très utile avec **`combineLatest`**
- **`pluck`** est déprécié : utilisez **`map(x => x.prop)`**

https://rxjs.dev/guide/operators

Filtrage
distinctUntilChanged, take, first, skip

- `filter` : garde les valeurs qui satisfont un prédicat
- `distinctUntilChanged` : ignore une valeur **identique à la précédente**
- `take(n)`, `first()` : prennent les premières valeurs puis **complètent**
- `takeWhile(pred)` : continue tant que la condition est vraie
- `skip(n)` : ignore les n premières valeurs
- `first()` lève une erreur sur un flux vide, `take(1)` non

Quelques opérateurs utiles de l'Observable

https://angular.io/guide/rx-library

http://reactivex.io/rxjs/manual/overview.html#operators

http://rxmarbles.com/

Installation de HTTP

- A partir de la version **18, le HttpClientModule est devenu deprecated.**
- Afin d'utiliser le service HttpClient, vous devez le configurez en utilisant la méthode **provideHttpClient**, que la plupart des applications incluent dans la clé providers de votre app.config.ts.

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(),
  ],
};
```

Installation de HTTP

- Une fois configuré, vous pouvez injecter votre service la ou vous en avez besoin.

```
http = inject(HttpClient);
```

Interagir avec une API Get Request

- Afin d'exécuter une requête **get** le module http nous offre une méthode **get**.
- Cette méthode retourne un **Observale**.
- Inscrivez vous à cet observable et définissez le comportement en cas de succès, erreur ou complétion

```
this.http.get(API_URL).subscribe({
  next:(response:Response)=>{
    //ToDo with DATA
  },
  error: (err:Error)=>{
   //ToDo with error
  },
  complete: () => {
    console.log('Data transmission complete');
  }
});
```

Interagir avec une API POST Request

- Afin d'exécuter une requête POST le module http nous offre une méthode post.
- Cette méthode retourne un Observale.
- Diffère de la méthode get avec un attribut supplémentaire : body

```
this.http.post(API_URL,dataToSend).subscribe({
  next:(response:Response)=>{
    //ToDo with DATA
  },
  error: (err:Error)=>{
   //ToDo with error
  },
  complete: () => {
    console.log('Data transmission complete');
  }
});
```

Documentation

https://angular.io/guide/http

Les headers

- Afin d'ajouter des headers à vos requêtes, le HttpClient vous offre la classe HttpHeaders.
- Cette classe est une classe immutable (read Only).

https://angular.io/guide/http#immutability

- Elle propose une panoplie de méthode helpers permettant de la manipuler.
- set(clé,valeur) permet d'ajouter des headers. Elle écrase les anciennes valeurs.
- append(clé,valeur) concatène de nouveaux headers.

Toutes les méthodes de modification retourne un HttpHeaders permettant un chainage d'appel.

https://angular.io/api/common/http/HttpHeaders

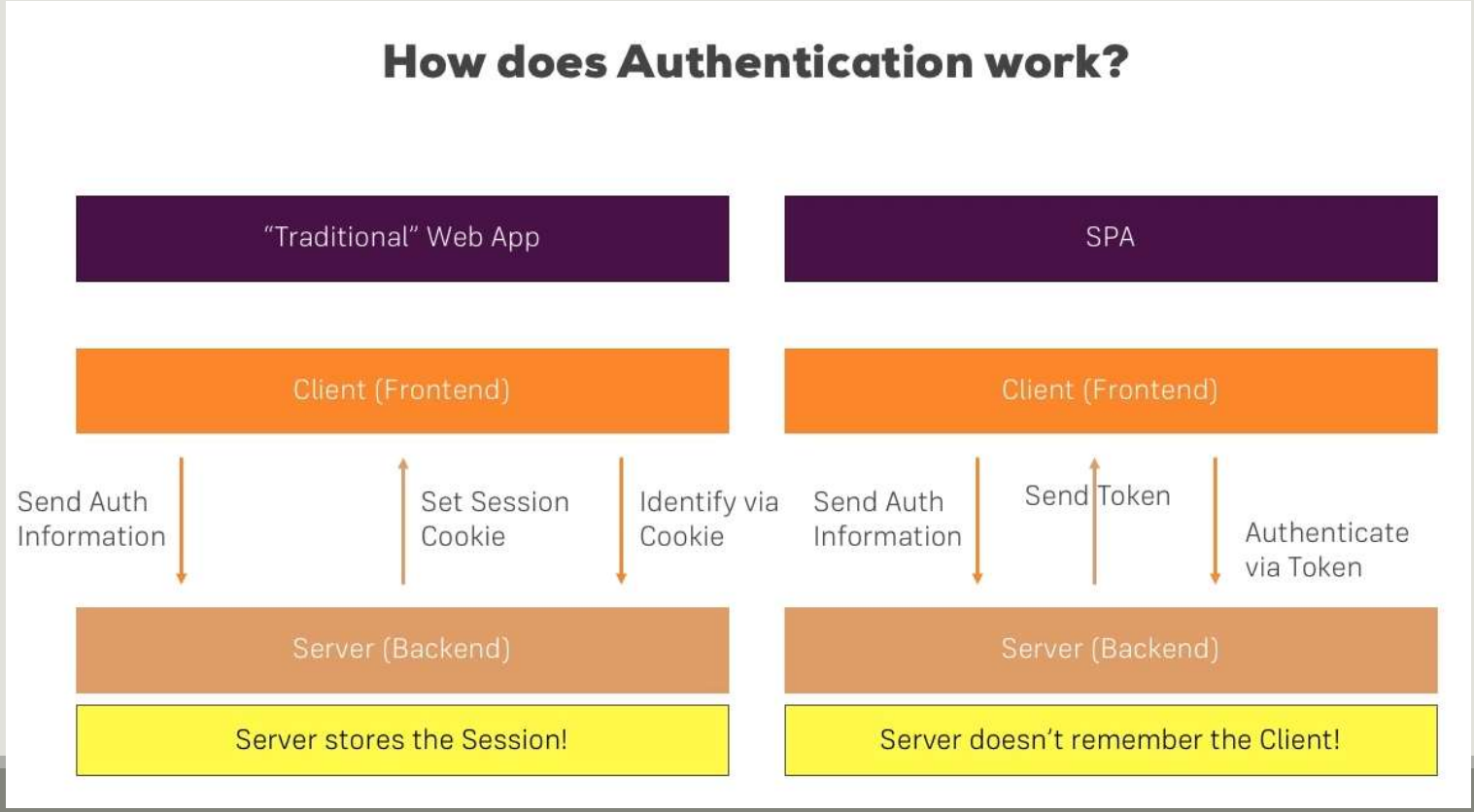
Les paramètres

- Afin d'ajouter des paramètres à vos requêtes, le HttpClient vous offre la classe HttpParams.
- Cette classe est une classe immutable (read Only).
- Elle propose une panoplie de méthode helpers permettant de la manipuler.
- set(clé,valeur) permet d'ajouter des headers. Elle écrase les anciennes valeurs.
- append(clé,valeur) concatène de nouveaux headers.

Toutes les méthodes de modification retourne un HttpParams permettant un chainage d'appel.

https://angular.io/api/common/http/HttpParams

Authentification



Ajouter le token dans la requête

- Si la ressource demandée est contrôlé avec un token, vous devez y insérer le token afin d'être authentifié au niveau du serveur.
- Pour ajouter un token vous pouvez le faire via un objet HttpParams. Cet objet possède une méthode set à laquelle on passe le nom du token 'access_token' suivi du token.
- Vous devez ensuite l'ajouter comme paramètre à votre requête.

```
const params = new HttpParams()
  .set('access_token', localStorage.getItem('token'));
return this.http.post(this.apiUrl, personne, {params});
```

https://angular.io/guide/http#immutability

Ajouter le token dans la requête

- Une seconde méthode consiste à ajouter dans le header de la requête avec comme name 'Authorization' et comme valeur 'bearer' à laquelle on concatène le Token.
- Pour se faire, créer un objet de type HttpHeaders.
- Utiliser sa méthode append afin d'y ajouter ses paramètres.
- Ajouter la à la requête.

```
const headers = new HttpHeaders();
headers.append('Authorization', 'Bearer ${token}');
return this.http.post(this.apiUrl, personne, {headers});
```

Sécuriser vos routes

- Dans vos applications, certaines routes ne doivent être accessibles que si vous êtes authentifié. Ce cas d'utilisation se répète souvent et s'est un sous cas de la sécurisation de vos routes.

- Angular a pris en considération ce cas en fournissant un mécanisme via l'utilisation des **Guard**.

Guard

- Ce sont des classes qui permettent de gérer l'accès à vos routes.
- Un guard informe sur la validité ou non de la continuation du process de navigation en retournant un booléen, une promesse d'un booléen ou un observable d'un booléen.
- Le routeur supporte plusieurs types de guards, par exemple :
  - CanActivate permettre ou non l'accès à une route.
  - CanActivateChild permettre ou non l'accès aux routes filles.
  - CanDeactivate permettre ou non la sortie de la route.

Guard / canActivate

- Afin d'utiliser le guard canActivate (de même pour les autres), vous devez créer un classe qui implémente l'interface CanActivate et donc qui doit implémenter la méthode canActivate de sorte qu'elle retourne un booléen permettant ainsi l'accès ou non à la route cible. 1
- Vous devez ensuite ajouter cette classe dans le provider. 2
- Finalement pour l'appliquer à une route, ajouter la dans la propriété canActivate. Cette propriété prend un tableau de guard. Elle ne laissera l'accès à la route qu esi la totalité des guard retourne true. 3
- **Vous pouvez utiliser la méthode : ng g g nomGuard**

Guard / canActivate

1

```
import { Injectable } from '@angular/core';
import {ActivatedRouteSnapshot, CanActivate, RouterStateSnapshot} from '@angular/router';
import {Observable} from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class AuthGuard implements CanActivate {
  constructor() {
  }
  // route contient la route appelé
  // state contiendra la futur état du routeur de l'application qui devra passer la validation du guard
  // https://vsavkin.com/routeur-angular-comprendre-l%C3%A9tat-du-routeur-5e15e729a6df
    canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot): Observable<boolean> |
  Promise<boolean> | boolean {
      if (// your condition) {
        return true;
      }
      return false;
    }
  }
```

Guard

2

- A partir d'Angular 14 et la possibilité d'utiliser la fonction inject dans tous les contextes d'injection, Angular préconise les fonctionnals Guards.
- Commencez par créer votre Guard et ajoutez-y votre logique.
- Si vous retourner true ou une promise de true ou un Observable de true la route sera activée.
- Si c'est false la navigation vers la route sera annulée

```
export const myGuard: CanActivateFn = (route, state) => {
  return true;
};
```

Guard / canActivate

2

```
providers: [
  TodoService,
  CvService,
  LoginService,
  AuthGuard,
 ],
```

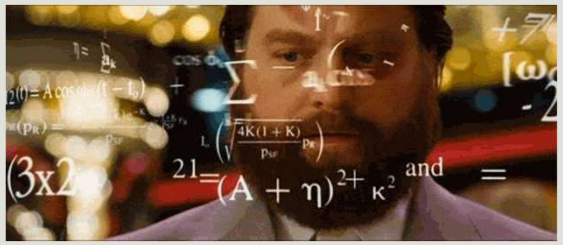
App.module.ts

Guard / canActivate

3

```
{
  path: 'lampe',
  component: ColorComponent,
  canActivate: [AuthGuard]
},
```

Exercice

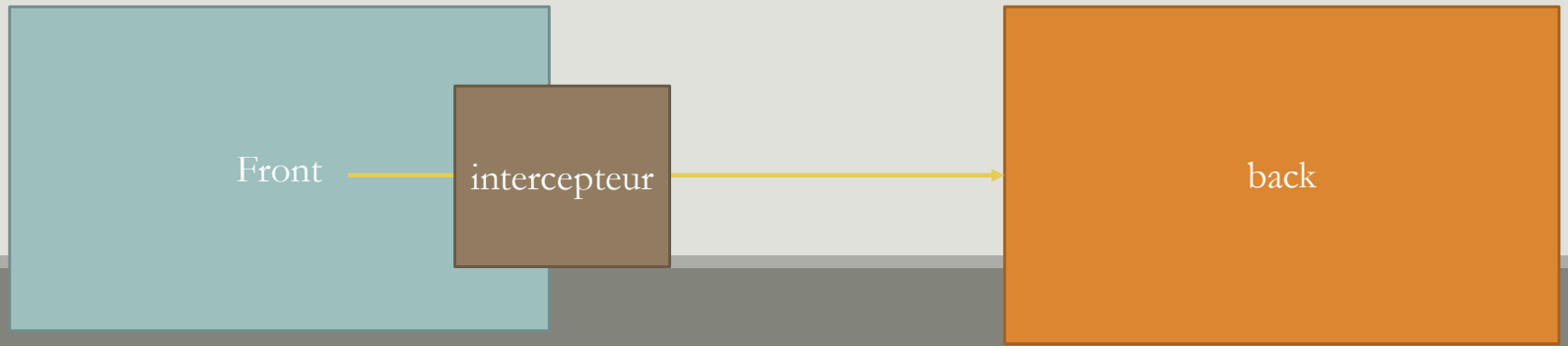


- Ajouter les guards nécessaires afin de sécuriser vos routes.

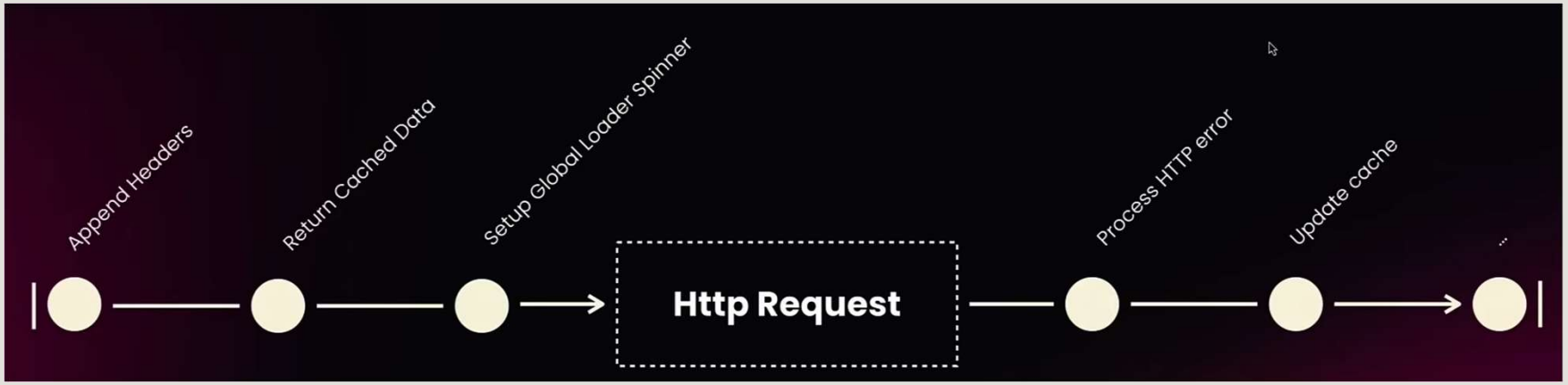
- Une personne déjà connectée ne peut pas accéder au composant de login.

Les intercepteurs

- A chaque fois que nous avons une requête à laquelle nous devons ajouter le token, nous devons refaire toujours le même travail.
- Pourquoi ne pas intercepter les requêtes HTTP et leur associer le token s'il est la à chaque fois ?
- Un intercepteur Angular (fournit par le client HTTP) va nous permettre **d'intercepter une requête à l'entrée et à la sortie de l'application.**
- Un intercepteur est soit une fonction (angular 17) ou une classe qui **implémente l'interface HttpInterceptor.**
- En implémentant cette interface, chaque intercepteur va devoir implémenter la méthode **intercept**.



Les intercepteurs



Les intercepteurs

- Dans les anciennes versions d'angular on devait passer par une class qui implémente l'interface HttpInterceptor.

```
import { Injectable } from '@angular/core';
import {HttpRequest,HttpHandler,HttpEvent,HttpInterceptor,} from '@angular/common/http';
import { Observable } from 'rxjs';
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(  request: HttpRequest<unknown>, next: HttpHandler): Observable<HttpEvent<unknown>> {
    return next.handle(request);
  }
}
```

Les intercepteurs

- Dans les nouvelles versions d'angular nous passons par une **fonction**.
- Dans cette fonction vous récupérer comme premier paramètre l'objet **request** qui représente la **requête HTTP.**
- En deuxième paramètre, vous avez la fonction next qui vous permet de remettre la requête de continuer son parcours.

```
import { HttpInterceptorFn } from "@angular/common/http";
export const newInterceptor: HttpInterceptorFn = (req, next) => {
  return next(req);
};
```

Les intercepteurs : changer la requête

- Par défaut la requête est immutable, on ne peut pas la changer.
- Solution : la cloner via sa méthode clone, changer les headers du clone et le renvoyer.

```
const cloneReq = request.clone({
  setHeaders: {
      'Authorization': token
} });
// Chainer la nouvelle requete avec next.handle
return next.handle(newReq);
```

Les intercepteurs
Appliquer l'intercepteur

- Afin d'appliquer l' intercepteur, vous devez le fournir au niveau du provider.
- Si vous utilisez un **intercepteur fonctionnel**, vous utilisez la fonction **withInterceptors** qui est une des options de **provideHttpClient**.
- Elle prend en paramètre un tableau d'intercepteur

```
bootstrapApplication(AppComponent, {providers: [
  provideHttpClient(
    withInterceptors([firsInterceptor, secondInterceptor]),
  )
]});
```

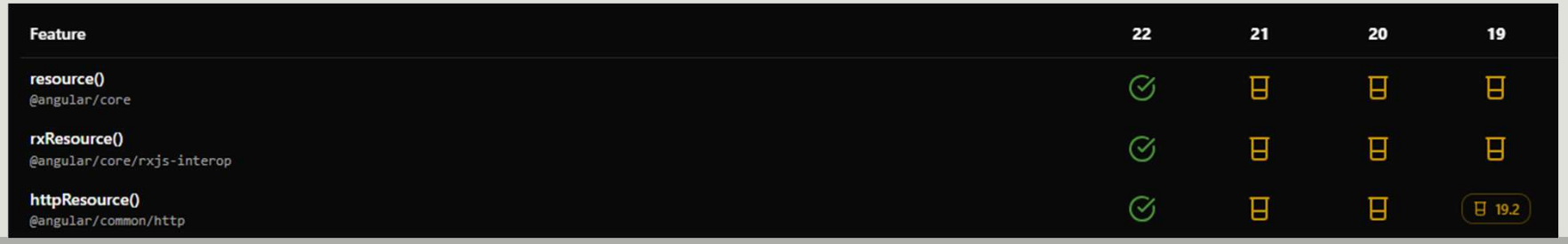
Angular
Les Resources

AYMEN SELLAOUTI

Signal et les appels HTTP

- Un **signal** est une opération **Synchrone**. Quand vous faites **un appel http** ceci revient à faire une **opération Asynchrone.**

- Afin de **briser le bridge entre ces deux mondes** et récupérer les informations concernant vos requêtes http pour **récupérer des données et les transformer en signals représentant votre State**, angular a introduit **3 API, expérimentales en v19 et v20, qui sont stables depuis la v22**. Ces API's sont

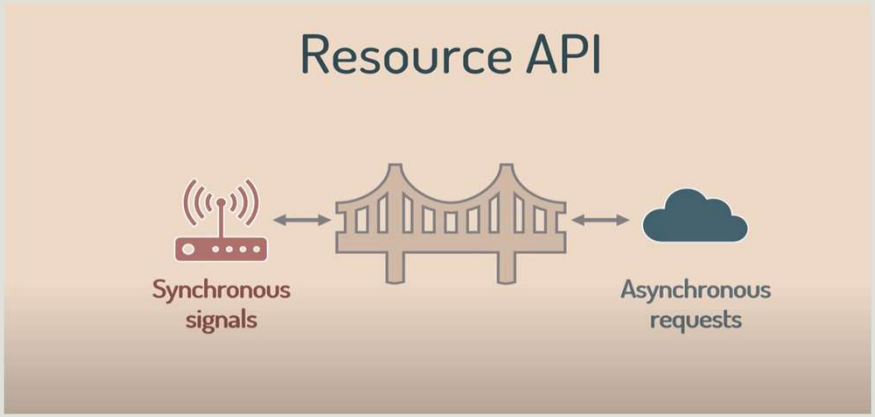


https://www.angular.courses/caniuse?from=19&to=20&search=resource

Signal et les appels HTTP

- Si on se pose la question suivante : De quoi pouvons nous avoir besoins lorsqu'on fait des appels HTTP ?

- Résultat de l'appel en cas de succès.
- Message d'erreur en cas d'erreur
- Savoir est ce que l'appel est en cours de traitement
- Pouvoir relancer un appel quand les paramètres dont il dépend changent



- Les nouveaux API Resource sont la pour faire le pont entre le monde asynchrone de RxJs et Synchrone des Signals et répondre à tous ces questions

https://www.youtube.com/watch?v=YHZkiUbbeOg&ab_channel=DeborahKurata

Les resources API

Basique	resource()	Se base sur les promesses	Utilisez le si vous préférer l'api fetch de ES pour récupérer vous données
Reactive	rxResource()	Utilises les Observables RxJs	Utilisez le si vous préférez travailler avec RxJs pour vos appels http ou que vous avez besoins d'utiliser les opérateurs de RxJs
Pragmatique	La plus simple et la plus flexible.	Le dernier de la série et le plus simple à implémenter.	C'est un Wrapper sur le HttpClient, il nécessite une url et les paramètres dont il dépend. Offre une deuxième version pour les requêtes les plus complexes

resource

- L'API resource est conçue pour gérer **des données asynchrones** basées sur des **Promises**.
- Elle offre une **gestion déclarative des états asynchrones** et **annule automatiquement les requêtes précédentes si les paramètres changent** (sémantique switchMap).
- Elle **s'intègre parfaitement avec la zoneless change détection** vue que les mises à jour des signaux (**value et status**) appellent la fonction notify qui planifie le change detection.

resource

- L'API resource prend en paramètre un objet avec les paramètres suivants :

Dans ng20, request a été renommé en params

Paramètre	Description	Obligatoire	Exemple
`request` params	Fonction retournant les paramètres de la requête (ex. : ID, query).	Non	`() => ({ id: id() })`
`loader`	Fonction retournant une Promise avec les données.	Oui	`({ request }) => fetch(...).then(res => res.json())`
`defaultValue`	Valeur initiale avant le chargement.	Non	`{ id: 0, title: '' }`
`equal`	Fonction de comparaison pour éviter les mises à jour inutiles.	Non	`(a, b) => a.id === b.id`
`injector`	Injecteur Angular pour résoudre les dépendances.	Non	`inject(Injector)`

resource

```
cvResource = resource<Cv | null, {id:number}>({
    // fonction qui détermine à quel moment on va déclencher le loader
    params: () => ({id: this.id()}),
    // loader va aller chercher le résultat du backend
    loader: ({params, abortSignal, previous}) => {
        return fetch(API.cv+params.id).then(res => res.json());
    },
    defaultValue: null
  });
```

- **params (request avant Angular 20) prend en paramètre une fonction** ou un **paramètre** spécifiant les signaux dont dépend votre **resource**.
- **loader** permet de spécifier la **promesse** qui ira **chercher les données**
- **defaultValue** permet de spécifier la **valeur par défaut**

resource
Le loader

- Le loader **est déclenché dès l'initialisation de la ressource**.
- La ressource peut éventuellement disposer d'un signal qui est l'attribut request (params dans la version 20) qui définit les critères de recherche pour le loader.
- Chaque modification du signal de paramètres déclenche à nouveau le loader.
- **ATTENTION** : **L'attribut request (params) est traqué mais pas le loader**. Le loader est dans un **untracked** ce qui signifie **qu'un signal appelé dans le loader n'au aucun impact sur la recharge du loader**.

resource
Le loader

```
interface ResourceLoaderParams<R> {
    params: NoInfer<Exclude<R, undefined>>;
    abortSignal: AbortSignal;
    previous: {
        status: ResourceStatus;
    };
}
```

- Le **loader** prend en **paramètre** un objet de type **ResourceLoaderParams**
  - **params** : Cette propriété contient les paramètres d'entrée ou les données nécessaires au calcul de la ressource. Son type est généralement déduit de la définition de la ressource, à l'exception de undefined.
  - **abortSignal** : Un objet AbortSignal permet d'annuler le chargement de la ressource en cours. Ceci est crucial pour gérer les requêtes asynchrones et prévenir les fuites de ressources lorsqu'une requête n'est plus nécessaire.
  - **previous** : Un objet contenant des informations sur l'état précédent de la ressource. Cela inclut généralement une propriété status indiquant le ResourceStatus (par exemple, « idle », « loading », « success », « error ») de la ressource avant la tentative de chargement en cours.

resource

```
cvResource = resource<Cv | null, {id:number}>({
    // fonction qui détermine à quel moment on va déclencher le loader
    params: () => ({id: this.id()}),
    // loader va aller chercher le résultat du backend
    loader: ({params, abortSignal, previous}) => {
        return fetch(API.cv+params.id).then(res => res.json());
    },
    defaultValue: null
  });
```

- Afin de définir les types de params et du loader, vous pouvez le spécifier dans la partie générique de la resource : resource<Cv | null, {id:number}>
- Ici on l'informe que le loader retourne un Cv ou null et que params fournit un objet avec un id de type number

Les propriétés d'une resource

- Tous les types de resources étendent l'interface Ressource :

```
interface Resource<T> {
    /**
     * The current value of the `Resource`, or `undefined` if there is no current value.*/
    readonly value: Signal<T>;
    /**
     * The current status of the `Resource`, which describes what the resource is currently doing and
     * what can be expected of its `value`.*/
    readonly status: Signal<ResourceStatus>;
    /**
     * When in the `error` state, this returns the last known error from the `Resource`.*/
    readonly error: Signal<unknown>;
    /**
     * Whether this resource is loading a new value (or reloading the existing one).*/
    readonly isLoading: Signal<boolean>;
    /**
     * Whether this resource has a valid current value.
     *
     * This function is reactive.     */
    hasValue(): this is Resource<Exclude<T, undefined>>;
    /**
     * Instructs the resource to re-load any asynchronous dependency it may have.
     *
     * Note that the resource will not enter its reloading state until the actual backend request is
     * made.
     *
     * @returns true if a reload was initiated, false if a reload was unnecessary or unsupported
     */
    reload(): boolean;
}
```

resource

- Donc **Tout objet qui étend Resource<T> encapsule une donnée asynchrone de type T** (par exemple, un objet JSON ou un tableau) et **fournit des signaux réactifs** pour suivre :
  - La **valeur** actuelle de la ressource.
  - **L'état** de la requête (en cours, réussie, échouée).
  - Les **erreurs** éventuelles.
  - Le **statut de chargement.**

resource
value

- **value**: Signal<T> est un signal réactif qui contient la valeur actuelle de la ressource.
- Cette valeur est de type T (le type générique défini lors de la création de la ressource).
- Si **aucune valeur n'est disponible** (par exemple, avant le chargement ou en cas d'erreur), la valeur est **undefined**.
- Exemple d'utilisation : Dans un template Angular, on peut afficher la valeur avec **{{ resource.value() }}**.Cas d'usage : Accéder aux données chargées, comme un objet JSON d'une API.

resource status

- status: **Signal**<**ResourceStatus**> est un signal réactif indiquant l'état actuel de la ressource. Dans la v19 status retourne un enum, dans la v20 elle s'est transformée en string

- Les valeurs possibles de **ResourceStatus** sont :
  - **idle** : Aucun chargement en cours, état initial.
  - **loading** : La ressource est en train de charger une nouvelle valeur.
  - **resolved** : La requête a réussi, une valeur est disponible.
  - **error** : La requête a échoué, une erreur est disponible.
  - **reloading** : La ressource recharge une valeur existante.

```
<!-- v19 -->
@if (cvResource.status() === 2 ) {
  <div >Chargement...</div>
}
<!-- v20 -->
@if (cvResource.status() === 'loading')
{
  <div >Chargement...</div>
}
```

resource
error et isLoading

- **error** : **Signal**<**unknown**> est un signal réactif indiquant l'erreur si elle existe, undefined sinon

- **isLoading** : Signal<boolean> est un signal réactif indiquant si la resource est en cours de chargement

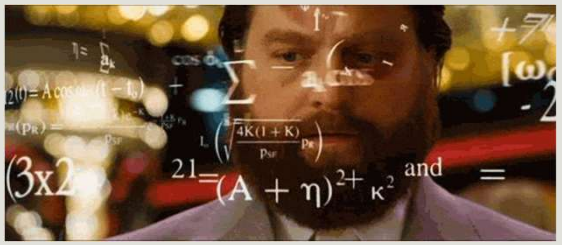
```
@if (cvResource.isLoading()) {
  <div class="alert alert-info">
    Loading ...
  </div>
}

@if (cvResource.error()) {
<div class="alert alert-error">
  Une erreur s'est produite
</div>
}
```

resource
reload et update

- **reload**: fonction qui permet de **relancer le loader**
- **update:** fonction qui **agit** comme le **update d'un signal** et qui **met à jour la value de la resource**
- **set:** fonction qui **agit** comme le **set d'un signal** et qui **met à jour la value de la resource**

Exercice



- Reprenez le composant Cv et faite en sorte qu'il utilise l'api resource
- Remarque avec fetch quand il y a une réponse 404 ou 500 elle ne déclenche pas une erreur. Vous devez donc la gérer manuellement.

```
loader: async ({params}) => {
      const response = await fetch(API.route);
      if (!response.ok) throw Error('Cv Not Found')
      return cv.json();
    }
```

rxResource

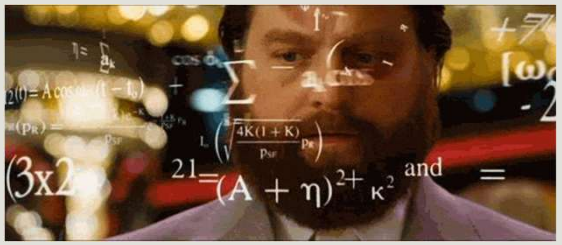
- L'API rxResource est conçue pour gérer **des données asynchrones** basées sur des **Observables**.
- Elle offre une **gestion déclarative des états asynchrones** et **annule automatiquement les requêtes précédentes si les paramètres changent** (sémantique switchMap).
- Elle est très ressemblante à resource mais **la propriété loader** est défini via **la propriété stream,** qui prend **exactement les mêmes paramètres que loader,** car **un rxResource** est une ressource dite **de streaming émettrice** qui peut **émettre plusieurs valeurs au fil du temps.**

rxResource

```
cvResource = rxResource<Cv | null, {id:number}>({
    // fonction qui détermine à quel moment on va déclencher le loader
    params: () => ({id: this.id()}),
    // loader va aller chercher le résultat du backend
    stream: ({params, abortSignal, previous}) => {
        return this.cvService.findById(params.id);
    },
    defaultValue: null
  });
```

- **params (request avant Angular 20)** prend en paramètre une **fonction** ou un **paramètre** spécifiant les **signals** dont dépend votre **resource**.
- **stream** permet de spécifier l'Observable qui ira **chercher les données**
- **defaultValue** permet de spécifier la **valeur par défaut**

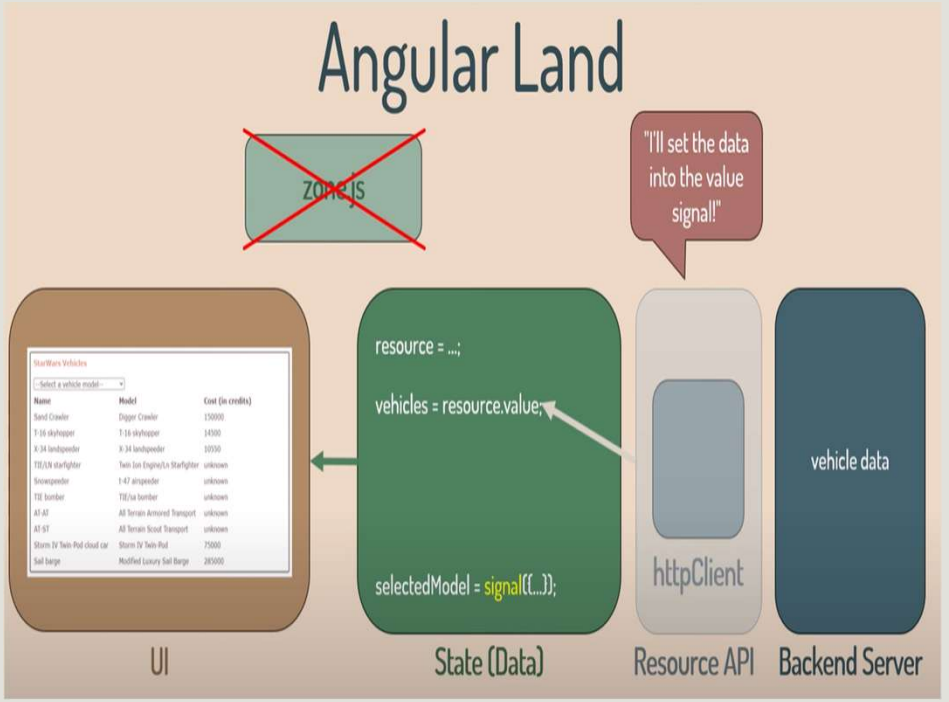
Exercice



- Reprenez le composant ProductComponent avec rxResource

Signal et les appels HTTP httpResource

- Une troisième api a été introduite dans **Angular 19.2**, c'est l'api **httpResource**.
- L'idée est de proposer un **wrapper sur le HttpClient** et qui soit plus simple et plus proche de la syntaxe du HttpClient que des apis resource et rxResource
- Comme pour les deux autres apis il retourne un objet de type **Resource, donc il expose tous ces propriétés comme value, error, status et isLoading.**
- Principalement destiné aux requêtes GET (non recommandé pour les mutations comme POST, PUT, DELETE).



https://www.youtube.com/watch?v=YHZkiUbbeOg&ab_channel=DeborahKurata

Signal et les appels HTTP httpResource

- Dans **Angular 19.2** elle pouvait récupérer une **chaine, une fonction ou un objet de type HttpResourceRequest** permettant de décrire l'appel http à effectuer.
- A partir de la **version 20** on ne peut utiliser **qu'une fonction qui retourne une chaine ou un objet de type HttpResourceRequest.**
- L'introduction de httpResource rend la gestion des requêtes HTTP **plus déclarative et réactive**.
- **Au contraire du HttpClient, la httpResource déclenche son appel http dés sa création**. Ensuite elle **se recharge automatiquement** et effectue une nouvelle requête à **chaque changement d'un signal source** dont dépend la fonction.
- Dans cet exemple, à chaque fois que le **signal search change un nouvel appel est exécuté**

```
userHttpRessource = httpResource(() => API.users + `?name_like=^${this.search()}`);
```

Signal et les appels HTTP httpResource

- Si vous avez besoin d'un **contrôle plus précis sur la requête**, vous pouvez transmettre une fonction qui renvoie un objet **HttpResourceRequest**.
- Outre la propriété **URL**, elle peut spécifier et réagir à des options telles que la **méthode**, les **paramètres** ou **les en-têtes.**

```
userHttpRessource = httpResource(() => ({
  url: API.users,
  params: new HttpParams().set('name_like', `^${this.search()}` ),
  method: 'GET'
}));
```

```ts
interface HttpResourceRequest {
    /**
     * URL of the request.
     * This URL should not include query parameters. Instead, specify query parameters through the
     * `params` field.
     */
    url: string;
    /**
     * HTTP method of the request, which defaults to GET if not specified.
     */
    method?: string;
    /**
     * Body to send with the request, if there is one.
     * If no Content-Type header is specified by the user, Angular will attempt to set one based on
     * the type of `body`.
     */
    body?: unknown;
    /**
     * Dictionary of query parameters which will be appeneded to the request URL.
     */
    params?: HttpParams | Record<string, string | number | boolean | ReadonlyArray<string | number | boolean>>;
    /**
     * Dictionary of headers to include with the outgoing request.
     */
    headers?: HttpHeaders | Record<string, string | ReadonlyArray<string>>;
    /**
     * Context of the request stored in a dictionary of key-value pairs.
     */
    context?: HttpContext;
    /**
     * If `true`, progress events will be enabled for the request and delivered through the
     * `HttpResource.progress` signal.
     */
    //...
}
```

https://angular.dev/api/common/http/HttpResourceRequest

Signal et les appels HTTP
httpResource
Default value

- Par défaut la resource retourne undefined
- **Si vous souhaitez définir une valeur par défaut, passez en deuxième paramètre à votre foncion httpResource, un objet avec l'option defaultValue**

```
userHttpRessource = httpResource<UserR[]>(
  () => ({
    url: API.users,
    params: new HttpParams().set('name_like',`^${this.search()}`),
    method: 'GET',
  }),
  {
    defaultValue: []
  }
);
```

Signal et les appels HTTP
httpResource
Retarder l'appel Http

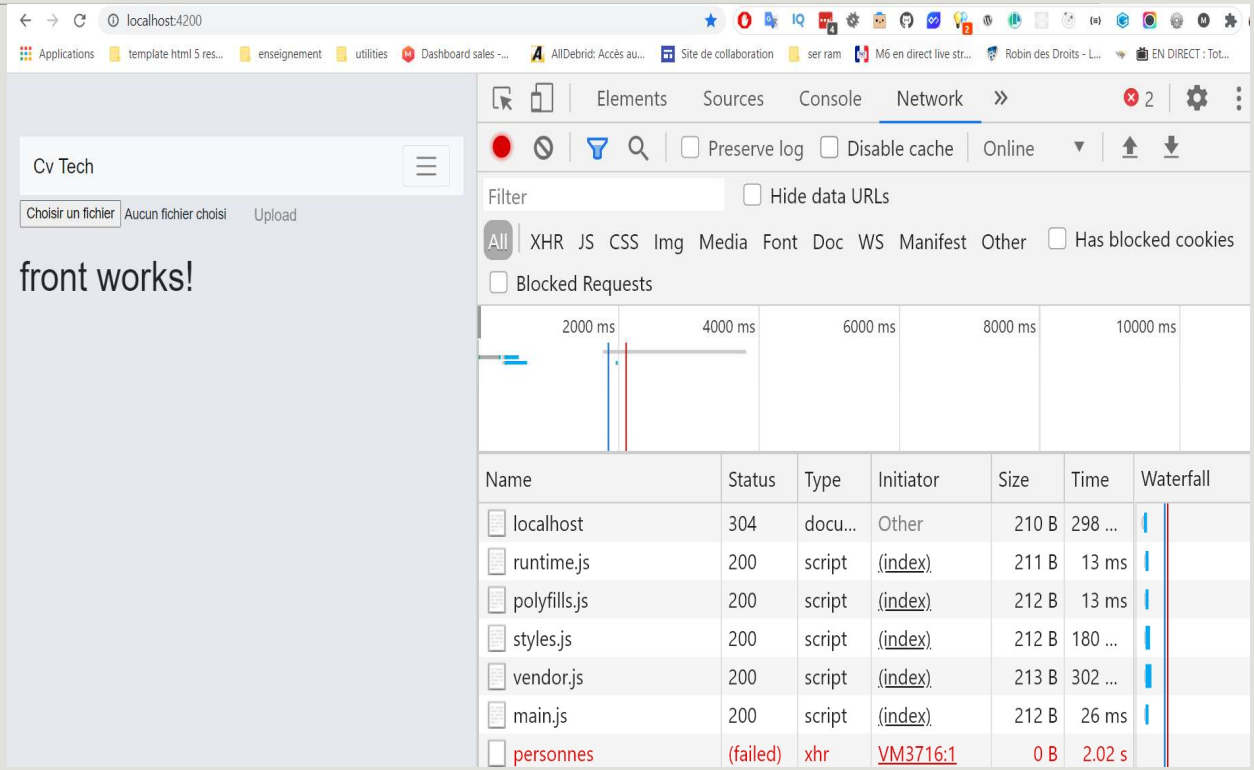
- Comme mentionné au début, la **httpResource déclenche son appel http dés sa création**.
- **Si vous souhaitez retardé l'appel vous pouvez retourné undefined jusqu'au besoin de l'appel http.**

```
userHttpRessource = httpResource<UserR[]>(
    () => {
      const allow = this.allowHttp();
      return allow
        ? {
            url: API.users,
            params: new HttpParams().set('name_like', `^${this.search()}`),
            method: 'GET',
          }
        : undefined;
    }
  );
```

Lazy Loading

- Par défaut, tout les modules que vous déclarer au niveau du AppModule sont chargé au lancement de l'application.
- Ceci pose un problème au niveau du Bundle généré de votre application.
- Une grande application aura une taille assez conséquente ce qui peut provoquer un problème au chargement de l'application et donc un problème d'expérience utilisateur.
- L'idée du lazyLoading et de charge au départ le module principale et puis de ne charger un module que si on appelle l'une de ses routes.
- Ceci va nous faire gagner en performance.

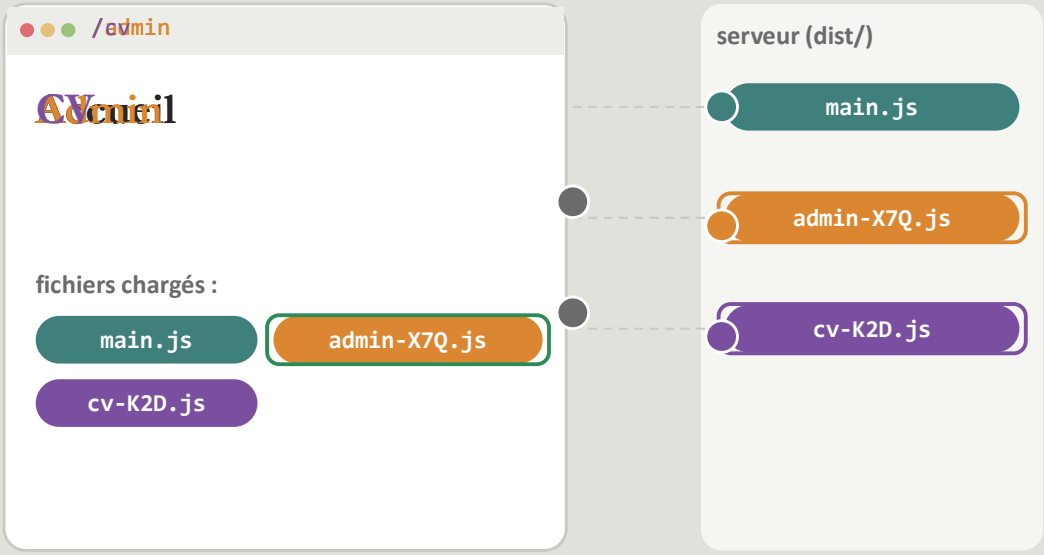




Lazy loading : chaque chunk arrive à la demande

```ts
export const routes: Routes = [
  { path: '', component: Accueil },
  { path: 'admin',
    loadComponent: () =>
      import('./admin/admin') },
  { path: 'cv',
    loadChildren: () =>
      import('./cv/cv.routes') },
];
```

au démarrage : main.js seulement, admin et cv ne sont pas déjà chargé aucune nouvelle requête téléchargés



Au démarrage, seul le bundle principal est téléchargé. `loadComponent` et `loadChildren` produisent chacun un chunk, téléchargé à la première navigation vers la route, puis gardé en mémoire.

Lazy Loading

- Afin d'implémenter l'ancienne façon d'implémenter le *Lazy Loading* des modules, on doit suivre les étapes suivantes :

1. Le **module** à charger doit **lui-même gérer sa partie routing**
2. Au niveau du routing principal (AppRoutingModule), créer une **route**, ajouter lui un path 'ca sera le **préfixe** de toutes les routes du module', et ajouter une nouvelle clé qui est **loadChildren.** Cette clé va informer angular et lui demander de ne charger le module associé que lorsque on appelle le path défini.
3. Ce **paramètre** prend ou une **chaine de caractère** qui spécifie le module à charger ou une callback function.
4. Finalement **enlever les imports des modules lazy loaded** au niveau du AppModule

```
{
  path: "cv",
  loadChildren: "./cv/cv.module#CvModule",
},
```

```
{
   path: "cv",
   loadChildren: () => import('./cv/cv.module').then(
     m => m.CvModule
   ),
 },
```

Lazyload Standalone Component

- Maintenant qu'un composant est devenu indépendant de tout module vous pouvez le charger d'une façon fainéante.
- Vous pouvez faire du **lazy loading** pour les **standalone component** avec la clé **loadComponent.**

```
{
  path: 'track',
  loadComponent: () => import('./track-by/track-by.component')
                         .then(c => c.TrackByComponent)
}
```

Lazyload groupement de Standalone Component

- Vous pouvez faire du **lazy loading** pour une configuration de routing (groupement de route de **standalone component**) avec la clé **loadChildren.**

```
{
    path: 'cv',
    loadChildren: () => import('./cv/cv.routing').then((m) => m.CV_ROUTES),
  },
```

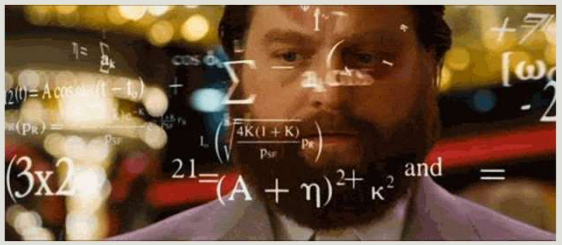
Lazyloading et default exports

- Lors de l'utilisation de loadChildren et loadComponent, le routeur **comprend et décompresse automatiquement les appels d'importation dynamiques import()** avec les **exportations par défaut.**
- Vous pouvez en profiter pour ignorer le .then() pour de telles opérations de chargement différé.

```
export default  [
  { path: '', component: CvComponent},
]
```

```
{
    path: 'cv',
    loadChildren: () => import('./cv/cv.routing')
  },
```

Exercice
LazyLoading



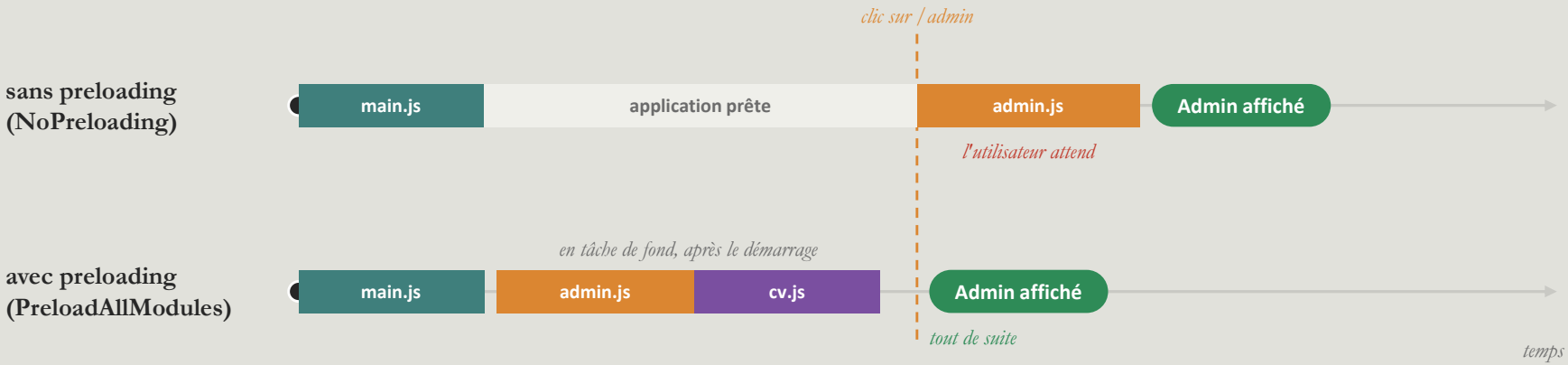
- Faites du lazyLoading pour le composant MiniWordComponent

Preloading Lazy Loading

- Le **problème** qu'on peut identifier avec le **lazy loading** est le fait qu'en passant d'un module à un autre on aura **toujours un chargement des nouveaux modules**.
- Si vos **modules** sont **très volumineux** ou que la connexion du client est mauvaise, il y aura **plusieurs latences**. Ceci va provoquer un problème avec l'utilisateur.
- La question qui se pose est : **Y a-t-il un moyen de personnaliser les stratégies de chargement** ???

Preloading : télécharger les chunks en tâche de fond

```
// app.config.ts
provideRouter(routes, withPreloading(PreloadAllModules))
```



`PreloadAllModules` télécharge tous les chunks lazy dès que l'application a démarré : la navigation devient immédiate, et le démarrage reste léger. Le prix : télécharger aussi ce que l'utilisateur ne visitera pas.

Preloading Lazy Loading

- **Dans les applications modulaires**, la méthode **forRoot** de votre RouterModule prend en **second paramètre un objet** vous permettant de **configurer** la **stratégie** de **chargement** avec la propriété **preloadingStrategy**.
- Cette propriété prend en paramètre, par défaut **NoPreloading**.
- La deuxième valeur qu'elle peut prendre est **PreloadAllModules. Elle demande à Angular de précharger tous les lazyLoaded Modules une fois le Module principal chargé.**

```
import { PreloadAllModules } from "@angular/router";
@NgModule({
  imports: [RouterModule.forRoot(routes, {
    preloadingStrategy: PreloadAllModules
  })],
  exports: [RouterModule],
})
```

vendor.js	200	script	(index)	213 B	270 ms
main.js	200	script	(index)	212 B	25 ms
personnes	(failed)	xhr	VM14822:1	0 B	2.01 s
personnes	(failed)		Other	0 B	2.00 s
common.js	200	script	bootstrap:149	210 B	9 ms
cv-cv-module.js	200	script	bootstrap:149	211 B	9 ms
todo-todo-module.js	200	script	bootstrap:149	211 B	8 ms

Preloading Lazy Loading

- **Dans les applications standalone**, passez la fonction withPreloading comme paramètre de votre fonction provideRouter.
- Passez lui le provider PreloadAllModules.

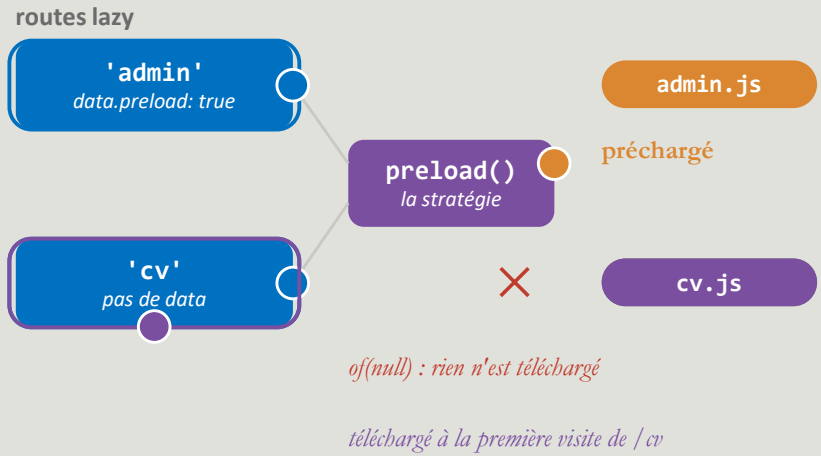
```
provideRouter(
      routes,
      withPreloading(LaStrategyDePreload)
),
```

Stratégie personnalisée : précharger ce qui compte

```
export const routes: Routes = [
  { path: 'admin', data: { preload: true }
    loadComponent: () => import('./admin') },
  { path: 'cv',
    loadChildren: () => import('./cv.routes') },
];

@Injectable({ providedIn: 'root' })
export class SelectivePreloading
    implements PreloadingStrategy {
  preload(route: Route, load: () => Observable<any>) {
    return route.data?.['preload'] ? load() : of(null)
  }
}

withPreloading(SelectivePreloading)
```



Le routeur appelle `preload()` pour chaque route lazy, après le démarrage. Retourner `load()` précharge le chunk ; retourner `of(null)` le laisse au lazy loading classique.

Preloading Lazy Loading
Créer votre propre startégie

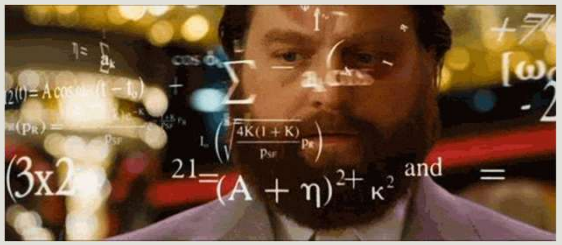
- Avec **PreloadAllModules**, tous les modules sont **préchargés**, ce qui peut en fait créer un **goulot d'étranglement** si l'application a un **grand nombre de modules à charger**.
- Une meilleure stratégie serait de **charger sélectivement les modules requis au démarrage**. Par exemple, module d'authentification, module principal, module partagé, etc.
- Pour **précharger sélectivement** un module, nous devons utiliser une **stratégie de préchargement personnalisée**.
- Créez d'abord une **classe** qui implémente l'interface **PreloadingStrategy**. La classe doit implémenter la méthode **preload().**
- C'est cette méthode qui détermine s'il **faut précharger le module ou non.**

Preloading Lazy Loading
Créer votre propre startégie

- La signature de la fonction **preload** prend en paramètre un **objet Route** représentant la route ciblée et en deuxième paramètre une fonction **load** qui retourne un Observable.
- Si vous retourne la méthode load, le module sera **préchargé**.
- Si vous **ne voulez pas le précharger** retourner un **Observable de null**.

```
@Injectable({providedIn: 'root'})
export class CustomPreloadingStrategy implements PreloadingStrategy {
   preload(route: Route, load: () => Observable<any>): Observable<any> {
      if (route.data["preload"]) {
         return load();
      }
      else {
         of(null);
      }
   }
}
```

Exercice
Preloading Stratégy



- Appliquez la Custom Preloading Strategy

Lazy Loading Migration

- Depuis Angular 18.2, il existe une migration permettant de transformer vos routes en lazy-loading components.

```
ng generate @angular/core:route-lazy-loading
```

- Vous pouvez aussi l'appliquer sur une partie de vos composant en passant le paramètre path

```
ng generate @angular/core:route-lazy-loading --path src/app/sub-component
```

https://angular.dev/reference/migrations/route-lazy-loading

Angular Reactive Form

AYMEN SELLAOUTI

Reactive form

- Les Reactive Form sont une deuxième méthode de gérer vos formulaires avec Angular.
- Au contraire des Template Driven Form, ses formulaires sont **générés programmatiquement** dans la partie TS.
- Ceci permet **d'alléger le template** des validateurs.
- D'autre part ca permet d'avoir un **formulaire testable**
- Finalement ceci permet de plus facilement **générer des Validateurs personnalisés.**

Reactive form
Créer un formulaire via FormGroup

- Commencer par ajouter le module **ReactiveFormModule**.
- Afin de créer un formulaire avec l'approche réactive, vous devez créer un objet **FormGroup**.
- Un **FormGroup** prend en paramètre un objet décrivant le formulaire. Chaque champ de l'objet a comme **première propriété le nom et comme valeur un objet définissant les champs associés à ce formulaire**. Ce sont les **FormControl**.
- Chaque **FormControl** définit un **champ du formulaire**. Il prend en paramètre, **la valeur initiale, un Validator ou un tableau de Validator et en troisième paramètre, un AsyncValidator ou un tableau d'AsyncValidator**

```
FormGroup.constructor(
    controls: {[p: string]: AbstractControl},
    validatorOrOpts?: ValidatorFn | ValidatorFn[] | AbstractControlOptions | null,
    asyncValidator?: AsyncValidatorFn | AsyncValidatorFn[] | null)
```

```
ngOnInit() {
  this.form = new FormGroup({
    name: new FormControl(null),
    firstname: new FormControl(null),
    age: new FormControl(null),
  });
```

Reactive form
Créer un formulaire

```
export declare interface AbstractControlOptions {
   validators?: ValidatorFn | ValidatorFn[] | null;
   asyncValidators?: AsyncValidatorFn | AsyncValidatorFn[] | null;
   /**
    * @description
    * The event name for control to update upon.
    */
   updateOn?: 'change' | 'blur' | 'submit';
}
```

```
form = new FormGroup({
  name: new FormControl(null),
  firstname: new FormControl("Aymen"),
  age: new FormControl(0, {
    asyncValidators: [],
    validators: Validators.required,
    updateOn: "blur",
  }),
});
```

Reactive form
Créer un formulaire via le service FormBuilder

- Vous pouvez aussi passer par le **service FormBuilder** et sa méthode **group**.
- Elle prend en **paramètre un objet** avec comme **clé le nom** du formControl et comme **valeur un tableau** avec comme **première propriété la valeur initiale du champ**.
- Ceci est **moins verbeux** et plus simple à gérer pour les grands formulaires.

```
this.form = this.formBuilder.group({
  email: [null],
  username: [null],
  userCategory: ['employee'],
});
```

```
constructor(
  private formBuilder: FormBuilder
) {}
```

Reactive form
Associer le FormGroup à votre form

- Une fois le formulaire définit, nous devons **l'associer au form de votre template**.
- Pour ce faire, vous devez ajouter la directive **formGroup** (qui se trouve dans le module **ReactiveFormsModule**) au niveau de la balise form et la binder à votre objet de type **FormGroup** au niveau de votre fichier TS.



Reactive form
Associer les FormControl à vos input

- Afin d'associer votre FormControl à votre champ input, utiliser la directive **formControlName** et associer le à **l'identifiant** du **FormControl**

```
this.form = new FormGroup( controls: {
  email: new FormControl( formState: null),
  username: new FormControl( formState: null),
  age: new FormControl( formState: null),
```

```
<div>
  <label for="email">Email</label>
  <input
    class="form-control" type="email" id="email"
    formControlName="email" placeholder="Enter email"/>
</div>
<div>
  <label for="username">Username</label>
  <input
    class="form-control" type="text" id="username"
    formControlName="username"  placeholder="Username"/>
</div>
```

Reactive form
Récupérer les FormControl dans le HTML

- Afin de **récupérer les FormControl** dans le **HTML**, utiliser la méthode **get** de votre **form group** et passer lui l'**identifiant** du **FormControl à récupérer**.

```
<button
  class="btn btn-primary"
  [disabled]="form.get('password').valid"
  (click)="process()">
  Submit
</button>
```

Reactive form
Récupérer les erreurs du FormControl dans le HTML

- Afin de **récupérer les erreurs de votre control** dans le **HTML**, utiliser l'attribut **errors**.

```
<div
  *ngIf="form.get('name')?.errors && form.get('name')?.touched"
   class="alert alert-danger“
>
```

Reactive form
Récupérer les FormControl dans le HTML

- Une deuxième méthode pour avoir un code moins verbeux est de créer des getters pour vos champs.

```
get name(): AbstractControl {
   return this.form.get("name")!;
 }
```

```
<div *ngIf="name.errors && name.touched">
   Ce champ est obligatoire
 </div>
```

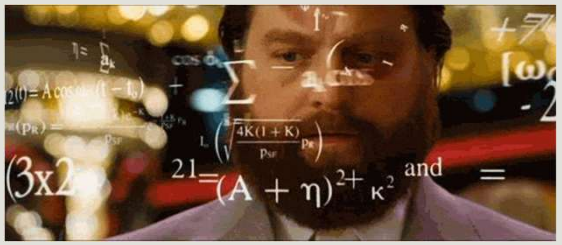
Reactive form
Soumettre le formulaire

- A l'inverse de l'approche 'template driven', vous n'avez pas besoin de récupérer la référence de l'objet form puisque vous l'avez déjà crée dans votre ts.
- Il suffit donc d'écouter le submit avec ngSubmit ou le clic sur un bouton pour déclencher la méthode du composant qui gérera l'envoi du formulaire.

```
process() {
  console.log(this.form);
}
```

```
34  <button
35    class="btn btn-primary"
36    (click)="process()">
37    Submit
38  </button>
```

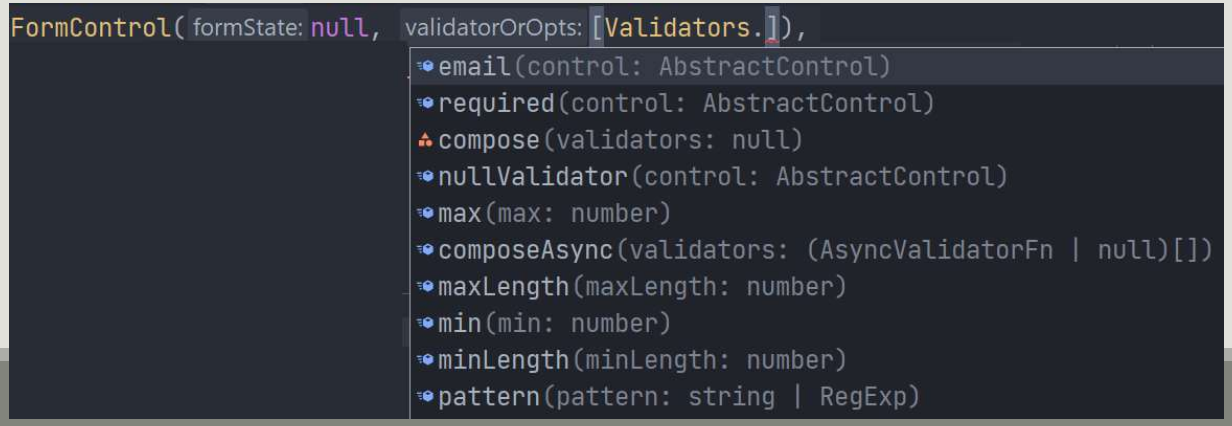
Exercice



- Reprenez le formulaire de login et gérer le avec les reactives form. Ne prenez pas encore en considération les validateurs.

Reactive form
Les Validateurs

- Un **validateur** est une **fonction** qui retourne **false si un champ est valide** selon une certaine condition **sinon elle retourne une information sur l'erreur.**
- Afin de valider votre formulaire, passer en deuxième paramètre de **FormControl** une **référence à une méthode de la classe Validators** ou un **tableau de ces méthodes**.



Reactive form
Les Validateurs

- **Validateurs synchrones** : Ce sont des fonctions qui contrôle votre élément et retournent un **ensemble d'erreurs de validation ou null**. C'est ce qu'on passe en deuxième paramètre lors de l'instanciation d'un FormControl.
- **Validateurs asynchrones** : Ce sont des fonctions asynchrones qui contrôle votre élément et retournent une **Promise ou un Observable** qui émettent un **ensemble d'erreurs de validation ou null**. C'est ce qu'on passe en troisième paramètre lors de l'instanciation d'un FormControl.
- Pour des raisons de **performance**, Angular commence par les validateurs synchrones, s'ils passent il déclenche les validateurs asynchrones.

Reactive form
Les Validateurs offerts par Angular

- Afin d'utilisez vos validateurs, ajoutez les dans les tableaux de validateurs associés à vos champs définis lors de la création de votre formulaire.

```
new FormGroup({
  email: new FormControl(null, [Validators.required, Validators.email]),
  username: new FormControl(null, [Validators.minLength(3)]),
  age: new FormControl(null, [
          Validators.required, Validators.pattern('[0-1]?\d{1,2}')
  ]),
});
```

Méthode 1

https://angular.io/api/forms/Validators

Reactive form
Les Validateurs offerts par Angular

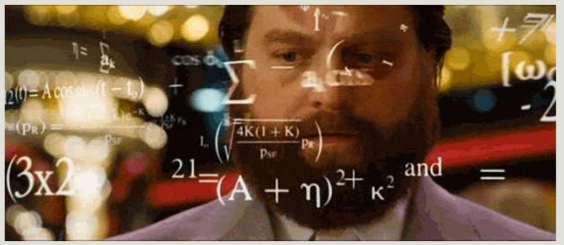
- Vous pouvez utiliser des validateurs offerts par angular ou créer vos propres validateurs.
- Les validateurs de base sont les mêmes que ceux de l'approche basée Template.

```
formGroup: FormGroup = new FormGroup({
   name: new FormControl(null, {
    validators: [Validators.required, Validators.minLength(3)],
    asyncValidators: [],
    updateOn: "change"
   }),
   age: new FormControl(null),
 });
```

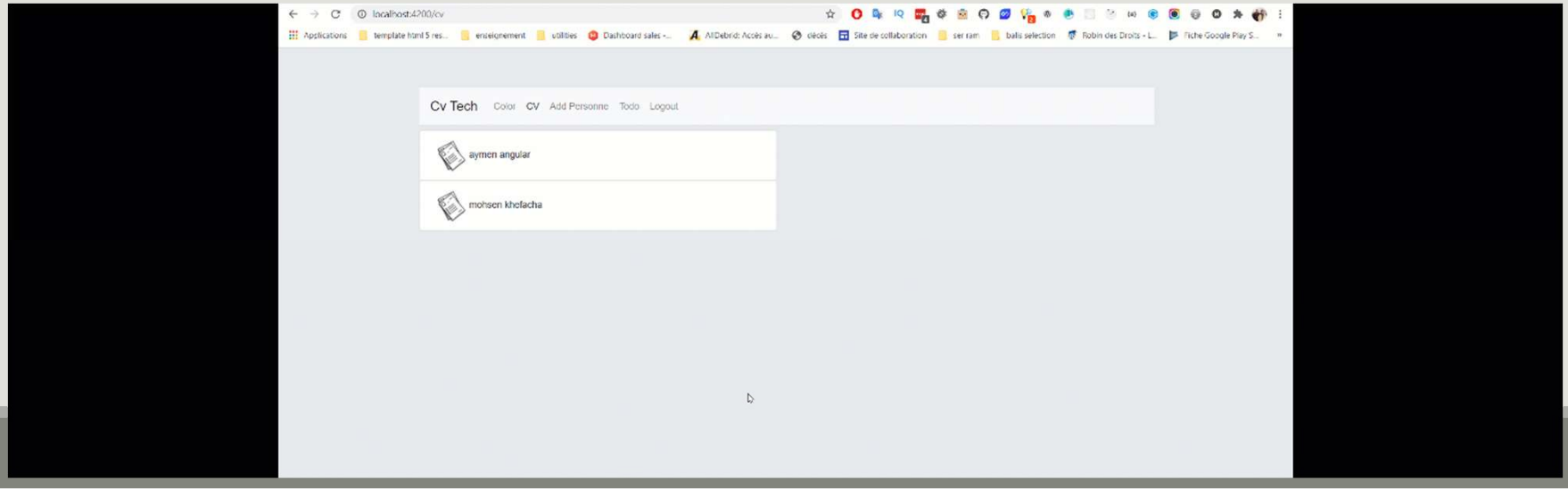
Méthode 2

https://angular.io/api/forms/Validators

Exercice



- Ajouter dans votre cvTech un composant contenant un formulaire. Ce formulaire devra vous permettre d'ajouter un utilisateur.
- Ajouter les validateurs nécessaires.
- Après l'ajout fowarder le user vers la liste des cvs.



Reactive form
Manipulez les valeurs de votre form

- Afin de **mettre à jour la valeur** d'un **FormControl ou d'un FormGroup**, vous pouvez utiliser la méthode **setValue()** qui met à jour la valeur du contrôle de formulaire et valide la structure de la valeur fournie par rapport à la structure du contrôle.
- Vous pouvez utiliser la méthode **patchValue** si vous modifiez uniquement **une partie de votre formulaire**.

```
this.form.setValue({ name: "Sellaouti", firstname: "Aymen" });

this.form.patchValue({firstname: "Aymen" });
```

https://angular.io/guide/reactive-forms#replacing-a-form-control-value

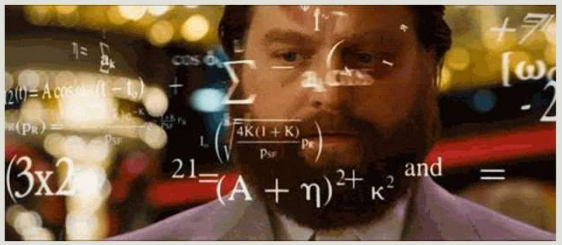
Reactive form
Suivre les modifications de vos FormControls et de vos FormGroup

- **FormControl et FormGroup**, vous fournissent deux **Observables** permettant le **suivi des changements de valeur et de statuts**.
- **ValueChanges** est un événement déclenché par les formulaires Angular **chaque fois que la valeur de FormControl, FormGroup** change. L'observable obtient la dernière valeur du contrôle. Il nous **permet de suivre les modifications apportées à la valeur en temps réel et d'y répondre**. Par exemple, nous pouvons l'utiliser pour valider la valeur, calculer les champs calculés, ...

Reactive form
Suivre les modifications de vos FormControls et de vos FormGroup

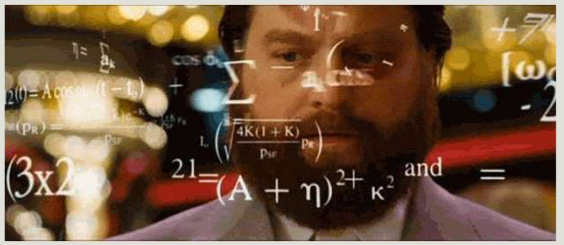
- **statusChanges** est un événement déclenché par les formulaires Angular **chaque fois que Angular calcule le statut de validation** de FormControl, FormGroup. Il renvoie un observable afin que vous puissiez vous y abonner. L'observable obtient le **dernier état du contrôle**.

Exercice



- Afin de protéger les informations personnelles des mineurs, faites en sortes que lorsque l'age de la personne possédant le Cv est inférieur à 18 ans, il ne puisse pas renseigner le path de l'image.

Exercice



- Etant donné que notre formulaire est assez volumineux et pour permettre une meilleure expérience utilisateur, nous voulons faire en sorte que si l'utilisateur saisisse un formulaire valide et qu'il sorte de la page sans l'envoyer, il puisse le retrouver rempli lorsqu'il retourne à la page d'ajout d'un cv.

Reactive form
Customiser vos validateurs

- Un custom validator est une fonction de type **ValidatorFn** qui prend en paramètre un control de type **AbstractControl** (qui contient une propriété **value** représentant la **valeur du champ à valider**) et **retourne null si la valeur est valide** ou un objet de type **ValidationErrors** s'il y a des **erreurs de validation**.

```
export declare interface ValidatorFn {
   (control: AbstractControl): ValidationErrors | null;
}
```

```
return !passwordValid ? {passwordStrength: true} : null;
```

Reactive form
Customiser vos validateurs

```
export function createPasswordStrengthValidator(): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const value = control.value;
    if (!value) {
      return null;
    }
    const hasUpperCase = /[A-Z]+/.test(value);
    const hasLowerCase = /[a-z]+/.test(value);
    const hasNumeric = /[0-9]+/.test(value);
    const passwordValid = hasUpperCase && hasLowerCase && hasNumeric;
    return !passwordValid ? {passwordStrength: true} : null;
  };
}
```

Reactive form
Customiser vos validateurs
Validateurs Asynchrones

- Dans certains cas d'utilisation, la validation de votre champ ne se fait pas d'une façon **asynchrone**.
- Le cas le plus répandu est la **validation par votre backend**.
- Imaginez que vous avez un champ email et qu'il ne doit pas être redondant. La solution et d'avoir une API qui vérifie ca et qui vous retourne la réponse.
- Ce traitement étant Asynchrone, le Validateur synchrone ne fait plus affaire.
- Il faut donc créer un validateur **ASYNCHRONE**.

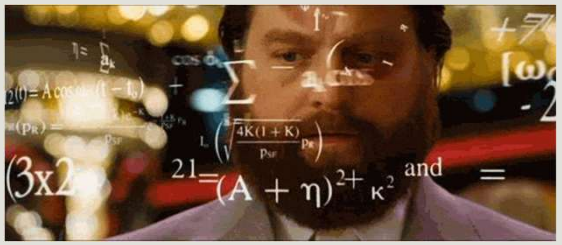
Reactive form
Customiser vos validateurs
Validateurs Asynchrones

- Le Validator, renvoyée par la fonction de création de validateur, doit suivre ces règles :
- **Un seul argument** d'entrée est attendu, qui est de type **AbstractControl**. La fonction validateur peut obtenir la **valeur à valider** via la propriété **control.value**
- La fonction de validation **doit renvoyer une Promise<null>, un Observable<null>** si **aucune erreur** n'a été trouvée dans la valeur du champ, ce qui signifie que la **valeur est valide**
- Si des **erreurs de validation sont trouvées**, la fonction doit renvoyer **une Promise ou un Observable** d'un objet de type **ValidationErrors.**

Reactive form
Customiser vos validateurs
Validateurs Asynchrones

```
export function userExistsValidator(authService: AuthService): AsyncValidatorFn  {
  return (control: AbstractControl) => {
    return authService.findUserByEmail(control.value);
  };
}
```

Exercice



- Le champ Cin étant unique, créer un validateur asynchrone permettant de gérer cette contrainte et tester le.

Reactive form
Customiser vos validateurs
Valider un form

- Afin de valider un form, vous devez faire la même chose que pour le validateur d'un FormControl.

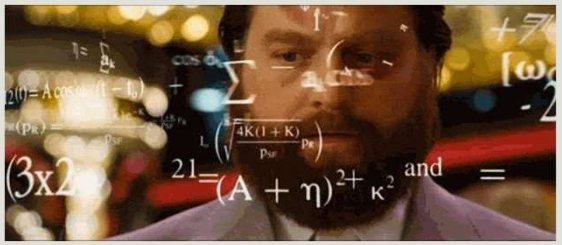
```
export function creatDateRangeValidator(): ValidatorFn {
  return (form: AbstractControl): ValidationErrors | null => {
    const start: Date = form.get('startAt').value;
    const end: Date = form.get('endAt').value;
    if (start && end) {
      const isRangeValid = (end.getTime() - start.getTime() > 0);
      return isRangeValid ? null : {dateRange: true};
    }
    return null;
  };
}
```

Reactive form
Customiser vos validateurs
Valider un form

- Maintenant et pour l'appliquer à votre FormGroup ajouter à la création de votre FormGroup un second paramètre qui est un objet options et utiliser la propriété validators

```
this.form = new FormGroup(
  {},
  {
    validators: VotreValidateur
  }
);
this.form = this.formBuilder.group(
 {},
 {
   validators: VotreValidateur
 }
);
```

Exercice



- Le champ Cin étant composé de 8 caractères numériques, il présente une corrélation entre l'age de la personne et les deux premiers caractères.
- Si la personne a un age >= 60 ans, les deux premiers caractères numériques doivent être entre 00 et 19.
- Sinon ca doit être supérieur à 19.
- Créer le validateur permettant de faire ca.

Angular
Les Signal Forms

AYMEN SELLAOUTI

Objectifs

- À la fin de ce chapitre vous serez capable de :
- Expliquer pourquoi Angular introduit une **troisième** approche de formulaires
- Créer un formulaire à partir d'un simple **signal** avec `form()`
- Valider vos formulaires
- Gérer les classes Css

Les formulaires dans Angular
Où en est-on ?

- Depuis ses débuts Angular propose **deux** approches pour gérer les formulaires :
  - L'approche **Template Driven** : la logique est dans le template, avec `ngModel`. Simple, mais peu testable et difficile à faire évoluer
  - L'approche **Reactive** : la logique est dans le composant, avec `FormGroup` et `FormControl`. Puissante, mais verbeuse
- Angular 22 stabilise une **troisième** approche : les **Signal Forms**, dans le package `@angular/forms/signals`
- Ce n'est pas une évolution des Reactive Forms, c'est une **remise à plat** complète construite autour des signaux
- Bonne nouvelle pour l'avenir, mais cela signifie aussi une troisième façon d'écrire des formulaires à apprendre

Signal Forms
Le modèle mental

- Le point de départ n'est plus le formulaire, c'est **votre donnée** : un simple `signal` qui contient la valeur du formulaire
- La fonction `form()` ne crée pas une copie de cette donnée, elle crée une **vue éditable** dessus : un `FieldTree`
- Une **seule** source de vérité : si l'utilisateur saisit dans un champ, le signal est mis à jour ; si le code modifie le signal, le champ affiche la nouvelle valeur
- Le `FieldTree` porte toute la **mécanique** : état d'interaction, validation, soumission
- Retenez la formule : **le signal porte la valeur, le FieldTree porte l'état**

Signal Forms
Le modèle mental

```
// La donnée métier                          La mécanique de formulaire
//   signal({ login, password })   ->    form(...)   ->   FieldTree
//          ^                                                  |
//          |_____________ liaison bidirectionnelle ___________|
```

Signal Forms
Disponibilité et import

- Introduits en **expérimental** dans Angular 21, les Signal Forms sont **stables** depuis Angular 22
- Tout se trouve dans le sous-package **`@angular/forms/signals`**, distinct de **`@angular/forms`**
- Aucun provider à déclarer, aucun module à importer : **`form()`** est une simple fonction et **`FormField`** une directive standalone
- Les deux approches historiques restent supportées : vous pouvez faire cohabiter Reactive Forms et Signal Forms dans une même application

Signal Forms
Disponibilité et import

```
import { form, required, email, minLength } from '@angular/forms/signals';
```

Créer un formulaire form()

- Première étape : déclarer un **signal** qui contient la valeur du formulaire
- Deuxième étape : appeler `form()` avec ce signal. La fonction retourne un `FieldTree`
- `form()` doit être appelée dans un **contexte d'injection**, donc à l'initialisation d'un champ ou dans le constructeur
- Le type du `FieldTree` est **déduit** de la forme du signal : aucune déclaration de type à écrire

Créer un formulaire form()

```ts
// login.ts
// 1. le signal qui porte la valeur du formulaire
private readonly credentials = signal({
  login: '',
  password: ''
});

// 2. le FieldTree créé à partir du signal
protected readonly loginForm = form(this.credentials);
```

Lier les champs
La directive FormField

- Une **seule** directive suffit pour lier un `input`, un `select` ou un `textarea` à un champ : `[formField]`
- Plus de `formControlName` ni de `formGroup` : la directive reçoit directement le champ du `FieldTree`
- La liaison est **bidirectionnelle** : la saisie met à jour le signal, et inversement
- Elle positionne aussi les attributs HTML `required`, `disabled` et `readonly`
- Et une pour le form `[formRoot]`

Lier les champs
La directive FormField

```
<!-- login.html -->
<form [formRoot]="loginForm">
  <label for="login">Login</label>
  <input id="login" [formField]="loginForm.login" />

  <label for="password">Mot de passe</label>
  <input id="password" type="password" [formField]="loginForm.password" />

  <button type="submit">Se connecter</button>
</form>
```

Le FieldTree
Le concept clé

- L'objet retourné par **`form()`** est un **`FieldTree`**. C'est le concept central des Signal Forms
- Il reproduit la **structure** de votre donnée : si le signal contient **`{ login, password }`**, alors **`loginForm.login`** et **`loginForm.password`** sont eux aussi des **`FieldTree`**
- Comme un signal, un **`FieldTree`** est un objet **qui est aussi une fonction** : si vous l'appelez, il retourne un objet **`FieldState`**
- Retenez la distinction : **`loginForm.login`** est le **champ** que l'on passe à la directive, **`loginForm.login()`** est son **état** que l'on lit dans le template

Le FieldTree
Le concept clé

```
loginForm                    // FieldTree de tout le formulaire
loginForm.login              // FieldTree du champ login
loginForm()                  // FieldState du formulaire
loginForm.login()            // FieldState du champ login
loginForm.login().value()    // la valeur du champ login
```

Le FieldState
Les signaux d'état

- Toutes les propriétés du **`FieldState`** sont des **signaux**, donc lisibles directement dans le template :
  - **`value()`** : la valeur courante du champ, elle peut être temporisée (debounce).
  - **`controlValue()`** : la valeur réellement présente dans le contrôle, toujours à jour
  - **`touched()`** : le champ a-t-il été visité
  - **`dirty()`** : le champ a-t-il été modifié
  - **`disabled()`** et **`disabledReasons()`** : le champ est-il désactivé, et pour quelles raisons
  - **`hidden()`** : le champ est-il masqué
  - **`readonly()`** : le champ est-il en lecture seule
  - **`submitting()`** : le champ est-il en cours de soumission

Le FieldState
Les signaux de validation

- **`valid()`** : tous les validateurs sont passés. **ATTENTION** : **`valid()`** vaut **`false`** tant que les validateurs asynchrones ne sont pas terminés
- **`invalid()`** : au moins un validateur a échoué
- **`pending()`** : une validation asynchrone est en cours
- **`errors()`** : la liste des erreurs **du champ**
- **`errorSummary()`** : la liste des erreurs du champ **et de tous ses sous-champs**. Très pratique pour un récapitulatif en haut de formulaire
- **`getError(kind)`** : récupère la première erreur d'un type donné, par exemple **`getError('required')`**

Le FieldState
Les méthodes

- Le **`FieldState`** expose aussi quelques méthodes pour agir sur l'état du champ :
  - **`reset()`** : remet le champ à l'état vierge et non visité. **ATTENTION** : cela ne remet pas la valeur à zéro, pour cela modifiez votre signal
  - **`markAsTouched()`** et **`markAsDirty()`** : marquent le champ comme visité ou modifié. Depuis la v22, **`markAsTouched()`** marque aussi **les descendants**, sauf si vous passez **`{ skipDescendants: true }`**
  - **`reloadValidation()`** : relance tous les validateurs asynchrones du champ et de ses descendants
- Pour remettre le formulaire à sa valeur initiale, il suffit de faire un **`set()`** sur votre signal source

Le FieldState
Les méthodes

```
resetForm() {
  this.credentials.set({ login: '', password: '' }); // la valeur
  this.loginForm().reset();                          // l'état
}
```

La validation
Les validateurs intégrés

- Angular fournit les mêmes validateurs que pour les formulaires classiques, mais sous forme de **fonctions** qui prennent le champ en premier paramètre :
  - `required(field)` : champ obligatoire
  - `minLength(field, n)` et `maxLength(field, n)` : longueur minimale et maximale, pour les chaînes **et** les tableaux
  - `min(field, n)` et `max(field, n)` : valeur minimale et maximale pour les nombres
  - `minDate(field, d)` et `maxDate(field, d)` : bornes pour les dates, nouveautés d'Angular 22
  - `email(field)` : adresse email valide
  - `pattern(field, regex)` : validation par expression régulière

La validation
Appliquer les validateurs

- Chaque validateur accepte un objet d'options avec un **`message`**. Il n'y a **aucun** message par défaut : c'est vous qui le fournissez
- Le message peut aussi être une **fonction** qui reçoit le champ, pratique pour un message dynamique

La validation
Appliquer les validateurs

```ts
// register.ts
protected readonly accountForm = form(this.credentials, form => {
  required(form.email, { message: 'Email obligatoire' });
  email(form.email, { message: 'Email invalide' });
  required(form.password, { message: 'Le mot de passe est obligatoire' });
  minLength(form.password, 6, {
    message: password =>
      `Au moins 6 caractères, actuellement ${password.value().length}`
  });
});
```

Les classes CSS automatiques provideSignalFormsConfig()

- Vous pouvez aussi appliquer **automatiquement** des classes CSS à vos champs selon leur état
- On déclare la configuration une seule fois dans **`app.config.ts`**, et toutes les directives **`formField`** de l'application en bénéficient
- Vous branchez ainsi les classes de Bootstrap sans écrire un seul **`[class.x]`** dans vos templates

```
// app.config.ts
provideSignalFormsConfig({
  classes: NG_STATUS_CLASSES
})
```

La validation
La ValidationError

- Quand un validateur échoue, une **`ValidationError`** est ajoutée au signal **`errors()`** du champ
- Une **`ValidationError`** est un objet avec trois propriétés principales :
  - **`kind`** : le type d'erreur, par exemple **`required`** ou **`minLength`**
  - **`field`** : le champ qui a provoqué l'erreur
  - **`message`** : le message lisible, optionnel, celui que vous avez fourni
- Certains validateurs enrichissent l'erreur : **`minLength`** ajoute par exemple une propriété **`minLength`** que vous pouvez réutiliser dans l'affichage
- Grosse différence avec les Reactive Forms : **`errors()`** est une **liste typée**, et non plus un objet dont il faut deviner les clés

La validation
Afficher les erreurs

- Dans le template, on récupère le **`FieldState`** avec **`@let`**, puis on itère sur **`errors()`**
- Le couple **`touched()`** et **`valid()`** permet de n'afficher les erreurs qu'après interaction de l'utilisateur

La validation
Afficher les erreurs

```
<input id="email" [formField]="accountForm.email" />

@let email = accountForm.email();
@if (email.touched() && !email.valid()) {
  <div class="invalid-feedback d-block">
    @for (error of email.errors(); track error.kind) {
      <div>{{ error.message }}</div>
    }
  </div>
}
```

Validateur personnalisé validate()

- Un validateur est simplement une **fonction** qui reçoit un `ChildFieldContext` et retourne `undefined` si tout va bien, ou une `ValidationError` sinon
- On l'attache à un champ avec `validate()`
- Le contexte est **typé** : `context.value()` a le type du champ. C'est un énorme progrès par rapport à `AbstractControl`

Validateur personnalisé validate()

```
// le mot de passe doit être assez robuste
validate(form.password, (context: ChildFieldContext<string>) =>
  isTooWeak(context.value())
    ? { kind: 'too-weak', message: 'Le mot de passe est trop faible' }
    : undefined
);
```

Validateur personnalisé
La validation croisée

- Pour comparer deux champs, on attache le validateur au **champ parent** et on lit les valeurs des sous-champs
- Plus besoin de remonter au `FormGroup` ni de fouiller dans `control.parent` comme en Reactive Forms
- L'erreur est alors portée par le formulaire lui-même, et se lit avec `accountForm().errors()`

Validateur personnalisé
La validation croisée

```
// le mot de passe doit être différent de l'email
validate(form, context => {
  const { email, password } = context.value();
  return email && password && email === password
    ? {
        kind: 'password-different-from-email',
        message: 'Le mot de passe ne doit pas être identique à l'email'
      }
    : undefined;
});
```

La validation asynchrone validateAsync()

- **`validateAsync()`** s'appuie sur une **`Resource`**, le pont entre le monde asynchrone et les signaux déjà vus dans le chapitre précédent
- **`params`** définit le signal déclencheur, **`factory`** crée la resource, **`onSuccess`** transforme le résultat en erreur ou en **`undefined`**, **`onError`** gère l'échec de l'appel
- Pendant l'appel le champ passe en **`pending()`** et **`valid()`** reste à **`false`**

La validation asynchrone validateAsync()

```
validateAsync(form.email, {
  params: (email: ChildFieldContext<string>) => email.value(),
  factory: (params: Signal<string | undefined>) =>
    resource({
      params,
      loader: async ({ params }) => await this.userService.isRegistered(params)
    }),
  onSuccess: response =>
    response.isRegistered
      ? { kind: 'email-already-registered', message: 'Email déjà utilisé' }
      : undefined,
  onError: () => ({ kind: 'email-check-failed', message: 'Vérification impossible' })
});
```

La validation asynchrone validateHttp()

- Quand la validation consiste simplement à appeler une **API**, **`validateHttp()`** évite d'écrire la resource à la main : vous ne fournissez que l'URL
- Sous le capot c'est une **`httpResource()`** : vos intercepteurs s'appliquent normalement
- L'option **`debounce`** évite d'envoyer une requête à chaque frappe. Nouveauté d'Angular 22, elle s'applique au niveau du **validateur**

La validation asynchrone validateHttp()

```
validateHttp(form.email, {
  request: (email: ChildFieldContext<string>) =>
    `/api/users/check?email=${email.value()}`,
  debounce: 400,
  onSuccess: response =>
    response.isRegistered
      ? { kind: 'email-already-registered', message: 'Email déjà utilisé' }
      : undefined,
  onError: () => ({ kind: 'email-check-failed', message: 'Vérification impossible' })
});
```